

C++知识总结

框架搭建时间：22.9.9，初始创作时间不知，大概是大一上吧。

作者：张知榆

学习经验：1.对于复杂的算法，带一个实例，耐心的用人脑去跑一遍代码，这样虽然麻烦，但是学习效果是最好的。

说明 1：感觉对 C 的掌握程度并不是自己预想那么好，得有个系统的框架总结一下，这学期应该来说也会有很多地方用到。搭建的框架可能不完善，在原来的基础上搭建的，所以应该会有很多需要更改删减、增加的。

说明 2：./基础 文件夹下面的目录结构就不要去改了，会破坏链接。

说明 2024.7.15：该框架搭建的思路，先是基础语法性的问题，然后是一些涉及库函数使用的问题，再然后是一些数据结构问题，然后是算法类型问题，最后是刷题心得。这几个板块之间关系可以理解为包含关系：语法<函数<数据结构<算法<心得。越往前语法成分越多，越往后算法思想越多，后者包含了前者。

文档规范：

1. 后面的代码在复习的时候会有意识的将其变成超链接的形式，以减缓代码冗杂使得文档太多了。2024.3.11
2. 关键代码模块将其放到方框里面，这样模块化处理，显得更有层次感。2024.7.30
3. 时间格式：年.月.日；AC 时间+拓展思考时间+归纳总结时间=该题总耗时。
4. 变体颜色：“矢车菊蓝”
5. 目录层级只要 2 级即可，对于内容比较多的再进一步划分下一级目录。级别太多了显得太乱了。

目录

第 1 章 语法	3
1.1 数据类型	3
1.2 数组	10
1.3 结构体	15
1.4 类和对象	16
1.5 文件	18
1.6 其他	19
第 2 章 函数	22
2.1 输入输出	22
2.2 字符串	25
2.2.1 基础知识	25
2.2.2 算法案例	29
2.3 子串	33
2.4 时间	33
2.5 STL 库	35
2.5.1 vector 容器	35
2.5.2 map 容器	37

2.5.3 set 容器	39
2.6 其他	44
第 3 章 数据结构	48
3.1 哈希表	48
3.2 双指针	51
3.2.1 滑动窗口	51
3.3 链表	54
3.4 栈和队列	59
3.4.1 表达式求值	60
3.4.2 单调栈、单调队列	62
3.5 二叉树	67
3.5.1 二叉树创建	69
3.5.2 二叉树遍历	72
3.5.3 二叉树序列化	82
3.5.4 应用案例	84
3.6 图	91
3.6.1 拓扑排序	91
3.6.2 最小生成树	92
3.6.3 最短路径问题	93
3.7 其他	94
3.7.1 并查集	94
3.7.2 堆	95
第 4 章 算法	97
4.1 快速幂、乘	97
4.2 搜索算法	98
4.3 递归、回溯	100
4.3.1 理论基础-递归	100
4.3.2 基础理论-回溯	103
4.4 贪心算法	107
4.5 动态规划	112
4.5.1 背包问题	115
4.5.2 子序列问题	123
4.5.3 打家劫舍和股票问题	128
4.5.4 其他案例	132
4.6 组合优化问题	134
4.7 排序算法	134
4.7.1 归并排序	134
4.8 其他算法	135
第 5 章 心得	136
5.1 报错	136

第 1 章 语法

1.1 数据类型

一、基本数据类型

1. int 类型

字节数量：4 字节， 2^{32} ，即 -2,147,483,648 到 2,147,483,647。约正负 20 亿。

unsigned int 也通常占 4 个字节（32 位系统和 64 位系统都是）。

2. bool 类型

作用：只返回真假。占一字节。常用于判断。通常在使用 sort 函数，或者定义结构体需要重构比较比较算子的时候会定义 bool 函数。

3. char 类型

定义：字符类型，占一字节。

知识点：理解 char *, char **, char a[], char *a[]

链接：[深入理解 char *, char **, char a\[\], char *a\[\] - Z--Y - 博客园 \(cnblogs.com\)](#)

时间：2024.7.31；40min

（1）字符数组

例子：定义。

定义字符数组

```
char str[10] = {"hello"}; printf("%s",str);
char *s;
s = "China";
```

（2）char* Vs char a[]

例子 1：两者的含义。

说明：a 是字符串的首地址；s 是指针，保存字符串的地址（其实首地址）。因此可以 s = a；但是不能 a = s。

```
char *s;
char a[];
printf("%c",*a);
```

例子 2：两者内存存储。

指针的地址 Vs 指针所指向的地址。

```
char a[10] ;//sizeof(a) = 10
char *s;//sizeof(s) = 4
printf("%p",s);// s 所指向的地址。
printf("%p",&s);//s 这个指针自己所在的地址。
```

（3）char ** 与 char *a[]

例子：两者含义。

说明：数组名 a 代表数组元素内存单元的首地址，即 a = &a[0] = 0012FF38;所以可以把 a 赋值给 s。

```
char *a[] = {"China","French","America","German"}; //sizeof(a) = 16;
```

说明：[] 的优先级高于*所以 a 先和[]结合，故 a 是一个数组，数组的元素是指针。

```
char **s;//二级指针，即指针的指针。
```

```
char **s = a;
```

注意：不能 `a = s`，因为 `a` 是常量。`s` 是指针变量。

二、拓展数据类型

1. const 类型

(1) 对于指针变量而言，若 `const` 在*的右边，表示指针所指的地址不能修改

(2) 若 `const` 在*左边，表示指针指的值不能修改

(3) 如果*左右两侧都有，表示指针所指的地址和值都不能修改

例子：字符串给字符指针赋值时用 `const`

如果不加 `const` 则会报错说： `deprecated conversion from const string to 'char'`，即是说不能这样将字符串常量赋给变量 `char *`。

```
const char *oprnd[7]={">><<<>>",
    ">><<<>>", ">>>><>>", ">>>><>>", "<<<<<=0",
    ">>>>0>>", "<<<<<0="};
```

2. auto 类型

作用：随机指定数据类型。在 `c` 里面一般不用。

`a=mp.begin();` //auto 类型，由系统根据赋初值表达式自动推出要定义变量的类型

3. unsigned int 类型

字节：两个字节

Unsigned int、int 都是两个字节

Float 4 个字节

Double 8 个字节

存储方式：计算机中，数据以二进制反码的形式存储

❖ 案例：[..\基础\unsigned int.cpp](#)

知识点：原码、反码、补码。

正数：原码、反码、补码三码合一。

负数：原码转反码，反码再转补码。

计算机中存储补码。

关键代码片段：

```
unsigned int a=1;
int b=-1;
if(a>b)
    cout<<"bigger one: "<<a;
else
    cout<<"bigger one:"<<b;
//输出: bigger one:-1
```

//自动代码转换机制，在进行比较时，`int` 的 `b` 会被视为无符号数，所以 `int` 的 `-1` 对应得二进制码会被视为 `unsigned int` 来进行输出，即 `1111 1111 1111 1111` 对应的无符号数 `4294967295`

补充：负数的原码转补码，补码转原码。

4. string 类型

❖ 案例：将标题首字母大写。

时间：2024.3.11

链接：[字符串变形_牛客题霸_牛客网 \(nowcoder.com\)](https://www.nowcoder.com)

描述：单词句子切割，大小写转换，颠倒顺序。

思路：s1.切割。s2.大小写反转。s3.合并。

关键代码：

细节：处理最后一个单词单独加了一个 `i==n` 的判断条件。

```
for(int i=0;i<=n;i++){
    if(i==n||s[i]==' '){
        str.push_back(s.substr(first,i-first));
        first=i+1;
    }
}
```

大小写反转：

方法一：采用 ASCII 码

```
if('a'<=strs[i][j]&&strs[i][j]<='z'){
    strs[i][j]-=32;}
```

方法二：采用 `<ctype>` 库, `islower(),tolower();isupper(),toupper()` 等函数

5. enum 枚举类型

定义：枚举类型。在定义一些常量的时候会用到。

说明：可以手动赋初值。如果没有手动赋初值，那么最开始的 `enum` 会从 0 开始默认赋初值，后面的数据会从前面的数值开始递增 1。

```
enum colors{
    red,green,yellow=5,blue
};
//这里，red=0,green=1,blue=6
```

三、类型转换

1. char 转 int 型

按照 ASCII 码来转换，就算是数字也是这样的。

知识点：ASCII 码

例子：将数字字符转换为 `int` 时，是按照 ASCII 码来的。

说明：即是 '0' -48， '9' -57，这里会将字符串最后一位给到变量 `a`

```
char ch='18234';
int a=ch;//按照最后一位'4'来算的。
cout<<"转换之后："<<a<<endl;//返回：52
cout<<"减法："<<ch-'0'<<endl;//返回：4 ，即是 52-48=4
```

四、运算符

1. 位运算符

位运算符有：&(按位与)、|(按位或)、^(按位异或)、~(按位取反)。

优先级从高到低，依次为~、&、^、|

1) 按位与操作

理解：两个都是 1 才是 1。

运算规则：0&0=0; 0&1=0; 1&0=0; 1&1=1

例子：10&9： 0000 1010 & 0000 1001 = 0000 1000 = 8

负数按补码形式参加按位与运算

“与运算”的特殊用途

(1) 清零。如果想将一个单元清零，即使其全部二进制位为 0，只要与一个各位都为零的数值相与，结果为零。

(2) 取一个数中指定位

方法：找一个数，对应 X 要取的位，该数的对应位为 1，其余位为零，此数与 X 进行“与运算”可以得到 X 中的指定位。

例：设 X=10101110，

取 X 的低 4 位，用 $X \& 0000\ 1111 = 0000\ 1110$ 即可得到；

还可用来取 X 的 2、4、6 位。

2) 按位或运算符 (|)

理解：有一个是 1 就是 1。

运算规则：0|0=0; 0|1=1; 1|0=1; 1|1=1;

详述：参加运算的两个对象，按二进制位进行“或”运算。即：参加运算的两个对象只要有一个为 1，其值为 1。

例如：3|5 即 $0000\ 0011 | 0000\ 0101 = 0000\ 0111$ 因此，3|5 的值得 7。

补充：另，负数按补码形式参加按位或运算。

“或运算”特殊作用：

常用来对一个数据的某些位设置为 1。

方法：找到一个数，对应 X 要置 1 的位，该数的对应位为 1，其余位为零。此数与 X 相或可使 X 中的某些位置 1。

例：将 X=10100000 的低 4 位置 1，用 $X | 0000\ 1111 = 1010\ 1111$ 即可得到。

3) 异或运算符 (^)

理解：不同的是 1。

运算规则：0^0=0; 0^1=1; 1^0=1; 1^1=0;

详述：参加运算的两个数据，按二进制位进行“异或”运算。即：参加运算的两个对象，如果两个相应位为“异”（值不同），则该位结果为 1，否则为 0。

例如：10^-9 即 $0000\ 1010 \wedge 1111\ 0111 = 1111\ 1101$ (补码) 原码即为 1000 0011 即 $10 \wedge -9 = -3$ 。

“异或运算”的特殊作用：

(1) 使特定位翻转：找一个数，对应 X 要翻转的各位，该数的对应位为 1，其余位为零，此数与 X 对应位异或即可。

例：X=10101110，使 X 低 4 位翻转，用 $X \wedge 0000\ 1111 = 1010\ 0001$ 即可得到。

(2) 与 0 相异或，保留原值， $X \wedge 0000\ 0000 = 1010\ 1110$ 。

(3) 交换 a 和 b ？？

方法一 方法二

1. $a = a \wedge b$ 1. $a = a - b$

2. $b = b \wedge a$ 2. $b = a + b$

3. $a = a \wedge b$ 3. $a = b - a$

4) 取反运算符 (~)

运算规则： $\sim 1 = 0$ ； $\sim 0 = 1$ ；参加运算的一个数据，按二进制位进行“取反”运算。即：对一个二进制数按位取反，即将 0 变 1，1 变 0。

取反运算特殊用法：

(1) 使一个数的最低位为零，可以表示为： $a \& \sim 1$ 。

解析： ~ 1 的值为 111111111111110，再按“与”运算，最低位一定为 0。因为“~”运算符的优先级比算术运算符、关系运算符、逻辑运算符和其他运算符都高。

~ 是按位取反运算符，它会将操作数的每一位进行取反操作，即 0 变为 1，1 变为 0。

! 是逻辑非运算符，它用于对布尔值进行取反操作。在 C++ 中，非零值被视为 true，零值被视为 false。

```
unsigned int a=0xFFFFFFFF;
cout<<~a<<endl; //0
cout<<!a<<endl; //0
```

5) 左移运算符 (<<)

运算规则：将一个运算对象的各二进制位全部左移若干位（左边的二进制位丢弃，右边补 0）。

例： $a = a \ll 2$ 将 a 的二进制位左移 2 位，右补 0，

应用：

(1) 左移 1 位后 $a = a * 2$;

分析：若左移时舍弃的高位不包含 1，则每左移一位，相当于该数乘以 2。

6) 右移运算符 (>>)

运算规则：将一个数的各二进制位全部右移若干位，正数左补 0，负数左补 1，右边丢弃。

例如： $a = a \gg 2$ 将 a 的二进制位右移 2 位，

左补 0 or 补 1 得看被移数是正还是负。

应用：

操作数每右移一位，相当于该数除以 2。

`expression1=expression1>>expression2` : `>>` 运算符把 `expression1` 的所有位向右移 `expression2` 指定的位数。`expression1` 的符号位被用来填充右移后左边空出来的位。向右移出的位被丢弃。

例如，下面的代码被求值后，`temp` 的值是 -4:

```
var temp = -14 >> 2
```

-14（即二进制的 11110010）右移两位等于 -4（即二进制的 11111100）。

结论：右移运算符等价于整除 2^n 。（默认向下取整）

7) 无符号右移运算符 (>>>)

运算规则：`>>>` 运算符把 `expression1` 的各个位向右移 `expression2` 指定的位数。右移后左边空出的位用零来填充。移出右边的位被丢弃。

8) 复合赋值运算符

位运算符与赋值运算符结合，组成新的复合赋值运算符，它们是：

位运算符 例子 相当于

`&=` 例：`a &= b` 相当于 `a=a & b`

`|=` 例：`a |= b` 相当于 `a=a | b`

`>>=` 例：`a >>= b` 相当于 `a=a >> b`

`<<=` 例：`a <<= b` 相当于 `a=a << b`

`^=` 例：`a ^= b` 相当于 `a=a ^ b`

运算规则：和前面讲的复合赋值运算符的运算规则相似。

知识点：不同长度数据位运算

运算规则：如果两个不同长度的数据进行位运算时，系统会将二者按右端对齐，然后进行位运算。

❖ 以“与”运算为例说明如下：我们知道在 C 语言中 `long` 型占 4 个字节，`int` 型占 2 个字节，如果一个 `long` 型数据与一个 `int` 型数据进行“与”运算，右端对齐后，左边不足的位依下面三种情况补足：

- (1) 如果整型数据为正数，左边补 16 个 0。
- (2) 如果整型数据为负数，左边补 16 个 1。
- (3) 如果整形数据为无符号数，左边也补 16 个 0。

❖ 如：`long a=123;int b=1;计算 a & b。`

`a&b`

`a` 0000 0000 0111 1011

`b` 0000 0000 0000 0001

`=` 0000 0000 0000 0001

❖ 如：`long a=123;int b=-1;计算 a & b。`

`a&b`

```
a  0000 0000 0111 1011
b  1111 1111 1111 1111
=  0000 0000 0111 1011
```

❖ 如: `long a=123; unsigned int b=1;` 计算 `a & b`。

`a&b`

```
a  0000 0000 0111 1011
b  0000 0000 0000 0001
=  0000 0000 0000 0001
```

2. 运算符优先级

解释: “>>” 优先级高于 “&&” 高于 “=”

`while((ch=getchar())&&ch!='@')` , 其中 `(ch=getchar())` 外面的括号丢掉之后就会出问题

说明: `ch=(getchar())&&ch!='@'` 这样的话等效于 `while(ch=getchar())&&ch!='@'` 不加这个括号。

❖ 案例: [..\基础\优先级.cpp](#)

说明: “>>” > “&&” > “=”

问题: 判断回文。

方案: 通过栈+队列来判断。

说明: 栈从后面访问, 队列从前面开始访问。

3. sizeof

说明: `sizeof` 是一个运算符。

细节:

1. 指针大小与系统位数的关系。

1) 32 位系统: 32 位系统的地址总线宽度为 32 位, 能够表示的最大地址范围是 2^{32} 个不同的地址, 所以指针需要 4 个字节来存储地址信息。

2) 64 位系统: 64 位系统的地址总线宽度为 64 位, 能够表示的最大地址范围是 2^{64} 个不同的地址, 所以指针需要 8 个字节来存储地址信息。

综上所述, 在不同位数的系统中, `sizeof(&a)` 的结果会有所不同, 分别为 4 字节 (32 位系统) 和 8 字节 (64 位系统)。

2. 当 `sizeof()` 里面是数组首指针到的时候, 返回整个数组字节的数量。

4. 运算符重载

说明: 对于结构体而言, 重新定义已有的计算符号, 使其能无缝衔接 结构体符号运算。

例子: 结构体排序。

重载小于运算符, 定义排序规则

```
struct MyStruct {
    int value;
    bool operator<(MyStruct& other) {
        return value < other.value;
    }
}
```

```
}  
};
```

❖ 案例: [..\基础\STL 库\map-运算符重载.cpp](#)

关键代码:

结合 map 容器自动排序功能, 定义结构体的比较方法。

```
struct Student{  
    int ID;  
    string name;  
    Student(){};  
    Student(int n,string name0):ID(n),name(name0){};  
    bool operator<(const Student &stu) const{ //如果 ID 相等, 则按照名字排序。  
        if(ID != stu.ID)  
            return ID < stu.ID;  
        return name < stu.name;  
    }  
};
```

1.2 数组

时间: 2024.12.17

- **数组定义**: 数组是存放在连续内存空间上的相同类型数据的集合。数组可以通过下标索引的方式获取到下标对应的数据。
- **数组特性**: 数组下标从 0 开始。数组内存空间的地址是连续的。
- **数组操作**: 删除或增添元素时, 可能需要移动其他元素的地址。删除下标为 3 的元素时, 需要对下标为 3 的元素后面的所有元素进行移动操作。
- **C++中的数组与向量**: C++中的数组元素不能被删除, 只能被覆盖。vector 的底层实现是数组, vector 是容器, 不是数组。
- **二维数组**: 不同编程语言的内存管理不同, C++中二维数组是连续分布的。
- **内存地址**: C++中可以通过指针和地址运算查看数组元素的内存地址。

1. 区分字符数组和字符串

- 字符串: 其指针是一个变量, 用于存放字符串的首地址, 其数据存放在一块连续的内存空间, 最后以'\0'结束。如 char *ps="nihao";
- 字符数组: 数组形式, 如: char st[]="nihao";

例子:

判断一个字符是数字: '0'<=a[i]&& a[i]<='9'

判断一个字符是英文字母: ('a'<=a[i]&& a[i]<='z')||('A'<=a[i]&& a[i]<='Z')

2. new—delete

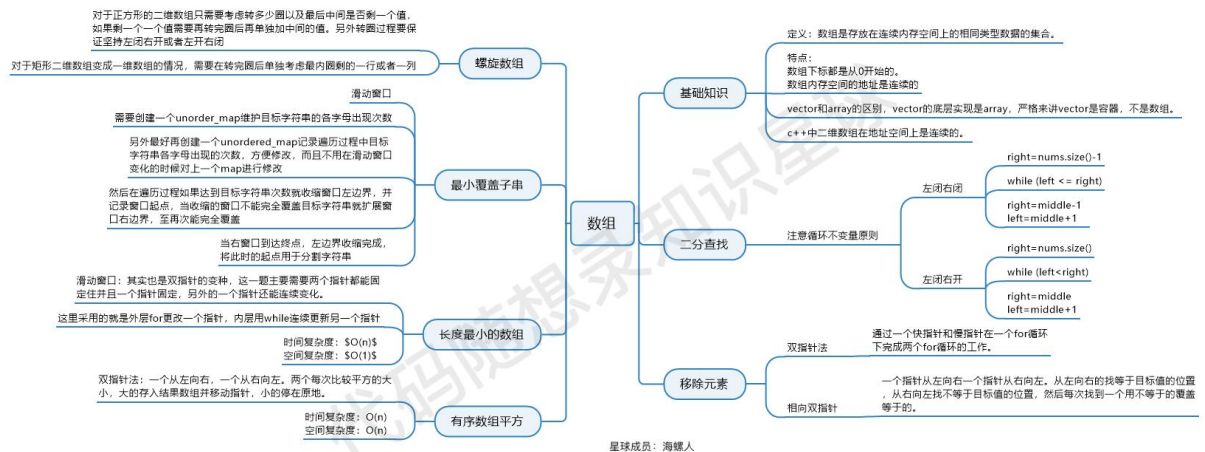
说明: new 之后要 delete。

string 对象实例化赋值

string* a=new string("father!");

```
cout<<*a<<endl;
delete a;//细节，不用之后要删除
```

数组部分算法概览：



一、二分法

描述：二分查找。

二、双指针

描述：用两个指针同时指向一个数组。同时遍历操作数组。

三、滑动窗口

描述：遍历数组，一个指针主遍历，一个指针逐渐搜索位置。

四、模拟行为

类型特点：需要通过调整数组角标变化，以按照预期规律输出数组元素。

结题要点：需要十分熟悉数组角标的和元素对应关系。特别是需要注意数组角标越界问题。

五、前缀和

描述：前缀和，即累加。通过 $acc[i]-acc[j]$ 来实现 $sum(num[i\sim j])$ ，对于需要连续求和的问题，这样计算效率更高。

❖ 案例：除了自身以外数组的乘积

链接：[238. 除了自身以外数组的乘积 - 力扣 \(LeetCode\)](#)

时间：2026.4.18 20min

思路：使用前缀和、后缀和类似的思想，前缀积、后缀积。

六、脚标控制

描述：需要控制脚标来实现数组的多样化遍历。

❖ 案例：螺旋矩阵

链接：[54. 螺旋矩阵 - 力扣 \(LeetCode\)](#)

时间：2026.4.21 40min（一次性解决）；

描述：按照螺旋的走向遍历数组。

思路：通过一个 `directions` 数组来控制移动方向，然后根据移动状态来修改移动方向，可以复用 `matrix` 数组来做访问标记。

关键代码：

对脚标进行控制。

```
vector<int> results;
vector<bool> directions(4,false);// 0-右, 1-下, 2-左, 3-上。表示当前走向。每次只有一个为true。
directions[0]=true;//初始的方向是向右
while(cnt>0){
    results.push_back(matrix[i][j]);
    matrix[i][j]=flag;
    cnt--;
    if(directions[0]){//当前方向是朝右，下一个朝向只能是下。
        if(j==m-1||matrix[i][j+1]==flag){// 方向从右到下
            i++;
            directions[0]=false;
            directions[1]=true;// 改变朝向
        }else{// 没有触发边界条件，正常向右
            j++;
        }
    }else if(directions[1]){
        if(i==n-1||matrix[i+1][j]==flag){
            j--;
            directions[1]=false;
            directions[2]=true;
        }else{
            i++;
        }
    }else if(directions[2]){
        if(j==0||matrix[i][j-1]==flag){
            i--;
            directions[2]=false;
            directions[3]=true;
        }else{
            j--;
        }
    }else if(directions[3]){
        if(i==0||matrix[i-1][j]==flag){
```

```

        j++;
        directions[3]=false;
        directions[0]=true;
    }else{
        i--;
    }
}

}

```

❖ 案例：搜索二维矩阵 II

链接：[240. 搜索二维矩阵 II - 力扣 \(LeetCode\)](#)

时间：2026.4.24 40min（一次性）

思路启发：在稿纸上模拟一下遍历即可。

思路：通过一个 directions 数组来控制移动方向，然后根据移动状态来修改移动方向。

细节：结束条件，方向从下转左的时候，如果 j 已经为 0，说明没有找到元素。

```
vector<bool> directions(3,false);//0-右，1-下，2-左
```

```
directions[0]=true;//默认先往右遍历
```

```
// 有三个方向，左右下可以走。
```

```
while(i<=m-1||j<=n-1){
    if(matrix[i][j]==target) return true;
    if(directions[0]){
        if(j<n-1&&matrix[i][j+1]<=target){
            j++;
        }else{
            directions[0]=false;
            directions[1]=true;
        }
    }else if(directions[1]){
        if(i<m-1&&matrix[i+1][j]<=target){
            i++;
        }else{
            directions[1]=false;
            directions[2]=true;
        }
    }else if(directions[2]){
        if(j>0){
            j--;
        }else if(j==0){
            return false;
        }
        directions[2]=false;
        directions[1]=true;
    }
}
```

```
}  
}
```

七、其他

❖ 案例：旋转数组

链接：[旋转数组_牛客题霸_牛客网 \(nowcoder.com\)](#)

时间：2024.7.14；20+10+20=50min

题目概要：把数组往后平移 m 。

掌握：**数组旋转的三逆序法**。

方法一：常规平移

S1.将后 m 个元素暂存到一个数组。

S2.将前 $n-m$ 个元素后移 m 个位置。

S3.最后将后暂存的 m 元素放到前 m 个位置。

方法二：逆序三次

S1.先将整个数组逆序。

S2.将前 m 个元素逆序。

S3.将后 $n-m$ 个元素逆序。

关键代码：

细节：采用 `<algorithm>` 头文件的 `reverse` 函数来实现逆序。（快一些）

`m=m%n;`//考虑到 m 大于 n 的情况

`reverse(a.begin(),a.end());`//逆序整个数组。

`reverse(a.begin(),a.begin()+m);`//逆序前 m 个元素

`reverse(a.begin()+m,a.end());`//逆序后 $n-m$ 个元素。

❖ 案例：合并区间

时间：2026.4.15 15min（完成）

链接：[56. 合并区间 - 力扣 \(LeetCode\)](#)

思路：对区间的左边界排个序即可。

知识点：sort 排序，定义一个 compare。

❖ 矩阵置零

时间：2026.4.20 10min；

链接：[73. 矩阵置零 - 力扣 \(LeetCode\)](#)

方法 1-直白：记录下 0 所在的点，分别将其所在行列置 0。

❖ 旋转图像

时间：2026.4.22 10min（一次性）

链接：[48. 旋转图像 - 力扣 \(LeetCode\)](#)

方法 1：直白：定义一个新的数组作为中转即可。

方法 2：脚标控制的方法，还没细看。

1.3 结构体

重点：

1. 内存偏移：是最大单个内存的整数倍，不足的会自动补齐。称为内存偏移。
2. 只能在定义的时候赋值，而且需要顺序赋值，定义之后只支持单个修改。
3. 默认构造函数。

❖ 案例：[..\基础\结构体.cpp](#)

1. 结构体内存：是最大单个内存的整数倍，不足的会自动补齐。称为内存偏移

2. sizeof()

//表明：虽然是指针类型只占两个字节，但是 sizeof 会返回其指向数据类型的字节数

//表明：当 sizeof() 里面是数组首指针到的时候，返回整个数组字节的数量

1. 结构体创建、构造、析构

案例：[【模板】循环队列_牛客题霸_牛客网 \(nowcoder.com\)](#)

关键代码：

知识点：结构体的创建、构造、析构。

```
struct cirDeque {
    int length;
    int front, rear;
    int *deque;
    cirDeque(int length_) : length(length_), front(0), rear(0) {
        deque = new int[length_];
    }
    ~cirDeque() {
        delete[] deque;
    }
    void push(int number) {
        if ((rear + 1) % length == front) { // 判断队列是否已满
            cout << "full" << endl;
        } else {
            deque[rear] = number;
            rear = (rear + 1) % length;
        }
    }
} //略去了一些不必要的构造内容
```

1. 常用结构体

1) pair

说明: `pair<typename T1,typename T2>` 这是一个模板结构体, 用来定义一个 `pair`
如果使用得比较频繁:

```
using position=pair<int,int>;
```

创建和查看:

```
position pos(1,2);  
pos.first; pos.second;
```

创建数组并赋初值:

```
position direction[4]={0,1},{0,-1},{1,0},{-1,0}};// 知识点  
访问 first 和 second: pos.first,pos.second
```

1.4 类和对象

一、基本概念

定义: 略。

❖ 案例: 类和对象使用的具体案例。

描述: stock 公司购买股票的案例。

主函数: [..\知识点测试.大一下\类\main.cpp](#)

细节:

```
#include"stock00.h"  
#include"stock00.cpp"
```

执行文件: [..\知识点测试.大一下\类\stock00.cpp](#)

细节: `#include"stock00.h"`

头文件: [..\知识点测试.大一下\类\stock00.h](#)

细节:

```
#ifndef STOCK00_H_  
#define STOCK00_H_  
内容  
#endif
```

二、派生

定义: 派生。

案例: [..\基础\Triangle.cpp](#)

描述: 点、三角形。由点派生出三角形。

三、继承

定义: 继承。

关键概念: 构造函数、虚构造函数。构造函数重载。

1. 虚函数继承

❖ 案例：[..\基础\继承\继承-点、圈、正方形.cpp](#)

描述：多态管理是以对象为主。对虚函数要刷新理解：只有声明了 `virtual` 的函数，才能起到虚函数的作用。

注意：虚函数的声明必须要完整，父类析构函数最好是虚函数。

关键代码：引入了虚函数，进行多态管理。

```
virtual void Position(Point &) {};
```

//卡点：就算没有内容也要定义 “{}”。

```
virtual ~Point() {};
```

//卡点：析构函数一般要定义为虚的，同时后面要加定义的 “{}” 表示定义完整。

❖ 案例二、brass 管理

● 主函数部分：[..\基础\继承\Brass\main.cpp](#)

● 头文件：[..\基础\继承\Brass\brass.h](#)

//卡点细节：这个虚析构函数后面必须要有 {} 才是完整的定义

//细节：在基类里面声明了虚方法之后，在派生类可以不用在声明，也可以声明

定义语法：

```
#ifndef BRASS_H_
```

```
#define BRASS_H_
```

```
内容
```

```
#endif
```

● 执行部分：[..\基础\继承\Brass\brass.cpp](#)

//目标：1.理解作用域算符的意义，区分用哪个类里面的方法

//2.公有继承时，不能直接访问基类的私有数据成员

2. 虚基类继承

意义：解决多层继承出现的歧义。

Key：先父母，再客人，后自己。

案例：[..\基础\继承\虚基类继承.cpp](#)

描述：多层继承，需要用到虚基类。D1 继承 Base，D2 继承 Base，D12 继承了 D1 和 D2，所以 D1 和 D2 在继承 Base 的时候要用到虚基类继承。

关键代码：

```
class D1:virtual public Base{略}
```

```
class D2:virtual public Base{略}
```

```
class D12:public D1,public D2{略}
```

```
void Display(){
```

```
    Base::Display(); //通过域作用符进行调用。
```

```
    D1::Display();
```

```
    D2::Display();
```

```
}}
```

1.5 文件

说明：文件读入读出都是先对于内存而言，即读入到内存，读出到文件。

一、fstream

全称：file stream,文件流。作用：将文件内容变成流对象。

用法：#include<fstream>

知识点：istreambuf_iterator<char>(file)。

全称：input stream buffer iterator,输入流缓存迭代。

细节：(istreambuf_iterator<char>(file)),外层还有一个括号。原因不知。

istreambuf_iterator<char>(file) 是一个用于迭代输入流中每个字符的迭代器。

istreambuf_iterator<char>()表示默认构造，代表输入流的结束（也就是文件结尾 EOF）。

string str((istreambuf_iterator<char>(file)), istreambuf_iterator<char>()); // 读取整个文件内容

1. 文件读入读出

❖ 例子：文件读入到内存。一般可用 fstream、ifstream。前者设置模式 ios::in。

文件内容："My eyes only for you!"

```
#include<ifstream>
```

```
ifstream fin(filename);
```

(1) fin>>b; //按照这样输出的话，会按照空格为分字符，遇到空格或者换行符则停止读取，所以最后只读出了一个“My”。

(2) fin.get(ch)

```
char ch;
```

```
while(fin.get(ch))
```

```
    cout<<ch; //按照这种读取方式可以一次性全部读完
```

常用步骤：1.创建文件对象并打开文件，2.判断成功打开，3.使用文件，4.关闭文件流。

fstream file; //创建文件对象并打开文件。

```
file.open(filename);
```

```
if(!file.is_open()){cout<<"No!";} //习惯性判断一下是否成功打开文件
```

```
string line;
```

```
while(getline(file,line)){//使用文件
```

```
    cout<<line<<endl;
```

```
}
```

```
file.close(); //关闭文件，养成关闭文件的习惯。
```

❖ 例子：文件从内存读出到硬盘。一般可用 fstream、ofstream。前者设置模式 ios::out。

一般步骤：1.创建文件对象，2.读入文件，3.关闭文件流。

```
#include<ofstream>
ofstream fout(filename);
fout<<"My eyes only for you!";
fout.close();
```

2. 命令行处理技术

理解：给主函数 `main()` 传递文件用的

案例： [..\基础\命令行处理技术.cpp](#)

补充： `cerr` 和 `cout`

`cerr` 不经过缓冲区输出，一般用来输出报错。

3. ios

知点： `ios::app`

一般可以作为参数控制 `ofstream` 和 `ifstream`

❖ 案例： [..\基础\ios.cpp](#)

关键代码：

```
fout.open(file,ios::app);
```

说明：一般打开文件都是覆盖性质的，写入新的内容会覆盖源文件。但是这里设置 `ios::app`，那么就变成续写性质的，可以续写。

知点： `get()` 与 `getline()` 函数

案例： [..\基础\getline.cpp](#)

一般有 `string` 类型的，那么一般用 `getline`，是 `string.h` 里面的函数。

1.6 其他

说明：这里是还没想好怎么分类的基础知识部分。

一、宏定义

引：常定义 `const`

1. 简单的常量宏定义

语法： `#define` 标识符 替换文本

功能：在编译预处理阶段，编译器会将代码中所有出现该标识符的地方替换为对应的替换文本。

例子： `#define PI 3.14159`

2. 带参数的宏定义

语法： `#define` 标识符(参数列表) 替换文本

功能：类似于函数调用，但它是在预处理阶段进行简单的文本替换，而不是像函数那样进行

参数传递和函数调用的开销。

3. 宏定义中的字符串化和连接操作

字符串化 (#)：在宏定义中，使用 # 可以将参数转换为字符串。

连接操作 (##)：使用 ## 可以将两个标识符连接成一个新的标识符。

#define STRINGIFY(x) #x 定义了一个宏 STRINGIFY，它将参数 x 转换为字符串。在 printf 中调用 STRINGIFY(Hello) 时，会将其替换为 "Hello"（字符串）。

#define CONCAT(x, y) x##y 定义了一个宏 CONCAT，它将两个参数连接成一个新的标识符。在 main 函数中，CONCAT(num, 3) 会被替换为 num3，并将 num1 和 num2 的和赋值给 num3。

// 字符串化示例

```
#define STRINGIFY(x) #x
```

// 连接操作示例

```
#define CONCAT(x, y) x##y
```

```
int main() {  
    // 字符串化  
    printf("字符串化结果: %s\n", STRINGIFY(Hello));  
    // 连接操作  
    int num1 = 10;  
    int num2 = 20;  
    int CONCAT(num, 3) = num1 + num2;  
    printf("连接后的变量值: %d\n", num3);  
    return 0;  
}
```

4. 宏定义的注意事项

- 1) 宏展开问题：宏只是简单的文本替换，可能会导致一些意外的结果，尤其是在带参数的宏中。

说明：如下定义如果加一个括号，#define MULTIPLY(a, b) (a) * (b)，那么即可返回预期的结果 20。

```
#define MULTIPLY(a, b) a * b
```

```
int main() {  
    int result = MULTIPLY(2 + 3, 4);  
    // 宏展开后为 2 + 3 * 4，结果为 14，而不是预期的 20  
    printf("结果: %d\n", result);  
    return 0;  
}
```

- 2) 作用域：宏定义的作用域从定义处开始，到文件结束或使用 #undef 取消定义为止。

描述：在文件结束前使用 #undef MAX 取消宏定义之后，就不能再使用了。

```
#define MAX 100
```

```
int main() {  
    printf("MAX 的值是 %d\n", MAX);  
    #undef MAX
```

```
// 此时再使用 MAX 会报错
// printf("MAX 的值是 %d\n", MAX);
return 0;
}
```

❖ 案例: [..\基础\宏定义关键字.cpp](#)

描述: 重定义关键字。

时间: 22.11.16

关键代码:

```
#define sb(i,a,n) for(int i=a;i<=n;i++)
#define nmb(i,n) while(i++!=n)
#define bb(a,b) if(a>b)
```

知识点:

- 1.bool 类型的变量全局定义时, 默认初始化为 false
- 2.数字之间的&, |运算是二进制之间的运算。
- 3.#define 可以实现关键词的重定义, 如下的 for 和 while 等等

二、其他

1. lambda 表达式

作用: 轻量级匿名函数, 常定义短小的回调函数或比较函数等。

语法: [capture-list] (parameter-list) -> return-type { function-body }

capture-list: 捕获列表, 用于指定哪些外部变量在 lambda 表达式中可以使用。比如 `&` 表示捕获所有外部变量并以引用方式使用, `=` 表示捕获所有外部变量并以值方式使用。

parameter-list: 参数列表, 与普通函数的参数列表相同。

return-type: 返回类型。

function-body: 函数体。

例子:

```
sort(events.begin(), events.end(), [](Event& a, Event& b) {
    if (a.time == b.time) return a.type < b.type;
    return a.time < b.time;
});
```

2. 内存

栈区和堆区

区别: 栈区由系统空间分配由系统进行, 堆区由程序员进行。

栈区: 用来存放局部变量, 形参之类, 未进行实例化的引用申请的变量。

❖ 经验 2024.9.1: 变量名不能命名为关键字, 否则报错。

3. 指针

说明：指针。创建的时候一定要初始化，不然野指针可能报错。

第 2 章 函数

2.1 输入输出

作用：输入输出控制。

一、iostream

英文名：input and output stream

1. cout 输出

❖ 案例：[..\基础\库函数\iostream-cout.cpp](#)

描述：设置输出精度

注意：如果是整型的数据，是不能够设置精度的

1) precision()

作用：设置 double 型的 cout 小数点后保留几位，如果后面小数点不够，不会自动补零，同时，该设置会持续作用到后面的 cout，直到下一次的更改。

细节：知道 precision() 的默认精度是 6。

❖ 案例一：[..\基础\库函数\iostream-cout-1.cpp](#)

关键代码：

```
cout.precision(6);  
cout<<a<<endl;
```

2. cin 输入

说明：标准输入流对象。输入的数据是保存在输入 I/O 缓存区中的。不吃空格。

基础用法：可以用于条件判断语句。

时间：2024.9.16, 20min

cin>>num; 返回 cin 对象本身，输入数字，可用于条件判断语句中。

cin>>str; 输入字符串，持续到空格之后的部分。

getline(cin, str); 输入字符串，可以读入空格，不读入换行符。

1) sync() 函数，clear() 函数。

作用：sync() 清除缓存区，clear() 清除错误。

❖ 案例：[..\基础\库函数\iostream-cin.cpp](#)

关键代码：

```
cin>>input; // 若输入的是字符，如“aa”，则会显示输出错误，同时错误会被留在缓存区里面  
cout<<"第一个数据: "<<input<<endl;  
cin.clear(); // 清除该错误  
cin.sync(); // 清除缓存区里面的错误字符串
```

//如果不清除的话，后面将不能继续输入

积累：若输入不满足规则，怎样重复输入。

2) 理解输入 I/O 缓存区

函数：fail()函数、eof()函数

理解：输入的数据是保存在输入 I/O 缓存区中的，清除错误字符后用 cin 可以继续流入下一个变量。

❖ 案例一： [..\基础\库函数\iostream-cin-io.cpp](#)

说明：clear()函数用法，清除错误的那一个输入，sync()清空缓存区。

❖ 案例二：

知识点：cin.get(a,length)，其中 a 必须是 char 类型数组，length 是长度。特点：cin.get()遇到空格会中断。

对比：cin.getline(a,100,'n')是专门为 string 类型设计的，遇到空格也会读入。可以手动设定终结符，这里设置终结符为'n'

补充 1：cin.peek() 返回当前观测指针的值，但是指针不会向后偏移。

补充 2：cin.get()返回当前观测指针下一个值，观测指针会往后偏移。

例子：用换行符作为循环的结束条件。使用 peek() Vs get()

公共部分代码：

```
vector<vector<int>> > nums;
```

```
int left,right;
```

```
while(true){
    cin>>left>>right;
    nums.push_back(vector<int>({left,right}));
    if (cin.get() == '\n') {
        break;
    }
}
```

```
while(cin.peek()!='\n'){
    cin>>left>>right;
    nums.push_back(vector<int>({left,right}));
}
```

3) fail() 函数

作用：判断是否输入格式正确

❖ 案例：输入失败处理方案。

```
cin>>choice;
```

```
if(cin.fail())//fail()函数，输入类型错误时返回 TRUE
```

```
{
    printf("输入的菜单序号错误\n");
    system("pause");
    system("cls");
}
```

```
cin.clear();
cin.get();
continue;
}
```

二、iomanip

英文名：input and output manipulate（控制）

作用：格式输入输出。

1. setprecision(n)

作用：设置浮点数的有效数字为 n 位。

例子：设置保留几位小数，设置几位有效数字。

```
double a=314.1591;
```

保留小数点后三位：cout<<setiosflags(ios::fixed)<<setprecision(3)<<a<<endl; //输出：314.159

保留三位有效数字：cout<<setprecision(3)<<a<<endl; //输出：314

2. setw()和 setfill()

说明：setw()设置宽度，setfill()设置填充字符。

例子：设置输出宽度，设置填充符号。

```
int number = 123; // 要打印的整数
```

默认左边填充：cout << setw(5) << setfill(' ') << number<<endl; //输出： 123

默认填充空格：cout << setw(5) << number<<endl; //输出： 123

设置左对齐：cout << left<<setw(5) <<setfill('*')<< number<<endl; //输出：123**

说明：left 表示左对齐，设置宽度之后不够的部分用填充符号填充。

三、sstream

全称：string stream，字符串流。作用：将字符串变成流对象。

用法：#include<sstream>

例子：使用 istream 排除空格。

说明：如果直接用 cin>>str 作为判断语句，那么不能终止循环。因为输入的'\n'也是合法的。

```
string str;
```

```
getline(cin,str);
```

```
istringstream iss(str);
```

```
string str;
```

```
while(iss>>str){略}
```

❖ 案例：将字符串按照空格拆分

链接：[..\基础\临时存储\2024.5.2.cpp](#)

关键代码：

思路：字符串流。

```
#include<sstream>
```

```
string a = "nishi baba!";
```

```
vector<string> b;
istringstream iss(a);
string word;
//使用 getline 从 iss 中读取单词，它会在遇到空格时停止读取。
while (iss >> word) {
    b.push_back(word);
}
```

四、stdio.h

全称：standard input and output

作用：标准输入输出库。

用法：#include<stdio.h>

1. printf ()

作用：输出的。

printf("20%02d-%02d",t,bd); //int 输出时，补零的操作。

2.2 字符串

2.2.1 基础知识

一、<cstring>

1. s.substr(i,length)

作用：string 类型找子串。细节：i 往后 length 长度字符串，包括 i，不包括 i+length。

❖ 案例：代码处理，去除注释。

链接：[..\机试准备\代码处理.cpp](#)

时间：2024.9.21，20min

描述：去除“//”、“/* */”。

关键思路：遍历字符串，跳过注释。开始遍历：

遇到“//”跳过后文，即判断'\n'和'\0'。（这里'\0'等效于 EOF）

遇到“/*”匹配“*/”进行跳过。

遇到换行符，换行。

遇到正常代码，正常输出。

istreambuf_iterator<char>(file) 是一个用于迭代输入流中每个字符的迭代器。

istreambuf_iterator<char>()表示默认构造，代表输入流的结束（也就是文件结尾 EOF）。

int row = 1; // 当前行号，全局变量

void print_row(){// 打印当前行号

if(row==1) cout<<setw(2)<<right<<row << "|";

```

        else cout << "\n" <<setw(2)<<right<<row << "|";
    }
}
// 跳过单行注释 // 后面的内容，直到遇到换行符 '\n'
int skip_single_line_comment(const string& str, int i) {
    while (str[i] != '\n' && str[i] != '\0') i++; // '\0' 是字符串结束符
    return i;
}
// 跳过多行注释 /* */ 之间的内容，直到找到 */。如果中途遇到换行符依然换行。
int skip_multi_line_comment(const string& str, int i) {
    while (!(str[i] == '*' && str[i + 1] == '/')) {
        if (str[i] == '\n') {
            row++; // 遇到换行符时，行号加 1
            print_row(); // 打印行号
        } i++;
    } return i + 1; // 跳过最后的 */
}
}
// file 是 ifstream 实例化的对象，打开了文件。
细节：(istreambuf_iterator<char>(file)), 外层还有一个括号。原因不知。
string str((istreambuf_iterator<char>(file)), istreambuf_iterator<char>()); // 读取整个文件内容
print_row(); // 打印第一行行号
for (int i = 0; i < str.size(); i++) {
    if (str[i] == '/' && str[i + 1] == '/') { // 如果遇到单行注释 //
        i = skip_single_line_comment(str, i); // 跳过单行注释的内容
        print_row();
    } else if (str[i] == '/' && str[i + 1] == '*') { // 如果遇到多行注释 /*
        i = skip_multi_line_comment(str, i + 2); // 跳过多行注释的内容
    } else if (str[i] == '\n') { // 遇到换行符
        row++; // 行号加 1
        print_row(); // 打印行号
    } else {
        cout << str[i]; // 输出非注释内容
    }
}
}

```

常见的库函数：

to_string(): 可以用来将 int 类型转 string 类型。

二、ctype

作用：有关字符判断和处理的库函数

tolower(): 如果参数是大写字符，返回其小写，否则返回该参数

toupper(): 如果参数是小写字符，返回其大写，否则返回该参数

islower(): 判断是否是小写。

isupper(): 判断是否是大写。

isalpha(): 判断字符是否是字母 (A-Z, a-z)。

isdigit(): 判断字符是否是数字 (0-9)。

isalnum(): 判断字符是否是字母或数字。

三、string.h

作用：包含各种字符串处理函数。

1. strcpy(被赋值, 值)

作用：赋值用的。

1) strcmp(a,b)

int strcmp(const char* str1, const char* str2);

strcmp 比较两个字符串的大小，一个字符一个字符比较，按 ASCII 码比较标准规定：

第一个字符串大于第二个字符串，则返回大于 0 的数字

第一个字符串等于第二个字符串，则返回 0

第一个字符串小于第二个字符串，则返回小于 0 的数字

2. getline(cin,a)

作用：用来写入 string 类型字符串的。对于 string 类型的数据：getline(cin,a)。

知识点：手动设置结束符，默认结束符是 '\n'。

输入：12310990

输出：12310

char a[100];

cin.getline(a,100,'9');

cout<<a;

replace ()

作用：替换函数中的某一部分。

(1) str=str.replace(i-cnt,cnt,"27"); //replace()函数的调用格式

//应该为 str.replace(start,length,"newstring");

//注意，这里是 length，长度

(2) 替换子串为另外一个串

//s=s.replace(s.find(a),a.length(),"a");

//replace 函数用法，其中后者为长度 length

//用于替换一长的字符串中某一具体的小部分

3. find()

作用：查找函数中的某个字符或字符串。找到了就返回子串初始位置。

❖ 例子：

string a="chi1 huo3 guo1";

int t=str.find(a);

if(t==-1) return 0;

//find 函数可以找长的字符串中具体的一个小部分。

//如果没找到，则返回-1；找到了则返回该具体字符串的初始位置

else return 1;

4. strstr()函数

头文件: #include<string.h>

作用: 判断 b 是否是 a 的子串。若是子串, 则返回子串所在初始位置, 未找到返回 NULL。

❖ 案例一: [..\基础\库函数\string-strstr.cpp](#)

关键代码: `strstr(a,b)`

案例二: [..\基础\库函数\string-strstr-char.cpp](#)

关键代码:

```
char str[]="I am your old Father";
char *p=strstr(str,"old");//返回字符串出现的第一个地址, 若未找到, 则返回 NULL
cout<<p;/"old Father"
```

5. memset()函数

头文件: include<cstring>

格式: `memset(数组(结构体), 赋值, 规格 (一般为 sizeof(数组)))`

注意:

- 1.在为字符串清除的时候, 可以赋任何值来清除。
- 2.在给整型数据赋值时, 只能赋值 0 和-1。

❖ 案例:

```
memset(s,'b',sizeof(char)*20);
```

❖ 案例: 清空结构体

```
memset(&stu,0,sizeof(Student));
```

❖ 案例: [..\基础\库函数\string-set.cpp](#)

描述: 在给整型数据赋值时, 只能赋值 0 和-1, 赋值其他值时会自动转换为 inf。

关键代码:

```
memset(a,-1,sizeof(a));
```

6. c_str()

理解: 就是将 string 类型的字符串转化为 char 类型的。

C++中的 `c_str()` 函数用法

语法:

```
const char *c_str();
```

`c_str()` 函数返回一个指向正规 C 字符串的指针常量, 内容与本 string 串相同。

这是为了与 c 语言兼容, 在 c 语言中没有 string 类型, 故必须通过 string 类对象的成员函数 `c_str()` 把 string 对象转换成 c 中的字符串样式。

❖ 案例: 一定要使用 `strcpy()` 函数来操作方法 `c_str()` 返回的指针。

比如: 最好不要这样:

```
1 char* c;2 string s="1234";3 c = s.c_str();
```

c 最后指向的内容是垃圾, 因为 s 对象被析构, 其内容被处理, 同时, 编译器也将报错——将一个 `const char *` 赋与一个 `char *`。

应该这样用:

```
1 char c[20];2 string s="1234";3 strcpy(c,s.c_str());
```

这样才不会出错，`c_str()`返回的是一个临时指针，不能对其进行操作。

❖ 再举个例子：

`c_str()` 以 `char*` 形式传回 `string` 内含字符串，如果一个函数要求 `char*` 参数，可以使用 `c_str()` 方法：

```
1 string s = "Hello World!";2 printf("%s", s.c_str());    // 输出 "Hello World!"
```

7. `stoi()`

作用：把 `string` 类型转换为 `int` 类型。

log:复习，2025.2.13。

2.2.2 算方案例

说明 2024.9.1：这里主要是关于字符串常见算法。

一、高精度

❖ 案例：大数加法。高精度。

时间：2024.8.30

链接：[大数加法_牛客题霸_牛客网 \(nowcoder.com\)](https://www.nowcoder.com)

思路：理解加法运算规则即可。

关键代码：大数加法。

知识点：`to_string()`，转换成字符串类型。

细节：`while` 判断里面加 `carry`

```
string solve(string s, string t) {
    string ans="";
    int i=s.size()-1;
    int j=t.size()-1;
    int carry=0;//用来承载进制的参数
    while(i>=0||j>=0||carry){
        int a=(i>=0)?s[i--]-'0':0;//三元表达式
        int b=(j>=0)?t[j--]-'0':0;
        int sum=a+b+carry;
        carry=sum/10;
        sum%=10;
        ans=to_string(sum)+ans;
    }
    return ans;
}
```

❖ 案例：大数乘法。

时间：2024.8.30，60min

链接: [大数乘法_牛客题霸_牛客网 \(nowcoder.com\)](https://www.nowcoder.com)

思路: 掌握**乘法原理**, 有意识注意到加法进位问题。

两个循环: 循环顺序从个位到十位。

外循环循环乘数。

内循环循环被乘数。

把个位拿出来

细节: 考虑到进位问题。

关键代码: 大数乘法; 去除前导 0。

两个部分: 处理大数相乘; 去除前导 0。

```
string solve(string s, string t) {
    int m=s.size();
    int n=t.size();
    vector<int> ans(m+n,0);
    for(int i=m-1;i>=0;i--){
        int a=s[i]-'0';
        for(int j=n-1;j>=0;j--){
            int b=t[j]-'0';//单独把 a,b 拿出来
            int sum=ans[i+j+1]+a*b;//把个位拿出来参与运算
            ans[i+j+1]=sum%10;//进位之后个位是多少
            ans[i+j]+=sum/10;//往前进位, 下一次循环会将个位单独拿出来, 不需要单独考虑连续进位。
        }
    }
    string res="";
    for(int i=0;i<ans.size();i++){
        if(!(res.empty()&&ans[i]==0)){
            res.push_back(ans[i]+'0');
        }
    }
    return res.empty()?"0":res;
}
```

二、其他案例

❖ 案例: **判断回文**。

时间: 2024.8.30 (二刷)

链接: [LCR 020. 回文子串 - 力扣 \(LeetCode\)](https://leetcode.cn/problems/longest-palindrome)

方案一思路:

s1.判断回文。

s2.遍历每一个子串。

方案二: **中心扩展法**: 从中间开始朝两边开始比较。

s1.中心法判断回文: 从中间扩展判断。

s2.遍历字符串, 奇数和偶数回文分开判断。

关键代码: 奇数回文、偶数回文。

细节：奇数和偶数分开判断。

中心法：

```
while(start>=0&&end<=s.size()-1&&s[start]==s[end]){
    start--;
    end++;
    cnt++;
}
```

奇数和偶数回文分开判断：

```
for(int i=0;i<s.size();i++){
    cnt+=center(s,i,i);
    cnt+=center(s,i,i+1);
}
```

方法：动态规划方法-区间 dp

时间：2024.10.16, 20min

链接：[647. 回文子串 - 力扣 \(LeetCode\)](#)

思路：区间 dp。dp[i][j]表示子串 s[i~j]是不是回文子串。

若 $s[i]==s[j]$ ，那么需要继续判断奇数还是偶数。

如果 $i-j \leq 1$ ，那么 dp[i][j]一定是回文。

如果 $i-j \geq 2$ ，那么如果 dp[i+1][j-1]是回文，那么 dp[i][j]就是回文。

若 $s[i] \neq s[j]$ ，那么 dp[i][j]一定不是回文。

关键代码：

细节：需要额外判断 i 和 j 相差小于等于 1 和大于 1 两种情况。

```
vector<vector<bool>> dp(n,vector<bool>(n,false)); //dp[i][j]表示从[i,j]回文子串的数量
int cnt=0;
for(int i=n-1;i>=0;i--){
    for(int j=i;j<n;j++){
        if(s[i]==s[j]){
            if(j-i<=1){
                dp[i][j]=true;
                cnt++;
            }else if(dp[i+1][j-1]==true){ //j-i>=2//偶数了
                dp[i][j]=true;
                cnt++;
            }
        }
    }
}
```

❖ 案例：[KMP 算法-模式匹配](#)。相等前进，不相等回退。

时间：2024.8.31 60min

链接：[kmp 算法_牛客题霸_牛客网 \(nowcoder.com\)](#)

基本思想：字符串匹配，如果模式串匹配失败，不需要全部回退，只需要局部回退即可。

求 next 数组：S[j]与自己 S[i]模式匹配，匹配成功则共加，更新 next[j]=j;不相等则 S[j]通过 next[j-1]回退。

模式串匹配：如果 S[j]与 T[i]相等，共加；不相等则 S[j]通过 next[j-1]回退。

关键代码：求 next 数组；利用 next 数组进行匹配。

注意：j 作模式串脚标，i 作为匹配串脚标。

细节：构建 next 数组的时候

```
int kmp(string S, string T) {
    if(S.empty()||S.size()>T.size()) return 0;
    int cnt=0;//计数用的
    //计算 next 数组
    vector<int> next(S.size());
    next[0]=0;
    int j=0;//指向模式串
    for(int i=1;i<S.size();i++){//S 和自己进行模式匹配，细节，这里 i 从 1 开始
        while(j>0&&S[i]!=S[j]) j=next[j-1];
        if(S[i]==S[j]) j++;
        next[i]=j;
    }
    //模式匹配
    j=0;
    for(int i=0;i<T.size();i++){
        while(j>0&&S[j]!=T[i]) j=next[j-1];
        if(S[j]==T[i]) j++;
        if(j==S.size()){
            cnt++;
            j=next[j-1];
        }
    }
    return cnt;
}
```

小结：next 数组求解与 S、T 匹配有共性，如果相等则共加，如果不相等则基于 next[j-1]回退。

❖ 案例：移掉 k 位数字。

时间：2024.8.31，120min

链接：移掉 K 位数字_牛客题霸_牛客网 ([nowcoder.com](https://www.nowcoder.com))

描述：有一个 n 位数，移掉 k 个使其最小。

思路：原始序列变成增序列。

关键代码：尽可能增序列；k 有剩余时移除多余部分；消除前导 0；

细节：使用 resize(size)来删除多余的 k。

```
string res;
for (char c : num) {
    //考虑到栈里面的元素一直比后面的大，这里使用循环来解决问题。
    while (!res.empty() && k > 0 && c<res.back()) {
```

```
        res.pop_back();
        k--;
    }
    res.push_back(c);
}
res.resize(res.size() - k); // 如果 k 仍然大于 0，继续移除末尾的字符
// 消除前导零
for(int i=0;i<res.size();i++){
    if(!(s.empty()&&res[i]=='0')){
        s.push_back(res[i]);
    }
}
```

同理：如果删掉之后最大，使得原序列变成减序列。

2.3 子串

时间：2026.4.13

说明：顾名思义，字符串的子串。

❖ 案例：和为 k 的子数组

时间：2026.4.13 15min

链接：[560. 和为 K 的子数组 - 力扣 \(LeetCode\)](#)

思路：遍历数组，基于当前元素往后遍历以此判断是否存在这样一个子数组。

❖ 案例：最长回文子串

时间：2026.9.7 25min 这段时间都在刷题，有题感。

链接：[5. 最长回文子串 - 力扣 \(LeetCode\)](#)

思路启发：只要知道判定回文串的“中心扩展法”一下子就能想到这个思路。

基本思路：遍历字符串，以每个字符为出发点，判断一下有没有包含该字符串的回文子串即可。

细节上需要重视的一点：需要定义变量来记录一下最优解，保持更新。

补充知识点：s.substr(pos, length)

2.4 时间

一、time.h

1. ctime()

函数: `ctime()`

功能: 把日期和时间转换为字符串。

1) 细节

时间: 2022.4.18

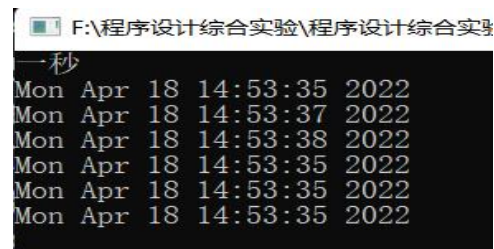
注意: `cout` 与 `printf` 在调用 `ctime()` 时需注意。

`ctime()` 函数使用静态缓存区缓存时间, 在同一个 `printf` 或者 `cout` 函数当中调用多个 `ctime()` 函数时, 时间会按顺序叠加到缓存区后面, 每次读取都是从缓存区第一个数据进行读取, 所以读到最后, 每次调用读出的数据都是第一个时间。

❖ 案例: [..\基础\库函数\ctime-注意点.cpp](#)

说明: `ctime()` 注意点。

输出结果:



```
F:\程序设计综合实验\程序设计综合实验
一秒
Mon Apr 18 14:53:35 2022
Mon Apr 18 14:53:37 2022
Mon Apr 18 14:53:38 2022
Mon Apr 18 14:53:35 2022
Mon Apr 18 14:53:35 2022
Mon Apr 18 14:53:35 2022
```

结论: `ctime()` 函数需要使用 `printf` 或者 `cout` 输出两个及以上的时间时, 一个 `printf` 或者 `cout` 只能输出一个时间。

2. `mktime()`

作用: 制定一个自己的时间。

头文件: `#include <ctime>`

作用: C 库函数 `time_t mktime(struct tm *timeptr)` 把 `timeptr` 所指向的结构转换为自 1970 年 1 月 1 日以来持续时间的秒数, 发生错误时返回 -1

案例一:

描述: 设定一个自己的时间。

关键代码:

```
int a,b,c,d;
cin>>a>>b>>c>>d;
time_t e;
struct tm fuck;
memset(&fuck,0,sizeof(fuck));
fuck.tm_year= a-1900;
fuck.tm_mon=b-1;
fuck.tm_mday=c;
fuck.tm_hour=d;
e=mktime(&fuck);
cout<<ctime(&e)<<endl;//把秒数转换为标准时间格式
cout<<e<<endl;
```

2.5 STL 库

英文名称：Standard Template Library，标准模板库。

总结：常用的包括 vector, map, set, list。

重点掌握：

1. 怎么创建和初始化。
2. 怎么增、删、查、改。
 - (1) 增：insert(s1.begin(), s1.end())
 - (2) 删：erase(s.begin(), s.end())
 - (3) 查：迭代器，iterator。begin(), end(), v[i]; push_back(), pop_back()。find()。
 - (4) 改：查到之后改。
3. 常见属性。如 size、empty。

2.5.1 vector 容器

头文件：#include<vector>

1. 元素索引

(1) at()和 v[i]

at()函数：可以检查下标是否越界。

数组下标索引：如 v[i]。

注意：采用 at()进行随机访问的时候，可以自动检测下标是否越界

(2) begin()与 end()

begin()函数：返回第一个元素的指针。

end()函数：返回指向 end 的下一个，一定是最后一个元素的下一个，即无效的指针。

注意：强调这里返回的是指针。

(3) rbegin()和 rend()

rbegin()函数：返回最后一个元素的指针。

rend()函数：返回第一个元素上一个元素的指针，即一个无效指针。

例子：创建，迭代。

创建，迭代

```
vector<int>::reverse_iterator ire, irend;
irend = ivec.rend();
for(ire = ivec.rbegin(); ire != irend; ire++) {
    cout << *ire << " ";
}
```

(4) front()和 back()

front()函数：返回第一个元素。

back()函数：返回组后一个元素。

注意：强调这里返回的是元素。

(5) find()

find()函数：返回要查找内容的指针，若没有，则返回 end()。不能直接作为 bool 函数进行条

件判断（2024.9.24）。

❖ 案例：[..\基础\STL库\vector-综合.cpp](#)

时间：2022.5.25

描述：综合运用。

常用的函数。

`vector<int> ivec(10,1);`//容器动态分配内存，每次内存满的时候，在原来内存的基础上，分配一倍的内存。

`ivec.reserve(15);`//提前分配 15 个内存，所以下一个 `capacity()`返回值是 15

`ivec.push_back(10);`

`ivec.push_back(20);`

`cout<<"Size:"<<ivec.size()<<endl;`//这里应该是 12

`cout<<"Capacity:"<<ivec.capacity()<<endl;`// 这里应该是 15

`ivec.clear();`//清楚容器中的所有元素，但是不会消除分配的内存。

`cout<<"Size:"<<ivec.size()<<endl;`//现在是 0

`cout<<"Capacity:"<<ivec.capacity()<<endl;`//现在是 15

2. 元素增删

(1) `insert()`和 `erase()`

`insert()` 函数：插入。`v.insert(v.begin(),array,array+SIZE)`

`erase()`函数：删除。`v.erase(v.begin(),v.end());`

例子：创建一个 `vector` 迭代器，使用加法作偏移，使用 `insert` 添加元素，`erase` 删除元素。

`ivec.erase(it+6,it+10)` 包含第 11 个元素，两边都是闭区间。

`vector<int>::iterator it=ivec.begin();`

`ivec.insert(it+5,100);`//在第 6 个元素前面加一个元素

`ivec.erase(it+6,it+10);`//删除第 7 到第 11 个元素。

例子：在 `vector` 末尾添加多个新元素。

代码：`vec.insert(vec.end(),{e1,e2,e3})`

时间：2024.11.2

初始化方法：`set` 初始化。可以通过集合的样式进行初始化。

例子：`result.push_back(vector<int>({start,end}));`

说明：通过集合样式进行初始化，这样快捷方便。

(2) `push_back()`和 `pop_back()`

`push_back(19)`：在最后一个位置添加。

`pop_back()`函数：移除最后一个元素。

(3) `clear()`

`clear()`函数：清除 `v` 中所有内容的函数。

❖ 案例: [..\基础\STL 库\vector.cpp](#)

描述: 详细使用描述。

(4) `push_back()`和 `emplace_back()`区别

`push_back()`添加的对象需要拷贝。

`emplace_back()`添加的对象会自动构造。效率会更高。

3. 属性函数

`size()` 函数: 获取动态数组的大小。

`max_size()`: 输出容器的最大容量。

`capacity()`函数: 返回当前已经分配的容量。

`empty()`函数: 检验容器是否为空。

`reserve(15)`函数: 提前分配 15 个内存, 所以下一个 `capacity()`返回值是 15。

4. 迭代器

迭代器 `iterator`: 用于访问动态数组。

迭代器对于 `int` 类型的使用

```
vector<int>::iterator it;
for(it=v.begin();it!=v.end();it++){
    cout<<*it<<" ";
}
```

❖ 案例 2: 迭代器对于 `string` 类型的使用。

知识点: 使用符号`*`表示指针的解引用运算符, 获取迭代器指向的那个元素的值 (“解引用”就是 “解开对地址的引用, 拿到实际值”)

```
vector<string> v;
// 初始化用的 push_back()
for(it=v.begin();it!=v.end();it++)
{
    cout<<*it<<" ";
}
```

2.5.2 map 容器

定义: 反应映射的一种容器, `map<key,value> name;map<int,int> mp; Vs vector<> Vs int[]`。

补充: **map 容器会按照元素 key 值进行自动排序**, 如果是结构体可以自定义排序 `operator`。

建立元素的时候有默认初始值。如果 key 值是 `vector<int>`, 会对 `vector<int>`进行内部排序。

分析: 如果有去重和排序的需求, 可以使用 `map` 容器。

1. 元素索引

1) `m[i]`和 `find()`

可以通过下标访问（不存在时会创建键并赋予默认值）。可以用 `find()` 函数查找（不存在返回 `end()`）。如：`stu.find(1010)`

❖ 案例：可以通过数组形式访问和赋值。

知识点：脚标索引，若不存在则返回默认值。

```
map<char,int> in;
in['+']=10;//可以按数组索引方式赋值
cout<<in['+']<<endl;
cout<<in['-']<<endl; //输出：0 。没有赋值时默认返回 0 。如果不想污染原数据则采用
find 进行查找（若不存则返回 end()），不然默认赋值机制会弄脏原数据。
```

2) `begin()`和 `end()`

3) `lower_bound()`和 `upper_bound()`

`upper_bound()`：返回当前元素下一个元素的指针。

`lower_bound()`：返回当前元素上一个元素的指针。

关键代码：

```
it1=stu.lower_bound(1005);
it2=stu.upper_bound(1007);
```

4) `rbegin()`和 `rend()`

✧ 类比 1: `first()`

✧ 类比 2: `second()`

`auto b=mp.rbegin();`//`rbegin()`函数，返回一个逆向的迭代指针

//`first()`函数，找 `map()` 的第一个元素，即 key 索引元素

//`second()`函数，找 `map` 的第二个元素，即 value 元素

```
cout<<(*a).first<<" "<<(*a).second<<endl;
```

```
cout<<(*b).first<<" "<<(*b).second;
```

2. 元素增删

`insert()`函数：添加。要用 `pair` 进行绑定。

`erase()`函数：删除。

❖ 例子：

```
stu.insert(pair<int,string>(1007,"son"));
```

```
stu.insert(map<int,string>::value_type(1006,"daughter"));
```

```
stu.erase(it1,it3); //两指针之间的元素。
```

3. 属性函数

`size()`：返回大小。

4. 迭代器

`map<int,string>::iterator it`;有两个属性，一个是 `first`，对应 `key`；一个是 `second`，对应是 `value`。

❖ 案例：[..\基础\STL 库\map-综合.cpp](#)

描述：综合案例。

5. `multimap` 容器

用法：`multimap<string,string> book;`

案例: [..\基础\STL 库\multimap.cpp](#)

描述: 与 map 的区别, multimap 容器中 find()函数返回的可能是一个指针数组, 因为它允许重复出现一样的 key。

6. unordered_map 容器

作用: 底层是哈希表, 用来做哈希存储的。

2.5.3 set 容器

定义: 集合。set<int> p 对于整形和字符型, 可以自动排序和去重。

初始化

```
int a[10]={4,1,2,3,4,5,6,7,8};  
set<int> se1(a,a+10);
```

1. 元素索引 (查)

- 1) begin()和 end()
- 2) find()函数: 返回要查找内容的指针, 若没有, 则返回 end()。
- 3) 范围 for ,比如: for(int num:set1) 。说明: 不支持角标索引。

2. 元素增删 (增删)

说明 2024.10.29: 一定不要在遍历集合的时候对集合进行增删操作, 这样会导致意想不到的错误。

insert(i)函数: 增加一个元素。

erase(i)函数: 删除某个元素。

3. 属性函数

- 1) size()

4. 迭代器

用迭代器输出。

如: set<int>::iterator it

```
cout<<*it<<endl;
```

5. 其他函数

set_union()函数: 求并集。

set_intersection()函数: 求交集。

set_difference()函数: 求差集。

set_symmetric_difference()函数: 求差异集。

❖ 案例: [..\基础\STL 库\set-functions.cpp](#)

描述: 具体使用。

关键代码:

```
set_union(se1.begin(),se1.end(),se2.begin(),se2.end(),inserter(sun,sun.begin()));  
cout<<"并集: ";Print(sun);
```

```

set_intersection(se1.begin(),se1.end(),se2.begin(),se2.end(),inserter(sin,sin.begin()));
cout<<"交集: ";Print(sin);
set_difference(se1.begin(),se1.end(),se2.begin(),se2.end(),inserter(sdi,sdi.begin()));
cout<<"差集: ";Print(sdi);
set_symmetric_difference(se1.begin(),se1.end(),se2.begin(),se2.end(),inserter(ssd,ssd.begin()));co
ut<<"对称差: ";Print(ssd);
小结: set_union(s1.begin(),s1.end(),s2.begin(),s2.end(), inserter(s,s.begin()))

```

❖ 案例 1: [..\基础\STL 库\set-compare.cpp](#)

描述: 求两个集合交并比。

❖ 案例 2: [..\基础\STL 库\set-iterator.cpp](#)

要说明的问题: 为何每次 insert 之后, 以前保存的 iterator 不会失效?

iterator 这里就相当于指向节点的指针, 内存没有变, 指向内存的指针怎么会失效呢(当然被删除的那个元素本身已经失效了)。相对于 vector 来说, 每一次删除和插入, 指针都有可能失效, 调用 push_back 在尾部插入也是如此。因为为了保证内部数据的连续存放, iterator 指向的那块内存存在删除和插入过程中可能已经被其他内存覆盖或者内存已经被释放了。即使 push_back 的时候, 容器内部空间可能不够, 需要一块新的更大的内存, 只有把以前的内存释放, 申请新的更大的内存, 复制已有的数据元素到新的内存, 最后把需要插入的元素放到最后, 那么以前的内存指针自然就不可用了。特别时在和 find 等算法在一起使用的时候, 牢记这个原则: 不要使用过期的 iterator。

一、list 链接

对指针的理解: 将指针视为箭头、动态分配空间视为固定不变的房间, 不同的箭头可以指向同一个房间并且对房间里面的元素进行调用, 特别是对房间里面的指针 next 进行调用。

说明:

1. list 是通过双向链表来实现的, 所以每次元素跨度只能是一, 可以双向跨, 即 it+1, it-1.
2. 线性表特点: 便于删减, 但不便于随机存取。

1. 元素索引

1) begin()和 end()

2) rbegin()和 rend()

rbegin()函数: 返回最后一个元素的指针。

rend()函数: 返回第一个元素上一个元素的指针, 即一个无效指针。

2. 元素增删

1) push_front()和 pop_back()

push_front(): 在首部加元素。

pop_front(): 删除首部元素。

2) push_back()和 pop_back()

push_back(): 在尾部加元素。

pop_back(): 在尾部删元素。

3) insert()和 erase()

insert(it,value/vector): 增加元素。

erase(it,value/vector): 删除[first,last)

3. 属性函数

empty(): 判断 list 是否为空。

表大小的函数

size()

max_size()

4. 迭代器

list<int>::iterator it;

5. 功能函数

sort(): 自己的排序函数。

splice():拼接。

❖ 案例：实操。

注意：b2 没有了，被拼接到了 b1 上。

```
it=b1.begin();
```

```
it++;//splice()拼接到指定的位置
```

```
b1.splice(it,b3);
```

```
cout<<"splice():";
```

```
Print(b1);
```

//注意：b2 没有了，被拼接到了 b1 上

merge():融合。

说明：相对的升序排，排出来整体不一定是升序的；

❖ 案例：实操。

```
b1.merge(b2);
```

```
cout<<"merge ():";
```

```
Print(b1);//merge()函数，相对的升序排，排出来整体不一定是升序的；
```

这里先不一一列举，用的时候再说

remove_if():根据判断条件移除。

❖ 案例：实操。

说明：函数参数必须是一个值，如果是判断条件则必须借助函数适配器。

//这里引入了适配器的概念

```
//函数原型 void remove_if(unpred pr)
//bind2nd(function(),val)将 val 绑定到 function()的第二个参数上
//这样的话 not_equal_to<int>()将变成参数只有一个的函数
//引: bind1st()表示绑定第一个参数
//not_equal_to<type>(parameter)是函数模板, 表示判断是否相等的。
ilist.remove_if(bind2nd(not_equal_to<int>(),5));
Print(ilist);
```

❖ 案例: [..\基础\STL 库\list.cpp](#)

描述: 综合应用。

STL 中的 list 是根据双向链表来实现的。线性表, 特点, 便于删减, 但不便于随机存取。

二、deque 队列

理解: 双端队列。特点: 先进先出, 后进后出。

1. 元素索引

1) front()与 back()

front()函数: 返回第一个元素。

back()函数: 返回最后一个元素。

2) begin()和 end()

返回指针

3) rebegin()和 rend()

2. 元素增删

1) push_front()和 pop_front()

在队列前面进行的操作。

2) push_back()和 pop_back()

在队列后面进行的操作。

3) insert()和 erase()

insert(it,value/vector)

erase(it,value/vector):删除[first,last)

3. 属性函数

表大小的函数

size()

max_size()

empty()

❖ 案例: [..\基础\STL 库\deque.cpp](#)

说明: 这个是双向的队列。并不是完全按照链表来的, 是 list 与 vector 的组合, 所以指针可以一次性跨多个。

it=it+2;//队列并不是完全的链表, 所以链表可以加 n (n>=2)

❖ 案例: [..\数据结构测试.大二上\模拟栈\模拟队列.cpp](#)

描述：通过结构体和结构体指针模拟队列。

大致思路：通过结构体 QNode 和一个以(QNode*类型的)front 和 rear 为元素的结构体 LinkQueue 来实现。主要是应用的指针。

关键代码：

```
struct QNode{
    int data;
    QNode *next;
};
struct LinkQueue{
    QNode *front;
    QNode *rear;
};//操作的时候从这两个指针去操作。
```

三、queue 队列

理解：先进先出队列。

细节：pop()函数返回 void。

front (),pop()

四、stack 栈

定义：先进后出。是容器适配器，不是容器。

说明：在进行 pop 或者 top 操作之前，需要判断 empty。不然当 stack 为空的时候，容易留下报错隐患。（2025.2.12）

1. 元素索引

top(): 获取栈顶元素。

2. 元素增删

push()入栈

pop()出栈。

3. 属性函数

size(): 判断栈大小。

empty(): 判断栈是否为空。

❖ 案例：[..\数据结构测试.大二上\模拟栈\栈.cpp](#)

描述：实际应用。

五、priority_queue

定义：优先队列，基于堆构建的。是容器适配器，不是容器。

1. 元素索引

top(): 获取堆顶元素。

2. 元素增删

push():增。会自动排序。

`emplace()`:增。

`pop()`:删掉堆顶元素，会自动排序。

✧ `push` 和 `emplace` 的区别

`push` 和 `emplace` 都可以用于插入元素到优先级队列中。

`push` 更适合已经有一个现成的对象需要插入。

`emplace` 更适合需要在队列中直接构造对象的场景，通常在性能上更优，因为它避免了额外的临时对象构造和拷贝/移动操作。

例子：有一个 `person` 结构体，有 `name` 和 `age` 两个属性，则 `emplace` 用法，`pq.emplace("Bob", 25);`

3. 属性函数

`size()`:返回堆大小。

`empty()`:返回是否为空。

例子：自定义优先队列。

```
#include <queue>
```

特别说明：后文的 `Compare` 必须是一个类型（`struct`，比较类型），不能是函数。

```
// 自定义比较函数
```

```
struct Compare {  
    bool operator()(pair<int, string> a, pair<int, string> b) {  
        return a.first > b.first; // 小顶堆  
    }  
};
```

```
priority_queue<pair<int, string>, vector<pair<int, string>>, Compare> pq;
```

六、STL 模板函数

1. `for_each`

案例：[..\基础\STL 库\for-each.cpp](#)

关键代码：

```
for_each(ivec.begin(), ivec.end(), Print<int>())
```

2.6 其他

一、`algorithm`

作用：提供一些算法函数。

用法: #include<algorithm>

1. sort()函数

作用: 排序。自定义一个 cmp 函数, 函数输入为两个比较类型, 输出为 bool 值。

❖ 案例: **层级排序**。多级排序。

链接: [成绩排序_牛客题霸_牛客网 \(nowcoder.com\)](https://www.nowcoder.com)

时间: 2024.9.21, 10min

描述: 自定义排序方法的排序。多级排序, 如果某一级相同, 那么继续比下一级。

凝练: 一级一级的往后排序。

关键代码 (有省略) 写法一: 重载结构体小于运算符。

重载结构体的小于运算符。

```
struct student{
    string name;
    int age;
    int score;
    student():name("name"),age(-1),score(0){}; //细节: 默认构造函数
    student(string name_, int age_, int score_):name(std::move(name_)),age(age_),
score(score_){};
    bool operator<(student &s2){
        if(score==s2.score){
            if(name==s2.name){
                return age<s2.age;
            }else return name<s2.name;
        }else return score<s2.score;
    }
};
主函数: sort(stu.begin(),stu.end());
```

关键代码 (有省略) 写法二: 单独写 cmp

单独写排序规则 cmp

```
bool cmp(student &s1,student &s2){
    if(s1.score==s2.score){
        if(s1.name==s2.name){
            return s1.age<s2.age;
        }else return s1.name<s2.name;
    }else return s1.score<s2.score;
}
主函数: sort(stu.begin(),stu.end(),cmp);
```

2. reverse()函数

作用: 逆序。

例子: 将前 pos 个元素放到数组末尾。

细节: 区间左闭右开。

```
reverse(cards.begin(),cards.begin()+pos);//对前 pos 个进行逆序，脚标为 pos-1  
reverse(cards.begin()+pos,cards.end());//对 pos 之后的进行逆序  
reverse(cards.begin(),cards.end());//对整体实现逆序。
```

3. count()

作用：计算序列中某个元素数量。

用法：count(ivec.begin(), ivec.end(), searchValue)

补充：count_if()参数传递。

示例代码：count_if()

说明：放入一个 bool 函数作为判断条件。

```
bool greater10(int value) {return value >10; }  
int result=count_if(v1.begin(), v1.end(), greater10);
```

二、functional

作用：函数适配器。为了让某些函数能少输入几个参数就能运行。

用法：#include<functional>

1. greater<type>()

作用：大于。

2. not_equal_to<type>(num1,num2)

作用：不等于。

3. bind1st()

作用：如，bind2nd(function(),val)将 val 绑定到 function()的第二个参数上。

类比：bind2nd()

❖ 案例：实操。

//这里引入了适配器的概念

```
//函数原型 void remove_if(unpred pr)  
//bind2nd(function(),val)将 val 绑定到 function()的第二个参数上  
//这样的话 not_equal_to<int>()将变成参数只有一个的函数  
//引：bind1st()表示绑定第一个参数  
//not_equal_to<type>(parameter)是函数模板，表示判断是否相等的。  
ilist.remove_if(bind2nd(not_equal_to<int>(),5));  
Print(ilist);
```

三、stdlib.h

全称：standard library

用法：#include<stdlib.h>

1. malloc()

作用：分配空间。

案例：实操。

```
Student *stu2=(Student*)malloc(sizeof(Student));  
strcpy(stu2->name,"SB");
```

```
cout<<"测试 10, malloc:"<<stu2->name<<endl;
```

四、numeric

用法: `#include<numeric>`

1. `accumulate(begin,end,初始值)`

作用: 求和。

例子: `int sum = accumulate(nums.begin(), nums.end(), 0);`//使用库函数求和

五、windows.h

作用: 调用 windows 系统的一些函数。

用法: `#include<windows.h>`

1. `Sleep()`函数

头文件: `#include<windows.h>`

注意: `Sleep()`函数的首字母必须要大写

六、bitset

作用: 位流。

用法: `#include<bitset>`

```
bitset<8> bs(c)
```

注意: `bitset` 对象存放的时候, 二进制串的末位放在前面, 高位放在后面。使用 `cout` 输出的时候按照正常的二进制串的顺序输出。

例如: 原始二进制串 11100011, 在 `bitset` 里面存储为 11000111, 即 `bs[7]`为最高位。

案例: 奇偶校验。

时间: 2024.9.21 20min

链接: [奇偶校验_牛客题霸_牛客网 \(nowcoder.com\)](https://www.nowcoder.com/ta/algorithm)

说明: 奇偶校验, 对于位数据。奇检验, 1 的个数为奇数, 如果为偶数则手动将首位设置为 1, 使其 1 的个数变为奇数。同理, 偶检验, 1 的个数为偶数, 如果为奇数, 手动将首位设置为 1, 使其变为偶数。

方法 1: 直接求余除计算二进制 1 的个数。

知识点: `char` 类型转换为 `int` 类型, `int` 数值为 `char` 类型的 `ascii` 码。 `int a=int(c)`。

'0'十进制 ASCII 是 48, 'A'十进制 ASCII 码是 65, 'a'十进制 ASCII 是 97。

方法 2: 使用 `bitset<>`

关键代码: 使用 `bitset` 位操作。

思路: 用 `bitset` 创建位向量, 再用 `count` 成员方法计算 1 的个数, 最后判断并调整最高位, 可以直接通过 `cout` 方法输出。

```
char s;
```

```
while (cin >> s) {
```

```
    bitset<8> bs(s);//固定大小的位向量
```

```
    auto c = bs.count();
```

```
bs[7] = (c % 2 != 1); //如果是偶数设置为 1，奇数设置为 0
cout << bs << endl;
}
```

七、climits

作用：常见的限制数据。

用法：#include<climits>

INT_MAX, INT_MIN：整数最大值、最小值。

第 3 章 数据结构

逻辑结构：集合、线性结构、树形结构、图状结构

存储结构：顺序结构、链式结构、哈希结构、索引结构

数据结构总述：

数组（Array）、队列（Queue）、链表（Linked List）、栈（Stack）

树（Tree）、散列表（Hash）、堆（Heap）、图（Graph）

3.1 哈希表

关键知识点：

哈希作用：快速判断一个元素是否出现在集合里。

常用的数据结构：数组、集合（set、multiset、unordered_set）、映射（map、multimap、unordered_map）。

一、理论基础

定义：哈希表是根据关键码的值而直接进行访问的数据结构。一般来说哈希表都是用来快速判断一个元素是否出现集合里。

理论基础：当我们想使用哈希法来解决问题的时候，我们一般会选择如下三种数据结构。

- 数组
- set（集合）：数组的大小是有限的，受到系统栈空间（不是数据结构的栈）的限制。如果数组空间够大，但哈希值比较少、特别分散、跨度非常大，使用数组就造成空间的极大浪费。所以此时一样的做映射的话，就可以使用 set 了。
- map(映射)：数组的大小是受限制的，而且如果元素很少，而哈希值太大会造成内存空间的浪费。set 是一个集合，里面放的元素只能是一个 key，而两数之和这道题目，不仅要判断 y 是否存在而且还要记录 y 的下标位置，因为要返回 x 和 y 的下标。所以 set 也不能用。

集合	底层实现	是否有序	数值是否可以重复	能否更改数值	查询效率	增删效率
std::set	红黑树	有序	否	否	$O(\log n)$	$O(\log n)$
std::multiset	红黑树	有序	是	否	$O(\log n)$	$O(\log n)$
std::unordered_set	哈希表	无序	否	否	$O(1)$	$O(1)$

std::unordered_set 底层实现为哈希表，std::set 和 std::multiset 的底层实现是红黑树，红黑树是一种平衡二叉搜索树，所以 key 值是有序的，但 key 不可以修改，改动 key 值会导致整棵树的错乱，所以只能删除和增加。

映射	底层实现	是否有序	数值是否可以重复	能否更改数值	查询效率	增删效率
std::map	红黑树	key有序	key不可重复	key不可修改	$O(\log n)$	$O(\log n)$
std::multimap	红黑树	key有序	key可重复	key不可修改	$O(\log n)$	$O(\log n)$
std::unordered_map	哈希表	key无序	key不可重复	key不可修改	$O(1)$	$O(1)$

std::unordered_map 底层实现为哈希表，std::map 和 std::multimap 的底层实现是红黑树。同理，std::map 和 std::multimap 的 key 也是有序的（这个问题也经常作为面试题，考察对语言容器底层的理解）。

补充 1: map 支持 vector 作为键值，并且定义了 vector 去重（顺序和元素都相同）和排序（字典序）的方法。unordered_map 不支持 vector 作为键值，可以重定义哈希函数和比较函数。

补充 2: map 等（multimap, unordered_map），如果对应键值没有元素，则会按照数据类型的默认构造进行赋值。

哈希函数：

哈希碰撞->拉链法，线性探测。哈希函数是把传入的 key 映射到符号表的索引上。哈希碰撞处理有多个 key 映射到相同索引上时的情景，处理碰撞的普遍方式是拉链法和线性探测法。

二、案例分析

● 求两数的和。

时间：2025.2.6；2026.3.15 60min（做错了）

链接：[1. 两数之和 - 力扣 \(LeetCode\)](#)

关键：计算 target-x，判断是否在 hash 表中。

错误分析 2026.3.15：忘记了 map 容器常见函数 find，导致替代方案不奏效。

使用 if(mymap[target-nums[i]])时，如果 mymap 不存在该元素，会返回 0，而 0 等效于 false，并且 map 会自动添加该元素，并默认赋值为 0。

● 四数相加

时间：2025.2.6

链接：[454. 四数相加 II - 力扣 \(LeetCode\)](#)

关键：分两组 hash。（上一题的衍生）

涉及知识点：map, unordered_map

● 三数相加

时间：2025.2.8；2026.3.22（90min）

链接：[15. 三数之和 - 力扣（LeetCode）](#)

方法 1：三重循环，通过排序和双指针提高效率。

方法 2：将 vector 作为键值构建 map 进行去重。（效率低于前者）

涉及知识点：vector 可以作为 map 的键值，甚至对于 vector<int> 作为键值而言，还能有自动排序功能，排序之后再 go。 （unordered_map 不行）

此题关键点：去重问题，可以使用 map 数据结构来实现。

```
vector<int> tmp_vec({nums[i],nums[left],nums[right]});
```

```
mp[tmp_vec]++;
```

补充知识点：emplace_back() 会返回新增元素的指针。

● 四数相加

时间：2025.2.10 100min；2026.3.22（20min，有三数求和基础，做的很快）

链接：[18. 四数之和 - 力扣（LeetCode）](#)

思路：跟“三数相加”是一致的。

方法 1：四重循环，通过排序和双指针，缩减为 2 重循环。

剪枝：如果确定 1 个元素之后，剩下的元素无论怎么组合都不能满足要求，则 break。

如果确定两个元素之后，剩下元素无论怎么组合都不能满足要求，则 break。

方法 2：采用 vector<int> 作为 map 的键值进行去重计算。

知识点 1：在循环一个 iterator 的时候，不要去增删该 iterator，不然会造成未知错误。在这里，通过 “[]” 访问 map 的时候，无键值对的时候，会创建一个键并赋予默认值。故而，采用 hash.find()。

知识点 2：语法 (long)int 表示数据类型强转。

● 字母异位词分组

时间：2026.3.15 40min

链接：[49. 字母异位词分组 - 力扣（LeetCode）](#)

思路：把单词 sort 排序之后作为哈希表的键。

知识点：vector 有一个函数 emplace_back() 函数。

❖ 缺失的第一个正数

时间：2026.4.19 60min

链接：[41. 缺失的第一个正数 - 力扣（LeetCode）](#)

方法 1-满足空间复杂度：一个数组 nums 最多能存 n 个数，如果极端情况下存储了 1~n 一共 n 个数，那么缺失的第一个正整数就是 n+1，如果少存了 1~n 中的一个，那么缺失的第一个正整数一定出现在 1~n 中。

将原来的 nums 数组作为 hash 表。

step1. 先把负数赋值为 $n+1$

step2. 基于脚标来表示 $1\sim n$ 中的数，使用负号来表示 $1\sim n$ 的数是否出现过。

step3. 遍历数组，第一个正数对应的脚标即为所求。

细节：脚标和值相差 1。

方法 2-好想一些：定义一个哈希表，出现的就存进去。最后遍历一下就知道了。

关键代码：

时间复杂度为 $O(n)$ ，空间复杂度为 $O(1)$

```
int n=nums.size();
for(int i=0;i<n;i++){
    if(nums[i]<=0){
        nums[i]=n+1;
    }
}
for(int i=0;i<n;i++){
    int val=abs(nums[i]);
    if(1<=val&&val<=n){
        nums[val-1]=-abs(nums[val-1]);
    }
}
for(int i=0;i<n;i++){
    if(nums[i]>0){
        return i+1;
    }
}
return n+1;
```

3.2 双指针

时间：2026.3.18

说明：对于一个数组，用两个索引来进行遍历。一般两个索引分别指向首尾。

说明：一般有两个指针，一个指针用于延伸，一个指针用于收缩。一般会有一个机制用来维护两个指针之间的信息。

3.2.1 滑动窗口

时间：2026.4.13

理解：特殊的双指针。

❖ 案例：找到字符串中所有字母异位词

时间：2026.4.13 60min

链接：[438. 找到字符串中所有字母异位词 - 力扣 \(LeetCode\)](#)

思路：遍历确定一个滑动窗口，将其中的元素和 p 串进行比对是否为异位词。

思考：一来就想用双指针的方法进行优化，把问题做太复杂了，结果做了很久都没写出来、还不如说用最笨的方法，就使用滑动窗口，时间复杂度虽然高一些，但是胜在最后能完成题目。

❖ 最大容器容量

时间：2026.3.18 25min

链接：[11. 盛最多水的容器 - 力扣 \(LeetCode\)](#)

思路：用两个指针指向首尾，计算容器容量，移动高度小的端。

❖ 无重复字符串的最长子串

时间：2026.4.13 35min

链接：[3. 无重复字符串的最长子串 - 力扣 \(LeetCode\)](#)

思路：两个指针，right 指针开始遍历，如果遇到重复的字符，则收缩 left 指针，直到没有重复字符，则继续移动 right 指针。

对于重复字符的判断使用 set 集合进行。

知识点：set 集合，find()方法返回的是指针。可以使用 count 方法来判断是否含有这个元素。

关键代码：

细节：收缩 left 指针的时候需要特判，特判等于 right 的时候，并且 right 此时也需要移动。

```
while(right<n){
    if(subset.find(s[right])==subset.end()){//如果不能找到，说明子串还能加
        subset.insert(s[right]);
        right++;
        max_cur=max(max_cur,right-left);// 细节：没有加 1
    }else{
        while(left<n && subset.find(s[left])!=subset.end()){
            if(s[left]==s[right]){
                left++;
                right++; //细节，需要移动 right 指针，不然就陷入死循环了。
                break;    // 这里直接 break，前面的字符没删干净
            }
            subset.erase(s[left]);
            left++;
        }
    }
}
```

❖ 案例：最小覆盖子串

时间：2026.4.15 60min（没有一次性做出来）

链接：[76. 最小覆盖子串 - 力扣 \(LeetCode\)](#)

思路：使用两个指针，一个指针负责延伸，一个指针负责收缩。

关键：使用两个指针

```
string minWindow(string s, string t) {
    if (s.size() < t.size()) return "";

    unordered_map<char, int> need, window;
    for (char c : t) need[c]++;

    int left = 0, right = 0;
    int valid = 0; // 满足要求的字符种类数
    int start = 0, len = INT_MAX;

    while (right < s.size()) {
        char c = s[right];
        right++;

        if (need.count(c)) {
            window[c]++;
            if (window[c] == need[c]) {
                valid++;
            }
        }

        // 当窗口已经覆盖 t 时，尝试收缩左边界
        while (valid == need.size()) {
            if (right - left < len) {
                start = left;
                len = right - left;
            }

            char d = s[left];
            left++;

            if (need.count(d)) {
                if (window[d] == need[d]) {
                    valid--;
                }
                window[d]--;
            }
        }
    }
}
```

```
return len == INT_MAX ? "" : s.substr(start, len);  
}
```

❖ 案例

时间：2026.9.11 80min 没有做出来，2026.9.12 60min 也没有做出来，结合 AI 改了一下，发现是指针和哈希表一致性的问题。2026.9.13 40min，还是没完全做出来，还是会有些代码上的小问题。

链接：[面试题 16.25. LRU 缓存 - 力扣 \(LeetCode\)](#)

链接 2：[146. LRU 缓存 - 力扣 \(LeetCode\)](#)

描述：设计和构建一个“最近最少使用”缓存，该缓存会删除最近最少使用的项目。

核心思路：使用双链和哈希表。双向链表可以用来表征使用的先后顺序，哈希表可以用来 O(1) 时间复杂度快速查询。

补充知识点 1：size 没有初始化的时候，系统默认初始化一个值，这个值依系统而定，通常情况下不是 0，是一个随机值。

补充知识点 2：定义函数的时候，`func(Node *p)` 和 `func(Node * &p)` 是有本质区别的，前者传递的是指针的 copy，对于指针的取值是一样的，但是指针本身是不一样的；后者是指针本身，可能会不小心修改到指针本身，导致意料之外的错误。

比如，`func(tail->pre)` 对于双链 “`head <-> A <-> B <-> tail`”，对于指令 `node->next->pre = node->pre`，传递指针 copy，正常实现将 `tail->A`，但是传递指针本身，会让 `tail->pre = node->pre`，然后在代码里面再删除 B，那么 `tail->pre` 指向就没有了，就成野指针了。

参考思路：

定义一个双向链表的数据结构，定义一个哈希表。定义一个删除双向链表里面元素的函数，定义一个在双向链表头部添加元素的函数，利用前面两个函数能够定义出将指定元素移动到双链首部的函数。

细节：双向链表需要定义 head 和 tail 两个节点，能使得链表节点的操作更具备通用性。

3.3 链表

一、单链表

描述：单向链表。

❖ 案例：链表局部反转

时间：2024.8.26 40+20+10=70min

链接：[链表内指定区间反转 牛客题霸 牛客网 \(nowcoder.com\)](#)

描述：反转链表局部[m,n]。

思路：

设定一个哑节点。

链表局部元素调换。

关键代码：链表局部元素调换。

mfront 为 m 前一个位置，mnow 为 m 当前位置，mback 为 m 后一个位置

```

// 反转 m 到 n 之间的节点
for (int i = 0; i < n - m; i++) {
    //细节：顺序不能变，先局部，再整体。
    mnnow->next = mback->next;
    mback->next = mfront->next;//原来错误代码:mback->next=mnnow,错误原因:更迭之后 mnnow
    不是 mfront 直接后面的元素
    mfront->next = mback;
    mback = mnnow->next;
}

```

❖ 案例：自定义排序。

链接：[..\数据结构测试.大二上\链表\list-sort.cpp](#)

说明：sort()函数，STL 中 list 专属的排序函数。

学习目标：学会按照结构体的关键字进行排序。

大致思路：

- 1.在定义结构体的时候重新定义">"和"<";
- 2.在 sort()的参数里面输入函数适配器。

关键代码：

定义结构体：

```
struct Student{
```

```
    char name[10];
```

```
    double total;
```

```
    public:
```

```
        //第一个关键：这里重定义的排序函数是关键，这里是重新定义的排序函数，
```

"<"是升序，">"降序排列。

```
        bool operator>(const Student &s)const{
```

```
            return total>s.total;
```

```
        }
```

```
};
```

定义链表：

```
list<Student> st(stu,stu+5);
```

结合函数适配器来使用 sort 函数。

```
st.sort(greater<Student>());
```

❖ 案例：链表合并-允许链表元素重复。

链接：[..\数据结构测试.大二上\链表\list-重复合并.cpp](#)

描述：允许原链表元素重复。

编码要求：将两个非递减的链表合并为一个非递增的链表。并且不创建新空间。允许有重合的元素。

编码思路：建表的时候用尾插法，非递减的。合并用头插法，先确定一个头节点 c，再比较，用头插法一个一个比较再插入。

知识点：链表创建：头插法 Vs 尾插法。

细节：见代码中，`c->next=NULL`；不然会陷入死循环。

补充：经测试，“！”的优先级高于“%”

代码说明：`if(!(cnt%4)) i--`

❖ 衍生：链表合并-不允许链表元素重复。

链接：[..\数据结构测试.大二上\链表\一元多项式的相加.cpp](#)

描述：在上面题目的基础上，不允许有重复到的元素。

编码思路：

s1.排除自身的重复元素。

关键代码：

```
while((a->next)&&(a->num==a->next->num)){//将自己链表里面重复到的删去。  
    p=a;//这里需要特别注意需要判断 a->next,不然代码不会正常运行，虽然不会报错。  
    a=a->next;  
    delete p;}
```

s2.排除另外一个链表中的重复元素。

关键代码：

```
(当两个链表元素相等的时候)  
else{//这里加了一步删除两链表中相同元素的代码。  
    p=b;  
    b=b->next;//把该 b 的这个节点保留下来  
  
    list *q=a;  
    a=a->next; //把 a 的这个节点保留下来  
  
    p->next=c->next;//这里单独把 c 作为了头结点的  
    c->next=p;  
    delete q;  
}
```

s3.把多余的部分借到末尾。

❖ 案例：【模板】链表_牛客题霸_牛客网 (nowcoder.com)

说明：模拟链表。

关键思路：关键还是需要明白指针的意义。

一个 node 结构体

数据域、指针域

一个 list 结构体

头节点（细节：区分头节点、首节点；额外加一个头节点能统一处理细节）

`find()`返回找到的元素上一个节点

`insert()`

`delete()`

❖ 相交链表

时间：2026.4.24 5min（一次性）

链接：[160. 相交链表 - 力扣 \(LeetCode\)](#)

描述：两个链表，找链表相交的位置。

思路 1-直白：固定一个链表，移动另一个链表一一比对，时间复杂度为 $O(m*n)$

思路 2-哈希：将一个链表存入哈希表或者集合，遍历另外一个列表，如果另外一个列表元素存在则返回该元素，否则返回空。

知识点：map 和 set 使用 insert 插入元素。

❖ 反转链表

时间：2026.4.24 25min（看了解析）

链接：[206. 反转链表 - 力扣 \(LeetCode\)](#)

思路：存储 pre 指针，遍历链表，挨个反转。

❖ 回文链表

时间：2026.4.24 10min（看了解析）

链接：[234. 回文链表 - 力扣 \(LeetCode\)](#)

思路-直白：将链表转成数组。

❖ 环形链表

时间：2026.4.24 10min（一次性）

链接：[141. 环形链表 - 力扣 \(LeetCode\)](#)

思路：使用一个集合即可。

✧ 衍生：环形链表 2

时间：5min（一次过）

链接：[142. 环形链表 II - 力扣 \(LeetCode\)](#)

思路：跟上面的一模一样，使用一个集合即可。

❖ 合并两个有序链表

时间：2026.4.26 10min（一次过）

链接：[21. 合并两个有序链表 - 力扣 \(LeetCode\)](#)

思路：定义一个头节点即可，谁小放入谁，末尾收个尾。

❖ 两数相加

时间：2026.4.26 30min（一次过）

链接：[2. 两数相加 - 力扣 \(LeetCode\)](#)

思路：按照竖式加法的计算规则计算即可。

❖ 删除链表的倒数第 N 个节点

时间：2026.4.26 10min（一次过）

链接：[19. 删除链表的倒数第 N 个结点 - 力扣 \(LeetCode\)](#)

思路：先遍历一遍链表，确定有几个节点，然后就能确定倒数第几个节点了。

❖ 两两交换链表中的节点

链接: [24. 两两交换链表中的节点 - 力扣 \(LeetCode\)](#)

时间: 2026.4.26 20min (一次过)

思路启发: 选择三个节点模拟一下交换, 很快就行了。

思路: 记录三个节点, 当前节点, 前一个节点, 后一个节点, 然后进行两两交换即可。

❖ 随机链表的复制

时间: 2026.9.14 20min 一次过;

链接: [138. 随机链表的复制 - 力扣 \(LeetCode\)](#)

思路启发: 仔细读题, 读完大致就知道算法了。

大致的思路: 肯定要先创建一个单链; 然后再额外循环一次去赋值 random, 这次循环里面还要再加一个循环去找 random, 找到了再复制。

❖ K 个一组翻转链表

时间: 2026.9.15 40min 一次过, 不过因为边界条件稍微卡了一会儿。

链接: [25. K 个一组翻转链表 - 力扣 \(LeetCode\)](#)

思路 1 (因为边界条件卡了一下): 选定一个长度为 k 的链表区间, 使用头插法进行反转。

解决卡点的一般思路: 一般带入一个案例在稿纸上推演一遍, 但是没有发现问题所在, 然后打印了关键变量 cnt, 发现 cnt 变量维护错误, 然后发现 “if(q && q->next)” 少了一个判定条件 q->next, 因为如果有 q, 仍然可以执行 q=q->next, 但是如果 q 是最后一个元素, q->next 应该是 null, 不该被纳入 cnt 计算。

二、循环链表

定义: 循环链表。

❖ 案例: [..\数据结构测试.大二上\链表\3559. 围圈报数.cpp](#)

描述: 报圈圈游戏。

写作思路:

s1. 先正常创建链表。注意预留一个头节点。

s2. 再将尾指针链接到首节点, 删除头节点。区别于头节点。

这样做: 好处在于能够先顺利创建链表, 再自由实现循环链表, 不需要考虑头结点数据无意义的情况。

s3. 开始游戏即可。

参考答案: 当输入 m=39 时, 活下来的人是 10 和 25

三、双向链表

定义: 双向链表。

❖ 案例: [..\数据结构测试.大二上\链表\双向链表.cpp](#)

说明: 包含双向链表的创建、增加、删除。

编码目的: 模拟双向链表。主要实现增、删操作。

细节: 1. 要从首元节点开始打印。

2.先让 pos-1

开始:

在草稿纸上画一下图就可以了。

四、双向循环链表

作用: 顾名思义, 类比单项循环链表。

❖ 案例:

时间: 22.9.18

编码目的: 模拟双向链表。主要实现增、删操作。

经试验, 自感: 因为 head 是全局的变量, 在函数中执行的时候不能修改其指向的 (即不能修改其值)。所以这个循环链表就像是一个铁环, 但是多了一小铁枝, 这个小枝即是 head, 在应用的时候进入循环之后就可以良性循环了。

开始:

定义好结构体, 然后画图即可。

3.4 栈和队列

一、基础理论

背景: STL(C++标准库)是有很多不同版本的, HP STL、SGI STL、P.J.Plauger 等版本。各种版本底层实现略有不同。

1. C++中 stack 是容器么?

不是容器, 是容器适配器 (container adapter)。栈是以底层容器完成其所有的工作, 对外提供统一的接口, 底层容器是可插拔的 (也就是说我们可以控制使用哪种容器来实现栈的功能)。

2. 我们使用的 stack 是属于哪个版本的 STL?

SGI STL。由 Silicon Graphics Computer Systems 公司参照 HP STL 实现, 被 Linux 的 C++ 编译器 GCC 所采用, SGI STL 是开源软件, 源码可读性甚高。

3. 我们使用的 STL 中 stack 是如何实现的?

deque 是一个双向队列, 只要封住一段, 只开通另一端就可以实现栈的逻辑了。SGI STL 中 队列底层实现缺省情况下一样使用 deque 实现的。

4. stack 提供迭代器来遍历 stack 空间么?

不提供, 栈提供 push 和 pop 等等接口, 所有元素必须符合先进后出规则, 所以栈不提供走访功能, 也不提供迭代器(iterator)。不像是 set 或者 map 提供迭代器 iterator 来遍历所有元素。

特点: 先进后出。

✧ 面试题: 栈里面的元素在内存中是连续分布的么?

这个问题有两个陷阱:

陷阱 1: 栈是容器适配器, 底层容器使用不同的容器, 导致栈内数据在内存中不一定是连续分布的。

陷阱 2: 如果采用 vector, 则在内存中是连续存储的。缺省情况下, 默认底层容器是 deque, 那么 deque 在内存中的数据分布是不连续的。deque (双端队列) 是一种可以在两端快速插

入和删除元素的容器。它的内存分配方式是将数据分成多个块，每个块内部是连续的，但不同块之间的内存地址是不连续的。这种设计使得 `deque` 在两端操作时效率很高，但也导致了其整体数据在内存中不是连续分布的。

❖ 案例：[..\数据结构测试.大二上\模拟栈\模拟栈.cpp](#)

描述：通过结构体和模板来模拟栈。主要实现的功能：创建，消除，清空，判断是否为空，增（入栈），删（出栈）。

大致思路：

- 1.创建栈：分配空间。
- 2.消除栈：释放空间。调整指针。
- 3.清空：调整指针。
- 4.判空：判断指针相对位置。
- 5.增删：在栈顶处理，先进后出。

3.4.1 表达式求值

❖ 案例：[..\数据结构测试.大二上\表达式求值\后缀表达式求值 2.cpp](#)

时间：22.12.15

描述：用了秦九韶算法。采用栈实现的。

1. 中缀表达式求值

说明：中缀表达式计算数值。分为有括号的、没括号的，计算方式一摸一样。

❖ 案例 2024.11.3：[..\数据结构测试.大二上\表达式求值\中缀表达式求值.cpp](#)

说明：直接在中缀转后缀里面修改的，只改了一点点。这个可以实现多位数的运算。

细节：在 `if-else` 语句里面不要加 `i++`，不然 `i` 每次判断的时候都会加。

❖ 案例：[中缀表达式求值，含括号。](#)

时间：2024.9.1 60min 重温

链接：[表达式求值_牛客题霸_牛客网 \(nowcoder.com\)](#)

思路：[遍历字符串，依据字符性质作不同判断。](#)

初始化一个操作符栈 `ops`；一个数字栈 `data`。

额外定义三个函数：秦九韶算法求数字，优先级函数，操作计算函数。

遍历中缀表达式：

如果是数字，则读取数字。

如果是运算符，进行判断。

如果栈内栈外运算符优先级相等，则出栈+遍历下一个符号。

如果栈内优先级小于栈外优先级，则入栈+遍历下一个符号。

如果栈内优先级大于栈外优先级，则进行运算。

关键代码：主体部分。

思路：遍历字符串，如果是数字则入数字栈，如果是符号，则根据优先级判断处理办法。

如果栈外优先级大，则入栈；小，则计算；等于，则消除。

经验：变量取名跟关键字一样会报错。

```
stack<int> data;
stack<char> ops;
ops.push('#');//加#号，确保 ops 有内容，不需要特判最后一个符号的情况
s=s+'#';
int i=0;
while(i<s.size()){
    if(isdigit(s[i])){
        data.push(number(s,i));
    }else{
        if(priority(s[i],"out")>priority(ops.top(),"in")){
            ops.push(s[i++]);
        }else if(priority(s[i],"out")<priority(ops.top(),"in")){
            char op=ops.top();ops.pop();
            int b=data.top();data.pop();
            int a=data.top();data.pop();
            int ans=operation(op,a,b);
            data.push(ans);
        }else{//内外优先级相等，只有当#和#、（和）相遇时才有
            ops.pop();
            i++;
        }
    }
}
```

代码：优先级设计

优先级设计理念：遍历数组。乘除运算优先级大于加减运算；遇到相同运算符，先计算栈内的；遇到括号要入栈；遇到反括号要计算括号内元素；反括号遇到括号要弹出括号。

```
int priority(char op,string inOut){
    if(inOut=="in"){
        if(op=='+'||op=='-') return 3;
        else if(op=='*'||op=='/') return 5;
        else if(op=='#') return 0;
        else if(op=='') return 6;
        else if(op=='(') return 1;
    }else{//inOut=="out"
        if(op=='+'||op=='-') return 2;
        else if(op=='*'||op=='/') return 4;
        else if(op=='#') return 0;
        else if(op=='') return 1;
        else if(op=='(') return 6;
    }return -1;
}
```

衍生 2024.9.1: [表达式求值，不包含括号](#)。

链接: [计算器\(二\)_牛客题霸_牛客网 \(nowcoder.com\)](#)

说明: 解法跟上面的一摸一样。

2. 后缀表达式求值

说明: 后缀表达式求值, 又叫逆波兰式求值。

思路:

逆波兰表达式严格遵循「从左到右」的运算。计算逆波兰表达式的值时, 使用一个栈存储操作数, 从左到右遍历逆波兰表达式, 进行如下操作:

如果遇到操作数, 则将操作数入栈;

如果遇到运算符, 则将两个操作数出栈, 其中先出栈的是右操作数, 后出栈的是左操作数, 使用运算符对两个操作数进行运算, 将运算得到的新操作数入栈。

整个逆波兰表达式遍历完毕之后, 栈内只有一个元素, 该元素即为逆波兰表达式的值。

● 案例: 逆波兰式求值。

时间: 2025.2.13 , 40min

链接: [150. 逆波兰表达式求值 - 力扣 \(LeetCode\)](#)

3.4.2 单调栈、单调队列

一、单调队列

描述: 队列单调增(减), 队首元素即是最大(小)值。

● 案例: 滑动窗口。

时间: 2025.2.16, 60min; 2026.4.14 65min

链接: [239. 滑动窗口最大值 - 力扣 \(LeetCode\)](#)

方法 1: 使用大顶堆, 见本文堆相关位置。

方法 2: 使用单调队列。(手写笔记见 pad)

说明: 维系三个方法。

`push()`: 往队列里面加元素 `value`, 如果 `value` 大于队尾元素, 则删除队尾元素。如果 `value` 小于队尾元素, 则正常添加。(采用 `while` 进行判断)

`pop()`: 删除队列中元素, 只有当 `value == 队首元素` 时, 才会 `pop` 队首元素, 其他条件不做操作。(调用单调栈的时候, 先 `push` 再 `pop`, 所以, 如果 `value < 队首元素`, 则说明在 `push` 的时候 `value_push` 已经挤占到队首来, 成为了新的最大值。故而不需要操作; 不会存在 `value < 队首元素` 的情况。)

`front()`: 默认是最大值。(return `que.front()` 即可)

关键代码:

采用双端队列 `deque`。(栈和队列底层默认情况下也是双端队列作为底层容器实现的。)

```
class increase_queue{
    deque<int> que;
public:
```

```

//先 push 再 pop
void push(int value){
    while(!que.empty() && value > que.back()){
        que.pop_back();
    }
    que.push_back(value);
}

void pop(int value){
    if(!que.empty() && value == que.front()){
        que.pop_front();
    }
}

int front(){
    return que.front();
}

};

```

时间：2026.4.14 65min 反复刷了之前的攻略做出来，最开始没有意识到这是单调栈的题目。

二、单调栈

基础，单调增：栈内存脚标。

判断栈顶元素和当前元素：

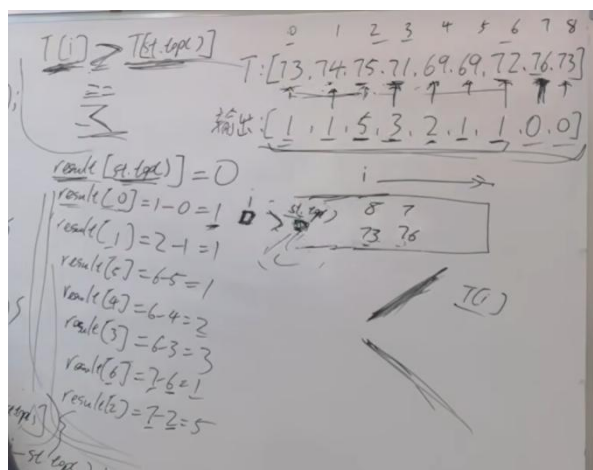
如果 $st.top()$ 小于当前元素：入栈。

如果 $st.top()$ 等于当前元素：入栈。

如果 $st.top()$ 大于当前元素：计算脚标插值，出栈。

单调栈适用领域：数组中求当前元素左边、右边第一个最大（最小）元素。

视频链接：[单调栈，你该了解的，这里都讲了！LeetCode:739.每日温度_哔哩哔哩_bilibili](#)



❖ 案例：每日温差

链接：[739. 每日温度 - 力扣 \(LeetCode\)](#)

时间：2024.9.4, 60min, 2025.5.17 二刷 20min

思路核心：栈存放的是数组脚标。

关键代码：

核心知识点：单调栈。

```
vector<int> dailyTemperatures(vector<int>& temperatures) {
    stack<int> st;//存放数组的脚标
    vector<int> ans(temperatures.size(),0);//记录差
    st.push(0);//初始化
    for(int i=1;i<temperatures.size();i++){
        if(temperatures[st.top()]>=temperatures[i]){
            st.push(i);
        }else{//temperatures[st.top()]<=temperatures[i]
            while(!st.empty()&&temperatures[st.top()]<temperatures[i]){
                ans[st.top()]=i-st.top();
                st.pop();
            }st.push(i);
        }
    }
    return ans;
}
```

衍生：如果数组是循环的。

时间：2024.9.4，60min

链接：[503. 下一个更大元素 II - 力扣（LeetCode）](#)

关键思路：将数组重复一遍，以达到循环数组的目的。

❖ 案例：最大矩形。

时间：2024.9.4 30min

链接：[84. 柱状图中最大的矩形 - 力扣（LeetCode）](#)

关键知识点：单调栈。

关键代码：

思维启发：思考怎样计算矩形面积，随后就能想出需要找左右最近小于的元素，从而想到需要使用单调栈。

```
int largestRectangleArea(vector<int>& heights) {
    if(heights.empty()) return -1;//特判
    heights.insert(heights.begin(),0);//首插零，针对第一个元素
    heights.push_back(0);//尾插零针对递减序列
    stack<int> st;
    int maxMost=0;
    st.push(0);
    for(int i=1;i<heights.size();i++){
        if(heights[i]>heights[st.top()]){
            st.push(i);
        }else if(heights[i]==heights[st.top()]){

```

```
        st.pop();
        st.push(i);
    }else{//heights[i]<heights[st.top()]
        while(!st.empty()&&heights[i]<heights[st.top()]){
            int mid=st.top();
            st.pop();
            if(!st.empty()){
                int h=heights[mid];
                int w=i-st.top()-1;
                maxMost=max(maxMost,h*w);
            }
        }
    }
    st.push(i);
}
return maxMost;
}
```

【类比】接雨水。

时间：2024.9.4 120min；2026.4.13 50min（有看解析，问豆包）

链接：[42. 接雨水 - 力扣（LeetCode）](#)

思路：跟上面题目一摸一样。

思维其他：思考怎样计算承接水的量，随后就能抽象出需要找到左右两边最近最大的矩阵，从而想到单调栈求解。

一、应用案例

1. 进制转换

❖ 案例：[..\数据结构测试.大二上\模拟栈\进制转换.cpp](#)

描述：进制转换。

时间：22.9.25

细节：1.这里一定注意，用指针，不然对原栈的修改是无效的。

2. 其他

❖ 案例：[..\数据结构测试.大二上\链表\递归.汉诺塔.cpp](#)

描述：栈+递归。函数的递归过程通过栈的数据结构来实现的。

说明：这是一个标准的动态规划问题。

关键代码：

```
if(n==1) move(a,c);//推出的位置
else{//递归部分。
    hanoi(n-1,a,c,b);
    move(a,c);
}
```

```
    hanoi(n-1,b,a,c);  
}
```

- 案例：判断一个序列是否是入栈之后出栈的结果。

链接：[..\机试准备\选球问题.cpp](#)

思路：

初始：栈、入栈序列、出栈序列

将入栈序列正常入栈，出栈序列与栈顶元素对比

如果相等，出栈序列指针后移，出栈，循环判断后移后的字符是否与当前栈相等

如果不相等，下一次循环

关键代码：

```
sta.push(in[i++]);  
while(!sta.empty() && sta.top() == s[j]) { // 需要判断栈是否为空  
    sta.pop();  
    j++;  
}
```

细节：注意栈中元素是否为空。若为空，会陷入死循环。

- 案例：双栈模拟队列。

说明：用双栈模拟队列， $O(1)$ 时间复杂度。

时间：2025.2.11

链接：[232. 用栈实现队列 - 力扣 \(LeetCode\)](#)

思路：单独用一个栈作为输入栈，一个栈作为输出栈。

- 案例：队列模拟栈。

时间：2025.2.11

说明：用队列来模拟栈。可以使用两个队列，也可使用一个队列。

链接：[225. 用队列实现栈 - 力扣 \(LeetCode\)](#)

方法 1：两个队列。一个为入队，一个为出队。入队之后，出队中元素放入出队后面，交换出入队。

方法 2：一个队列。入队之后，将队首元素依次放入队尾（提前记录需要放入 n 次）。

- 案例：括号匹配

时间：2025.2.12

链接：[20. 有效的括号 - 力扣 \(LeetCode\)](#)

细节：对于 stack 在获取 top 或者 pop 操作的时候，需要首先判断是否为 empty。

1. 循环队列

定义：循环队列。

案例：[【模板】循环队列_牛客题霸_牛客网 \(nowcoder.com\)](#)

描述：模拟循环队列。

关键思路：

结构体：

有 int[]、front、rear、length 属性
有 push、pop、front_、isEmpty 函数。

关键判断：

判断队列是否空：front==rear

判断队列是否满：(rear+1)%length==front

pop 实现：front=(front+1)%length

3.5 二叉树

二叉树种类：满二叉树、完全二叉树、二叉搜索树、平衡二叉树（map、multimap、set、multiset）。

存储方式：链式存储、线性存储（已知该节点 i，左节点等于 2i+1，右节点等于 2i+2）。

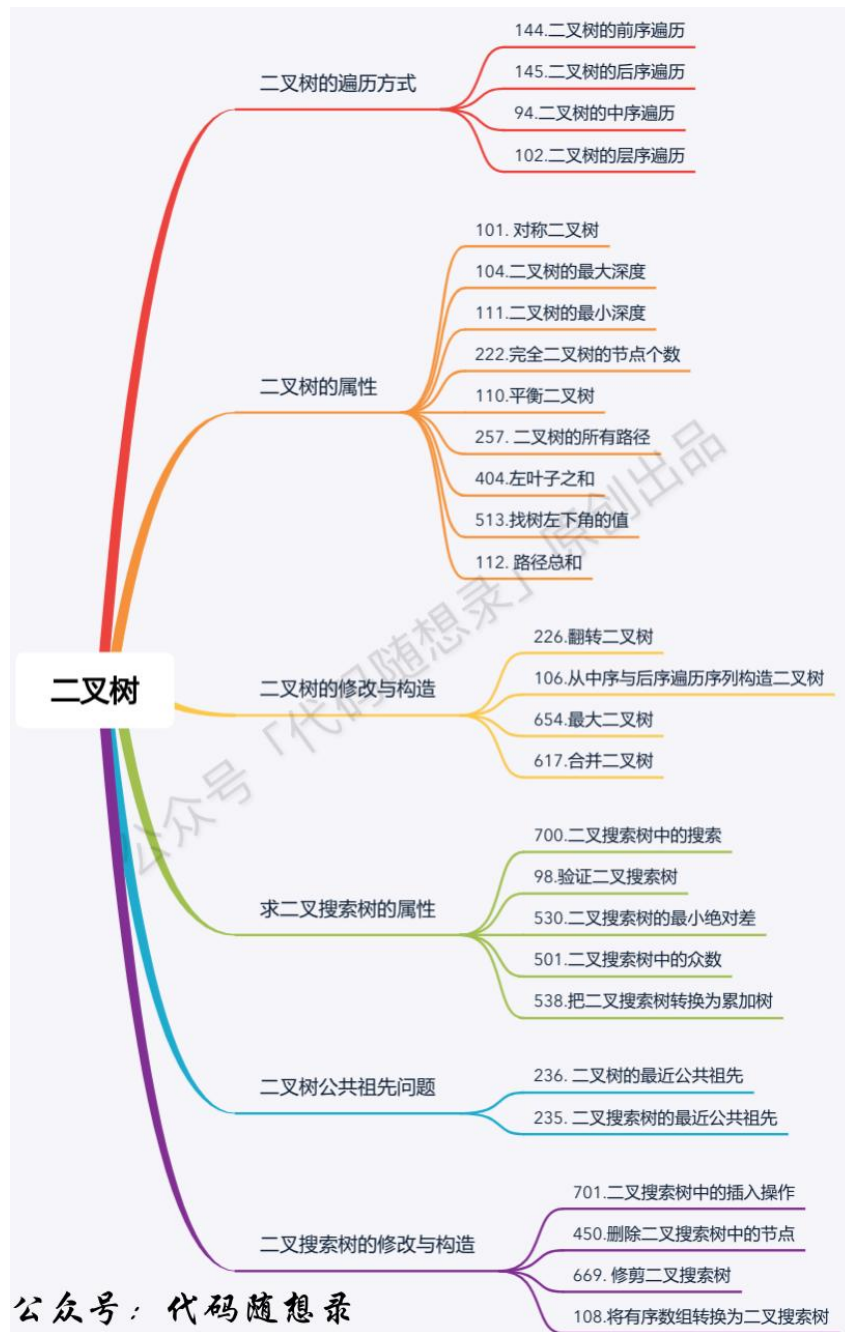
二叉树遍历：

深度优先搜索：前、中、后序遍历。

广度优先搜索：层次遍历。

定义方式

```
struct BiTree{  
    int val;  
    BiTree *left;  
    BiTree *right;  
};
```



图：二叉树重要知识点

心得：二叉树部分，最关键的知识点只有三个。2024.7.30

1. 创建：已知前序和中序；已知中序和后序。
2. 遍历：前序；中序；后序；层次。递归遍历与迭代遍历。
3. 序列化：

递归三要素：递归参数和返回值，递归终止条件，递归体。

理解：二叉树的递归包括了两个部分，第一个部分是递归框架需要递归（前、中、后序遍历），第二个部分是递归逻辑需要递归（比如对左、右子树也进行裁剪）。对于每一个节点的操作应该是一模样的，通过条件判断来实现对不同性质的节点有不同类型的操作。

```

void build(Tree &root,vector<int> &inorder,vector<int> &postorder,int inLeft,int inRigt,int
postLeft,int postRight){
    if (inLeft>inRigt) {
        return;
    }else {//如果没有到达底部，那么进行递归创建
        root=new TreeNode(postorder[postRight]);//先把节点创建好
        //通过后续序列去找先序序列的中间划分;细节：从左边开始去定位方便一些
        int mid = inLeft;
        while (inorder[mid] != postorder[postRight]) {
            ++mid;
        }
        int leftSize = mid - inLeft;
        build(root->left, inorder, postorder, inLeft, mid - 1, postLeft, postLeft + leftSize - 1);
        build(root->right, inorder, postorder, mid + 1, inRigt, postLeft + leftSize, postRight - 1);
    }
}

```

✧ 二刷，时间：2025.2.24，25min

链接：[106. 从中序与后序遍历序列构造二叉树 - 力扣 \(LeetCode\)](#)

细节：递归函数中，需要传递指针的引用。

❖ 案例：已知前序和中序创建二叉树。

链接：[二叉树遍历_牛客题霸_牛客网 \(nowcoder.com\)](#)

时间：2024.9.21；20min

关键代码：

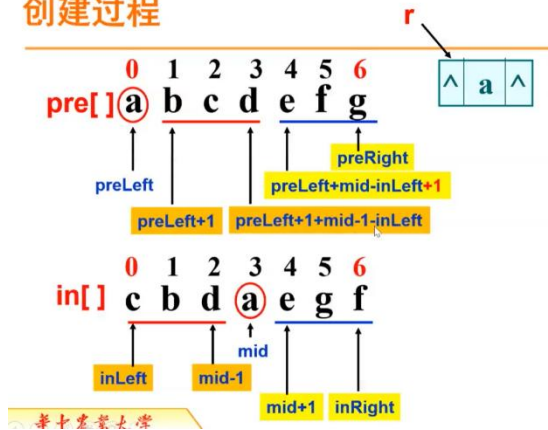
```

void create(node*&root,int preLeft,int preRight,int midLeft,int midRight,string &preStr,string
&midStr){
    if(midLeft>midRight) return;//细节：递归窗口定义掉了
    root=new node(preStr[preLeft]);
    int mid=midLeft;
    while(preStr[preLeft]!=midStr[mid]) mid++;
    int size=mid-midLeft;
    create(root->left,preLeft+1,preLeft+size,midLeft,mid-1,preStr,midStr);
    create(root->right,preLeft+size+1,preRight,mid+1,midRight,preStr,midStr);
    return;
}

```

补充：

创建过程



```
else
{
    // 二叉树有结点,非空二叉树
    r = (BiTree)malloc(sizeof(BiTreeNode));
    r->data=pre[preLeft]; // 生成根结点
    int mid = inLeft;
    while (in[mid] != pre[preLeft])
    {
        // 查找pre[preLeft]在in[]中的位置
        // 也就是中序序列中根的位置
        mid++;
    }
    CreateBinaryTree(r->leftChild, pre, in,
        preLeft+1, preLeft + mid - inLeft,
        inLeft, mid - 1); // 生成左子树
    CreateBinaryTree(r->rightChild, pre, in,
        preLeft + mid - inLeft + 1, preRight,
        mid + 1, inRight); // 生成右子树
}
```

❖ 案例：..\数据结构测试,大二上\二叉树\二叉树的创建与遍历_final.cpp

描述：二叉树的创建与遍历。

时间：22.11.15

细节：用引用进行参数的传递，这样才能实现创建。不然用指针的时候传入的是指针，是临时变量，函数结束之后就消失了。

知识点：

- 1.掌握二叉树的前、中、后序创建。
- 2.掌握二叉树的递归的前、中、后序遍历。
- 3.掌握二叉树的非递归前、中、后序遍历。

//测试样例：423##7##5#8##

关键代码：二叉树创建。

描述：这里以前序创建为例子。前中后三序遍历的区别在于 data 赋值的位置不一样。

void pre_cre(tree &bt){//细节，这里用引用传递参数，不然无法修改外部指针的指向。

char ch=getchar();//这里采用了前序创建二叉树，把输入 data 的顺序换换就可以

if(ch!='#'){//相应的变成中序和后续创建二叉树

bt=new Tree;

bt->data=ch;

pre_cre(bt->lchi);

pre_cre(bt->rchi);

}else{

bt=NULL;

}

}

关键代码：

有点困难，需要学习的时候查看原码。

❖ 案例：最大二叉树。

时间：2025.2.24，10min

链接：[654. 最大二叉树 - 力扣 \(LeetCode\)](https://leetcode.cn/problems/maximum-binary-tree/)

描述：递归创建（有点像前序遍历进行创建）。

3.5.2 二叉树遍历

关键知识点：二叉树前、中、后序遍历（递归、迭代、迭代的统一方式），层次遍历。

建议掌握：二叉树前、中、后序遍历的递归形式，迭代统一形式；层次遍历。

一定要掌握前中后序一种迭代的写法，并不因为某种场景的题目一定要用迭代，而是现场面试的时候，面试官看你顺畅的写出了递归，一般会进一步考察能不能写出相应的迭代。

一、前序遍历

定义：根左右。

❖ 案例：二叉树前序遍历。

时间：2025.2.19, 10min; 2026.3.27, 10min。

链接：[二叉树的前序遍历_牛客题霸_牛客网 \(nowcoder.com\)](#)

✧ 思路 1：递归，根左右。类似的代码见中序遍历-递归。

✧ 思路 2：迭代遍历。

细节：这里入栈顺序是右左，以保证出栈的时候是左右。

```
stack<TreeNode*> TreeStack;
vector<int> results;
if(root == NULL) return results;
TreeStack.push(root);
while(!TreeStack.empty()){
    TreeNode *cur=TreeStack.top();
    results.push_back(cur->val);
    TreeStack.pop();
    if(cur->right) TreeStack.push(cur->right);
    if(cur->left) TreeStack.push(cur->left);
}
```

✧ 思路 3：迭代遍历。统一风格，见中序遍历思路 3，只需要调换顺序即可。

说明：这里入栈顺序根右左，出栈的时候即根左右。

```
st.push(make_pair(node,true));
if(node->right) st.push(make_pair(node->right,false));
if(node->left) st.push(make_pair(node->left,false));
```

❖ 案例：二叉树所有路径。

时间：2025.2.22, 20min

链接：[257. 二叉树的所有路径 - 力扣 \(LeetCode\)](#)

描述：前序遍历（确保从根节点出发）。然后记录路径。

特别说明：这里使用到了回溯。在这里，对于 res 传递的是内容拷贝（非引用类型），所以递归完后回溯的之后 res 是上一轮的值。

```
void visit(TreeNode * root,string res,vector<string> &results){
    if(!root) return;
    if(res.empty()) res=res+to_string(root->val);
    else res=res+"->"+to_string(root->val);
    if(!root->left&&!root->right){//说明是叶子节点
        results.push_back(res);
    }
    if(root->left) visit(root->left,res,results);
    if(root->right) visit(root->right,res,results);
}
```

❖ 案例：左叶子之和。

时间：2025.2.23，15min

链接：[404. 左叶子之和 - 力扣 \(LeetCode\)](#)

思路：前序遍历。设定一个 bool 值来判断是否为左节点（往左递归的时候是左节点，再判断是否为叶子节点，即可判断出是否为左叶子节点）。

❖ 案例：路径总和

时间：2025.2.23，5min

链接：[112. 路径总和 - 力扣 \(LeetCode\)](#)

思路：前序遍历。通过参数拷贝传递实现回溯效果，记录总和 sum，当为叶子节点的时候判断一下是否和目标相等。

❖ 案例：二叉树中搜索

时间：2025.2.25，5min

链接：[700. 二叉搜索树中的搜索 - 力扣 \(LeetCode\)](#)

思路：前序遍历。

二、中序遍历

定义：左根右。

❖ 案例：二叉树中序遍历。

时间：2025.2.19，10min；2026.3.27，15min 统一思路的递归。

链接：[二叉树的中序遍历_牛客题霸_牛客网 \(nowcoder.com\)](#)

✧ 思路 1：递归遍历，左根右。

关键代码：

```
void inorder(TreeNode *root,vector<int> &ans){
    if(!root) return;
    inorder(root->left,ans);
    ans.push_back(root->val);
    inorder(root->right,ans);
}
```

```
}
```

✧ 思路 2：迭代遍历。

说明：代码风格和前序、中序不一样。这是因为前序遍历中访问节点（遍历节点）和处理节点（将元素放进 **result** 数组中）可以同步处理，但是中序就无法做到同步！

```
stack<TreeNode*> TreeStack;
vector<int> results;
if(root==NULL) return results;
TreeNode *cur=root;
while(cur||!TreeStack.empty()){
    if(cur){//cur 有内容，说明还没到叶子节点，还需要往左找
        TreeStack.push(cur);
        cur=cur->left;
    }else{//cur 是空，则说明到叶子节点了
        cur=TreeStack.top();
        TreeStack.pop();
        results.push_back(cur->val);
        cur=cur->right;
    }
}
```

✧ 思路 3：迭代遍历，弹出 top 元素，手动调整与左右元素的相对位置。跟前、后序统一遍历风格。

关键代码：采用 bool 值标记一个节点是否调整好了位置，位置调整到合适的遍历顺序，即可实现一边访问一边处理（放入结果数组）。

说明：对于前序和后序遍历，只需要调换 `st.push(make_pair(node,true))` 这句指令的位置即可。

```
stack<pair<TreeNode *,bool> > st;
vector<int> results;
if(!root) return results;
st.emplace(root,false);
while(!st.empty()){
    TreeNode * node = st.top().first;
    bool visited=st.top().second;
    if(visited){//visited==true 表示可以处理。
        results.push_back(node->val);
        st.pop();
        continue;
    }//visited==false 表示不能处理，需要手动调整位置。
    st.pop();
    if(node->right) st.push(make_pair(node->right,false));
    st.push(make_pair(node,true));
    if(node->left) st.push(make_pair(node->left,false));
}
```

❖ 案例：将二叉树改为累加二叉树。

时间：2024.8.29 60min，2025.3.2 二刷 10min

链接：[将二叉搜索树改为累加树 牛客题霸 牛客网 \(nowcoder.com\)](#)

二刷链接：[538. 把二叉搜索树转换为累加树 - 力扣 \(LeetCode\)](#)

思路：中序遍历的时候，采用“右根左”的方式访问，可以逆序访问。二叉搜索树正常中序遍历的时候是递增的，“右根左”的时候是递减的。

技巧：这里额外利用了 nowMAX 存储中间变量，同时可以用来处理最后一个元素。这样的处理逻辑可以挑出最后一个元素不参与计算。

```
void RinOrder(TreeNode *root,int &nowMax){
    if(!root) return;
    nowMax=max(root->val,nowMax);
    RinOrder(root->right,nowMax);
    if(nowMax>root->val){
        root->val+=nowMax;
        nowMax=root->val;
    }
    RinOrder(root->left,nowMax);
}
```

三、后序遍历

定义：左右根。

❖ 案例：二叉树后序遍历

时间：2025.2.18；2026.3.27 5min，递归。

链接：[145. 二叉树的后序遍历 - 力扣 \(LeetCode\)](#)

✧ 思路 1：递归遍历。类似的代码见中序遍历-递归。

✧ 思路 2：迭代遍历。

说明：与前序遍历风格类似。这里是根右左，然后颠倒一下编程左右根。

```
stack<TreeNode*> TreeStack;
vector<int> results;
if(root == NULL) return results;
TreeStack.push(root);
while(!TreeStack.empty()){
    TreeNode *cur=TreeStack.top();
    results.push_back(cur->val);
    TreeStack.pop();
    if(cur->left) TreeStack.push(cur->left);
    if(cur->right) TreeStack.push(cur->right);
}
reverse(results.begin(),results.end());
```

✧ 思路 3：迭代遍历。统一风格，见中序遍历思路 3，只需要调换顺序即可。

说明：这里入栈顺序右左根，出栈的时候为根左右。

```
if(node->right) st.push(make_pair(node->right,false));
if(node->left) st.push(make_pair(node->left,false));
st.push(make_pair(node,true));
```

❖ 案例：判断是否为平衡二叉树

时间：2025.2.22, 20min

链接：[110. 平衡二叉树 - 力扣 \(LeetCode\)](#)

思路：采用自底向上的递归，类似与后序遍历。

说明：在求最大高度的时候，也可以采用递归遍历的方式。

```
int visit(TreeNode * root,bool &flag){
    if(!root) return 0;
    int left=visit(root->left,flag);
    int right=visit(root->right,flag);
    if(abs(left-right)>1) flag=false;
    return max(left,right)+1;
}
```

❖ 案例：找最近公共祖先。

链接：[二叉搜索树的最近公共祖先_牛客题霸_牛客网 \(nowcoder.com\)](#)

链接 2：[在二叉树中找到两个节点的最近公共祖先_牛客题霸_牛客网 \(nowcoder.com\)](#)

时间：2024.7.30；40+40+10=90min

说明：两个链接本质上是一样的。

思路：后序遍历。

递归窗口：如果当前节点是目标节点，则返回自己（自己也算是自己的祖先）。

递归公式：若不能找到，则递归查找左右子树。

关键代码：

说明：递归三要素：递归参数，终止条件，递归条件。

```
TreeNode * findAncestor(TreeNode* root,int p,int q){//采用了先序遍历，找最近
    if(!root||root->val==p||root->val==q) return root;//如果当前节点为所求。终止条件。
    TreeNode *left=findAncestor(root->left,p,q);
    TreeNode *right=findAncestor(root->right,p,q);
    if(left&&right){//如果 left 和 right 都有的话，那么返回当前节点，因为当前节点是
        return root;
    }else{
        return left? left:right;//返回左右中不是空的那边，如果都是空则会返回空
    }
}
```

✧ 三刷，时间：2025.2.26, 60min+；五刷，2026.4.24 40min（提前看了题解）

链接：[236. 二叉树的最近公共祖先 - 力扣 \(LeetCode\)](#)

思路：跟如上思路一模一样。

1) 求最小公共祖先，需要从底向上遍历，那么二叉树，只能通过后序遍历（即：回溯）实

现从底向上的遍历方式。

- 2) 在回溯的过程中，必然要遍历整棵二叉树，即使已经找到结果了，依然要把其他节点遍历完，因为要使用递归函数的返回值（也就是代码中的 `left` 和 `right`）做逻辑判断。
- 3) 要理解如果返回值 `left` 为空，`right` 不为空为什么要返回 `right`，为什么可以用返回 `right` 传给上一层结果。

可以说这里每一步，都是有难度的，都需要对二叉树，递归和回溯有一定的理解。明确如下问题：

- 1) 如何从底向上遍历？->采用后序遍历（实质是自顶向下），处理得时候自实现自底向上的效果。
- 2) 遍历整棵树，还是遍历局部树？->是的，即使中途找到了节点。
- 3) 如何把结果传到根节点的？->将结果一层层的往上回溯。

说明：后序遍历（实现由底向上的遍历效果，本质还是自顶向下），回溯效果（递归函数返回值，层层往上返回）。

递归三要素：递归参数，终止条件，递归体逻辑。

```
TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
    if(root==NULL) return NULL; //终止条件 1
    if (root == q || root == p) return root; //终止条件 2
    TreeNode* left = lowestCommonAncestor(root->left, p, q);
    TreeNode* right = lowestCommonAncestor(root->right, p, q);
    if (left != NULL && right != NULL) return root;
    if (left == NULL && right != NULL) return right;
    else if (left != NULL && right == NULL) return left;
    else{ // (left == NULL && right == NULL)
        return NULL;
    }
}
```

✧ 四刷，时间：2025.2.27，25min

链接：[235. 二叉搜索树的最近公共祖先 - 力扣 \(LeetCode\)](https://leetcode.cn/problems/lowest-common-ancestor-of-a-binary-tree/)

关键思路：因为二叉搜索树是有规律的，如果 `p`、`q` 都比 `root` 大，则都在 `root` 右边，反之 `p`、`q` 都比 `root` 小，则都在左边。如果 `p`、`q` 与 `root` 出现 `p` 小于等于 `root` 且 `q` 大于等于 `root` 的情况，则是最近公共父节点（在这里最后一个判断条件 `else` 包含）。

关键代码：

```
TreeNode* ancestor = root;
while (true) {
    if (p->val < ancestor->val && q->val < ancestor->val) {
        ancestor = ancestor->left;
    }
    else if (p->val > ancestor->val && q->val > ancestor->val) {
        ancestor = ancestor->right;
    }
    else {
        break;
    }
}
```

四、层次遍历

定义：图论中的广度优先搜索在二叉树中的应用，需要借助队列来实现（此时又发现队列的一个应用了）。

感悟：判断如果是层次遍历，代码风格大概率满足层次遍历的框架（**队列，一边遍历一边处理**），不同题目，处理模块可能略有不同。

❖ 案例：二叉树层次遍历。一如下代码可以作为层次遍历模板代码。

时间：2025.2.20，30min。

链接：[102. 二叉树的层序遍历 - 力扣 \(LeetCode\)](#)

关键代码：

说明：使用队列。取队列大小作为当前层节点的数量。这个代码可以作为层次遍历的模版代码，除了可变部分，其他的部分都可以写成这样一样的。

核心：一边访问队首元素，一边处理（将队首元素左右孩子入队，其他题目要求的操作）。

vector<vector<int>> results; //可变，这里用来记录结果。

if(!root) return results;

queue<TreeNode*> que;

que.push(root);

while(!que.empty()){

 int size=que.size();

 vector<int> res; //可变，与结果相关。

 for(int i=0;i<size;i++){

 TreeNode *node=que.front();

 res.push_back(node->val); //可变，与结果相关。

 que.pop();

 if(node->left) que.push(node->left);

 if(node->right) que.push(node->right);

 }

 results.push_back(res); //可变，与结果相关。

}

❖ 案例：判断是否为完全二叉树。

链接：[判断是不是完全二叉树_牛客题霸_牛客网 \(nowcoder.com\)](#)

时间：2024.7.29；40+10+5=55min

思路：层次遍历，如果遍历节点为空，该节点之后还有非空节点，那么不是完全二叉树。

关键代码：

设定判断变量 end

bool isCompleteTree(TreeNode* root) {

 if(!root) return true;

 queue<TreeNode*> que;

 que.push(root);

```

bool end=false;//判断是不是最后一个元素
while(!que.empty()){
    TreeNode* now=que.front();
    que.pop();
    if(!now) end=true;
    if(end&&now) return false;
    if(!end&&now){//如果没遇到空，并且 now 有数值，入队列
        que.push(now->left);
        que.push(now->right);
    }
}
return true;
}

```

❖ 案例：输出二叉树右边视图

时间：2024.7.30；30+5+20=55min，2025.2.20 二刷，5min。

链接：[199. 二叉树的右视图 - 力扣 \(LeetCode\)](#)

思路：采用层次遍历输出最右节点：同一层队列最后一个即是最右边的节点。

关键代码：

```

vector<int> rightSideView(TreeNode *root) {
    vector<int> rightView;
    if (root == nullptr) return rightView;
    queue<TreeNode*> que;
    que.push(root);
    while(!que.empty()){
        int size=que.size();
        TreeNode *current=nullptr;
        for(int i=0;i<size;i++){//把当前队列里面的元素都出队列
            current=que.front();
            que.pop();
            if(current->left) que.push(current->left);
            if(current->right) que.push(current->right);
        }
        rightView.push_back(current->val);//每层最后一个元素是右视图
    }
}

```

❖ 同类型题目：二叉树的最大宽度。

链接：[二叉树的最大宽度_牛客题霸_牛客网 \(nowcoder.com\)](#)

时间：2024.8.29 60min，2025.2.21 二刷 5min。

关键代码：

与上一题相似之处：通过 `que.size()` 获取该层元素数量，以定位层，这样就可以在同层里面进行各类运算。

```

int widthOfBinaryTree(TreeNode* root) {
    if(!root) return 0;

```

```
queue<TreeNode*> seq;
root->val=1;
seq.push(root);
int maxDepth=0;
while(!seq.empty()){
    int laySize=seq.size();
    int left=seq.front()->val;
    while(laySize--){
        TreeNode *now=seq.front();
        seq.pop();
        maxDepth=max(maxDepth,now->val-left+1);
        if(now->left){
            now->left->val=2*now->val;
            seq.push(now->left);
        }
        if(now->right){
            now->right->val=2*now->val+1;
            seq.push(now->right);
        }
    }
}
return maxDepth;
}
```

❖ 案例：返回节点值自底向上的层序遍历结果。

时间：2025.2.20，5min

链接：[107. 二叉树的层序遍历 II - 力扣 \(LeetCode\)](#)

思路：层序遍历，然后颠倒一下结果的次序。

关键代码：reverse(results.begin(),results.end());

❖ 案例：二叉树的层平均值

时间：2025.2.20，5min

链接：[637. 二叉树的层平均值 - 力扣 \(LeetCode\)](#)

思路：层序遍历，每层计算一次层平均值。

❖ 案例：N 叉数层次遍历

时间：2025.2.20，5min

链接：[429. N 叉树的层序遍历 - 力扣 \(LeetCode\)](#)

思路：层序遍历。二叉树层序遍历添加的是左右孩子，N 叉数即把下一层孩子都添加上。

❖ 案例：二叉树每层最大值

时间：2025.2.20，5min

链接：[515. 在每个树行中找最大值 - 力扣 \(LeetCode\)](#)

思路：层次遍历。每层判断一个最大值即可。

❖ 案例：填充每一个节点的下一个右侧节点指针。

时间：2025.2.20，5min

链接：[116. 填充每个节点的下一个右侧节点指针 - 力扣 \(LeetCode\)](#)

思路：层次遍历。每次遍历的时候，如果存在下一个右侧节点（通过 size 进行判断），则将指针赋值。

❖ 案例：二叉树最大深度。

时间：2025.2.21，5min

链接：[104. 二叉树的最大深度 - 力扣 \(LeetCode\)](#)

思路：层次遍历。每次取 size 的时候表示一层深度。

✧ 衍生：二叉树最小深度。

时间：2025.2.21，5min

链接：[111. 二叉树的最小深度 - 力扣 \(LeetCode\)](#)

思路：层次遍历。第一次访问到的叶子节点一定是处于最小深度的位置，放回当前深度即可。

❖ 案例：二叉树翻转

时间：2025.2.21，5min；2026.8.2 10min，一次过。

链接：[226. 翻转二叉树 - 力扣 \(LeetCode\)](#)

思路启发：二叉树关键知识点即 4 个遍历方法，层次遍历、前、中、后序遍历。

思路：层次遍历。对于每一个节点，交换左右孩子。

思路 2：任意序列的遍历。对于每一个节点，交换左右孩子。

关键代码：

说明：代码主体是“模版代码”。

```
TreeNode *tmp=node->left;
node->left=node->right;
node->right=tmp;
```

❖ 案例：对称二叉树

时间：2025.2.21，30min；2026.8.2 30min（看了解析）。

链接：[101. 对称二叉树 - 力扣 \(LeetCode\)](#)

思路：层次遍历+虚拟节点（INT_MIN）+中心扩展。以层次遍历作为基本框架，每层遍历定义一个 res 数组存放结果，为了应对同值不同位置的情况，对于节点下的空节点设定为一个虚拟值 INT_MIN，然后采用中心扩展法判断每层 res 数组是否相等即可。

❖ 案例：完全二叉树节点数量

时间：2025.2.22，5min

链接：[222. 完全二叉树的节点个数 - 力扣 \(LeetCode\)](#)

思路：层次遍历。记录节点数量。

❖ 案例：找树左下角的值。

时间：2025.2.23，5min

链接：[513. 找树左下角的值 - 力扣 \(LeetCode\)](#)

思路：层次遍历。维护一个变量来记录和更新左下角的值，最后一次更新之后的值一定就是

树左下角的值。

3.5.3 二叉树序列化

学习目标：掌握二叉树的前、中、后序列化。

1. 二叉树中序遍历序列化

定义：把中序遍历变成双向循环链表。

❖ 案例：二叉树中序遍历序列化

时间：2024.7.29；20+20+20=60min

链接：[二叉搜索树与双向链表_牛客题霸_牛客网 \(nowcoder.com\)](#)

思路：

初始化：指向当前节点的前驱节点 prev，头节点 head

递归遍历，当前节点为 node：

如果 prev 为空，说明是头节点，将 head 赋值为 node。结束特判

如果 prev 不为空：

将 prev->right 指向 node

将 node->left 指向 prev

更新 prev 为 node

关键代码：

中序序列化

```
void BST(TreeNode* root,TreeNode *&head,TreeNode *&prev){
    if(!root) return;//特判
    BST(root->left,head,prev);//左递归
    if(!prev){//如果 prev 是空，说明是头节点，把 head 初始化，后面就不管了
        head=root;
    }else{//不为空，说明 prev 已经是当前处理节点 node 的前驱节点
        prev->right=root;
        root->left=prev;
    }
    prev=root;
    BST(root->right,head,prev);//右递归
}
```

❖ 案例：二叉树序列化+反序列化

时间：2024.7.31；50+10+0=60min

链接：[序列化二叉树_牛客题霸_牛客网 \(nowcoder.com\)](#)

思路：先序遍历二叉树，先序创建二叉树。

细节：

- (1) 自定义了定界符"!"。
- (2) 对于空节点用"#"进行表示。
- (3) 对 char 类型的处理。

同理：对于层次遍历、中序遍历、后序遍历二叉树的序列化反序列化原理类似。

关键代码：

前序遍历序列化二叉树（前序遍历，对空节点加以表示）

```
void preOreder(TreeNode *root,string &res){
    if(!root){
        res+="#";
        return;
    }
    res+=to_string(root->val);
    res+="!";//自定义定界符为"!"
    preOreder(root->left,res);//左递归
    preOreder(root->right,res);//右递归
}
```

前序创建二叉树（char 数组对定界符特判）

```
TreeNode *preCreate(char**str){
    if(**str=='#'){
        (*str)++;
        return nullptr;
    }
    int val=0;//数字转换，秦九韶算法
    while(**str!='!'&&&**str!='\0'){
        val=val*10+**str-'0';
        (*str)++;
    }
    TreeNode *node=new TreeNode(val);//创建一个节点
    if(**str=='\0') return node;
    else (*str)++;
    node->left=preCreate(str);
    node->right=preCreate(str);
    return node;
}
```

❖ 案例：[..\数据结构测试.大二上\二叉树\二叉树线索化_final.cpp](#)

描述：中序线索化二叉树及输出（循环）。

时间：22.10.21

说明：这个二叉树链表是循环的。

❖ 案例：[..\数据结构测试.大二上\二叉树\线索二叉树_two.cpp](#)

描述：中序线索化二叉树并输出。说明：这个不涉及循环的情况，更加基础一些。

❖ 案例: [..\数据结构测试.大二上\二叉树\二叉树线索化_前序线索化.cpp](#)

描述: 前序线索化二叉树及输出

//这里实现的是线索二叉树的前序线索化, 即前序遍历。

//提问: 这里在线索化到的时候为什么要判断是否有左右孩子, 中序的时候就没有。

细节: 这里要 `pre` 是全局变量。

❖ 案例: [..\数据结构测试.大二上\二叉树\线索二叉树后续遍历.cpp](#)

描述: 后序线索化二叉树。

说明: 明显后序线索化之后想要输出比较鸡肋, 而且还没调试成功, 所以这里先跳过。

3.5.4 应用案例

一、二叉排序树

1. 二叉搜索树

定义: 树的左边都小于根节点, 树的右边都大于根节点; 中序遍历之后是单增序列。

解题核心: 中序遍历二叉树。

❖ 案例: [判断是不是二叉搜索树](#)。

时间: 2024.7.29; 20+5+0=25min

链接: [判断是不是二叉搜索树 牛客题霸 牛客网 \(nowcoder.com\)](#)

关键代码:

思路: 中序遍历一下, 看是不是单增序列。

```
void inorder(TreeNode *root,TreeNode *&prev,bool &flag){
    if(!root) return;
    inorder(root->left,prev,flag);
    if(prev&&prev->val>=root->val){//如果 prev 是空, 那么一定是 root 节点
        flag=false;
    }
    prev=root;
    inorder(root->right,prev,flag);
}
```

✧ 二刷, 验证二叉搜索树, 时间: 2025.2.25, 10min

链接: [98. 验证二叉搜索树 - 力扣 \(LeetCode\)](#)

思路: 中序遍历, 然后判断序列是否递增。

细节: 这里在访问容器的时候, 需要提前判断容器是否为空, 不然会发生空指针的错误。

说明: 相对来说, 第一次的方案会更加节省空间。

```
void visit(TreeNode *root,vector<int> &nums,bool &result){
    if(!root) return;
    if(root->left) visit(root->left,nums,result);
    if(!nums.empty()&&nums.back()>=root->val){
```

```
        result=false;
        return;
    }
    nums.push_back(root->val);
    if(root->right) visit(root->right,nums,result);
}
```

❖ 案例：二叉排序树。

时间：2024.9.20，10min

链接：[二叉排序树_牛客题霸_牛客网 \(nowcoder.com\)](https://www.nowcoder.com)

知识点：二叉排序树的创建。

关键代码：

细节 1：递归窗口要有 return 返回。

细节 2：指针创建的时候一定要初始化。初始化为空。node *root=nullptr;

```
void insert(node *&root,int val){
    if(!root){//如果是根节点单独创建
        root=new node(val);
        return;
    }
    if(val<root->val){
        if(!root->left){//创建
            root->left=new node(val);
        }else{//往左递归
            insert(root->left,val);
        }
    }else if(val>root->val){
        if(!root->right){//创建
            root->right=new node(val);
        }else{//往右递归
            insert(root->right,val);
        }
    }
    //val==root->val 相等的情况下也是直接略过，所以整理省略了
    return;
}
```

❖ 案例：二叉排序树的最小绝对差

时间：2025.2.25，15min

链接：[530. 二叉搜索树的最小绝对差 - 力扣 \(LeetCode\)](https://leetcode.cn/problems/minimum-absolute-difference-in-bst/)

思路：中序遍历。计算差值。

❖ 案例：二叉搜索树中的众数。

时间：2025.2.25，35min

链接：[501. 二叉搜索树中的众数 - 力扣 \(LeetCode\)](https://leetcode.cn/problems/find-mode-in-bst/)

思路：中序遍历。采用 hash 表统计每个值出现的数量。因为涉及到重复的众数，需要采用 vector 存储结果。（最开始因为想要用优先队列做，但是发现优先队列不能修改队列中的元素值，所以写一半写不下去了。）

❖ 案例：最近公共祖先

链接：二叉树遍历-中序遍历-二叉树最近公共祖先。

❖ 案例：二叉搜索树中插入操作

时间：2025.2.27，10min

链接：[701. 二叉搜索树中的插入操作 - 力扣 \(LeetCode\)](#)

思路 1-迭代：二叉搜索树性质（定义）。往下循环判断。

思路 2-递归：

说明：根据二叉搜索树有序的性质，可以优化搜索效率。下一层将加入节点返回，本层用 root->left 或者 root->right 将其接住，通过递归函数返回值完成了新加入节点的父子关系赋值操作。

```
TreeNode* insertIntoBST(TreeNode* root, int val) {  
    if(root==nullptr){  
        TreeNode *node=new TreeNode(val);  
        return node;  
    }  
    if(root->val<val) root->right=insertIntoBST(root->right,val);  
    if(root->val>val) root->left=insertIntoBST(root->left,val);  
    return root;  
}
```

❖ 案例：二叉搜索树中删除操作。

时间：2025.2.28，60min

链接：[450. 删除二叉搜索树中的节点 - 力扣 \(LeetCode\)](#)

思路：递归遍历。对于找到的节点分 4 种情况。（视频将没找到算作一种）

如果该节点是叶子节点，则直接删除。

如果该节点有左孩子，无右孩子，则将左孩子接到父节点上。

如果该节点无左孩子，有右孩子，则将右孩子接到父节点上。

如果该节点有左孩子，有右孩子，则将左孩子接到右孩子最右侧节点上，再将右孩子接到父节点上。（同理，将右孩子接到左孩子最左侧节点上，再将左孩子接到父节点上。）

关键知识点：递归三要素。（1）递归参数和返回函数（回溯）。（2）递归终止条件。（3）递归体逻辑。

关键代码：

重点：递归函数有返回，这里体现了回溯算法的思想。

理解：先序遍历+递归。

```
TreeNode* deleteNode(TreeNode* root, int key) {  
    if(root==nullptr) return nullptr;  
    if(root->val==key){  
        if(root->left==nullptr&&root->right==nullptr){//如果左右都为空
```

```

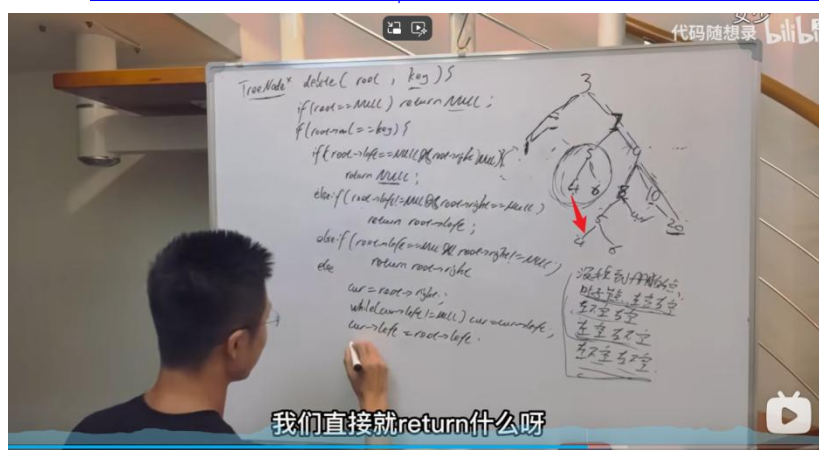
        delete root;
        return nullptr;//直接删除该节点，并且给上一层返回一个空值，让上一层接一个空值。
    }else if(root->left==nullptr&&root->right){
        TreeNode *cur=root->right;
        delete root;
        return cur;
    }else if(root->left&&root->right==nullptr){
        TreeNode *cur=root->left;
        delete root;
        return cur;
    }else{//root->left&&root->right
        //让 root 左节点整体挪到右子树最左边的位置。
        TreeNode *cur=root->right;//先找到右子树最左边的位置
        while(cur->left) cur=cur->left;
        cur->left=root->left;
        cur=root->right;
        delete root;
        return cur;
    }
}

if(key<root->val) root->left=deleteNode(root->left,key);
if(key>root->val) root->right=deleteNode(root->right,key);
return root;//如果什么都没找到，或者找过了，直接返回 root
}

```

补充：代码随想录卡尔视频。

链接：[调整二叉树的结构最难！| LeetCode: 450.删除二叉搜索树中的节点_哔哩哔哩_bilibili](https://leetcode.cn/problems/delete-node-in-bst/)



❖ 案例：裁剪二叉树

时间：2025.3.1，80min

链接：[669. 修剪二叉搜索树 - 力扣 \(LeetCode\)](https://leetcode.cn/problems/cut-off-tree-for-given-height/)

关键代码：

说明：这道题和删除二叉树中的节点类似但又不一样。

类似：都是删除节点。

区别：裁剪二叉树，需要针对子树做专门的递归裁剪。先序遍历，在递归中加递归。

理解：递归包括了两个部分，第一个部分是递归框架需要递归（前、中、后序遍历），第二个部分是递归逻辑需要递归（这里需要对左、右子树也进行裁剪）。对于每一个节点的操作应该是一模样的，通过条件判断来实现对不同性质的节点有不同类型的操作。

```
TreeNode* trimBST(TreeNode* root, int low, int high) {
    if(root==nullptr) return nullptr;
    if(root->val<low){
        TreeNode *cur=trimBST(root->right,low,high);
        return cur;
    }
    if(root->val>high){
        TreeNode *cur=trimBST(root->left,low,high);
        return cur;
    }
    root->left=trimBST(root->left,low,high);
    root->right=trimBST(root->right,low,high);
    return root;
}
```

❖ 案例：已知升序序列，建平衡二叉搜索树。

时间：2025.3.2，10min

链接：[108. 将有序数组转换为二叉搜索树 - 力扣 \(LeetCode\)](#)

思路：前序遍历。前序遍历，控制左右边界建树。

二、其他

1. 求节点数、叶子数、深度

❖ 案例：[..\数据结构测试.大二上\二叉树\求节点数、叶子数、深度.cpp](#)

说明：求节点数、叶子数、深度；这里主要是用的递归实现的。

核心思维：极限+递归

时间：22.10.21 已粘贴

关键代码：求深度

```
int countdepth(Tree T){
    if(T==NULL) return 0;
    else if(T->lchild==T->rchild){
        return 1;
    }
    else{
```

```

        return max(countdepth(T->lchild)+1,countdepth(T->rchild)+1);
    }
}

```

关键代码：求叶子数量。

```

int countleaf(Tree T){
    if(T==NULL) return 0;//细节：这里有两个出口
    else if(T->lchild==T->rchild){//只有等于空的时候才会相等
        return 1;
    }
    else{
        return countleaf(T->lchild)+countleaf(T->rchild);
    }
}

```

关键代码：计算节点数量

```

int countnode(Tree T){
    if(T==NULL) return 0;
    else return countnode(T->lchild)+countnode(T->rchild)+1;
}

```

2. 中序转后续

❖ 案例：[..\数据结构测试.大二上\表达式求值\中序转后续.cpp](#)

时间：22.11.3

描述：定义栈内外优先级。

❖ 案例：将两个子树合并一下。

链接：[合并二叉树_牛客题霸_牛客网 \(nowcoder.com\)](#)

时间：2024.7.29；40+10+0=50min

思路：创建新的节点

关键代码：

细节：动态创建节点，避免指针共享问题。

说明：下边二刷的时候发现，即便共享指针也行。这里是比较标准的前序遍历创建二叉树的写法。

```

TreeNode* mergeTrees(TreeNode* t1, TreeNode* t2) {
    if (!t1) return t2; // 如果 t1 为空，则直接返回 t2
    if (!t2) return t1; // 如果 t2 为空，则直接返回 t1
    // 新建根节点，值为 t1 和 t2 的值之和
    TreeNode* merged = new TreeNode(t1->val + t2->val);
    // 递归合并左子树
    merged->left = mergeTrees(t1->left, t2->left);
}

```

```
// 递归合并右子树
merged->right = mergeTrees(t1->right, t2->right);
return merged;
}
```

✧ 二刷：时间：2025.2.24，40min

链接：[617. 合并二叉树 - 力扣 \(LeetCode\)](#)

思路：同时进行前序遍历。这里支持 root1 和 root2 同时指向同一个节点。

关键代码：

说明：这里没有按照严格前序遍历风格进行编写，所以最开始第一版写错了。

```
void merge(TreeNode* &root1, TreeNode* root2){
    if(root1 && root2) root1->val = root1->val + root2->val;
    // 处理左子树
    if (root2->left) {
        if (!root1->left) {
            root1->left = root2->left;
        } else {
            merge(root1->left, root2->left);
        }
    }
    // 处理右子树
    if (root2->right) {
        if (!root1->right) {
            root1->right = root2->right;
        } else {
            merge(root1->right, root2->right);
        }
    }
}
```

❖ 二叉树的直径

时间：2026.8.2 25min（一次过）；

链接：[543. 二叉树的直径 - 力扣 \(LeetCode\)](#)

大致思路：正常左右遍历即可。

3.6 图

3.6.1 拓扑排序

定义：top 排序。

案例：[【模板】拓扑排序_牛客题霸_牛客网 \(nowcoder.com\)](#)

描述：top 排序基础案例。

关键思路：

设计数据结构并读入数据

邻接表

入度数组

top 排序

初始化队列（栈），将入度为 0 的点入队列

遍历队列，通过邻接表来实现入度数组的更新

细节：设定一个 cnt 判断不能构成 top 排序的图

关键代码：

```
// 遍历栈，直到栈为空
while (!que.empty()) {
    // 将栈顶元素出栈
    int top = que.front();
    result.push_back(top);
    que.pop();
    cnt++;
    // 出栈之后更新入度
    for (int to : adjlist[top]) { // 遍历邻接表
        inDegree[to]--;
        // 如果入度变为 0，则将其加入栈中
        if (inDegree[to] == 0) {
            que.push(to);
        }
    }
}
```

细节：使用 vector 实现邻接表；数组初始化。

// 设计数据结构，并将数据读入

vector<vector<int>> adjlist(n + 1); // 邻接表

// 初始化入度数组

int *inDegree = new int[n + 1](); // 使用 () 进行初始化

补充：

设计数据结构

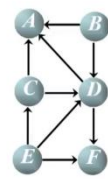
1. 图的存储结构：采用邻接表存储，在顶点表中增加一个入度域。



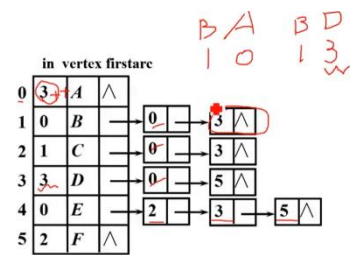
2. 栈S：存储所有无前驱的顶点。

思考：可以用队列吗？

拓扑排序



(a) 一个AOV网



(b) AOV网的邻接表存储

3.6.2 最小生成树

1. kruskal+并查集

案例：[最小生成树_牛客题霸_牛客网 \(nowcoder.com\)](https://www.nowcoder.com)

描述：采用并查集+kruskal 算法。

思路描述：

并查集

并查集查找

并查集合并

排序：采用 vector 的 sort 函数来排序，需要自定义 cmp。（static）

kruskal 算法：遍历边；采用并查集判断是否构成环。

如果顶点构成环，则跳过。

如果顶点不构成环，则合并并查集。

关键代码：

```
int result = 0;
for (vector<int>& edge : cost) {
    // 把边读出来
    int v1 = edge[0];
    int v2 = edge[1];
    int value = edge[2];

    // 判断是否是在一个圈里面
    int pa1 = find(parent, v1);
    int pa2 = find(parent, v2);
    if (pa1 != pa2) { // 如果不在一个圈子里面，可以添加这条边
        result += value;
        parent[pa1] = pa2; // 将其放到一个并查集里面
    } // pa1==pa2 在一个边里面，跳过即可
}
```

细节：采用 vector 实现并查集。

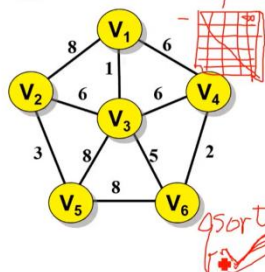
```
vector<int> parent(n + 1); // 注意大小为 n + 1
```

补充:

3.1.1 从边的角度

执行过程

第1步: 选择权值最小的边



顶点1 顶点2 权值 是否选中

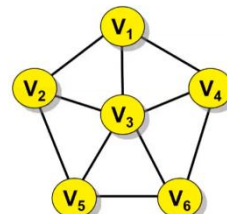
V ₁	V ₃	1	F
V ₄	V ₆	2	F
V ₂	V ₅	3	F
V ₃	V ₆	5	F
V ₁	V ₄	6	F
V ₂	V ₃	6	F
V ₃	V ₄	6	F
V ₁	V ₂	8	F
V ₃	V ₅	8	F
V ₅	V ₆	8	F

1 引 → 2 析 → 3 解 → 4 用



城市
管道

→ 顶点
→ 边



3.6.3 最短路径问题

一、Dijkstra 算法

案例: Dijkstra 单源最短路径算法。

链接: [【模板】单源最短路 2_牛客题霸_牛客网 \(nowcoder.com\)](https://www.nowcoder.com)

说明: Dijkstra 算法

关键思路:

初始化

邻接矩阵 Graph, 初始化 INT_MAX

visit, visit 全部初始化为 false, visit[1]=true

dist, 用 Graph[1][j]来初始化。dist[1]=0

dijkstra 算法: 循环 N-1 次

先在 dist 里面找最短的一个点 index

将 index 放入已访问的集合, 设置 visit[index]=true

更新 dist, 以 index 来更新 dist, 在未访问可达的集合里里面进行判断, 若绕道的路程更小则绕道。

查看 dist[n]得出结论

关键代码:

细节: 更新 dist 时, 判断, 是未访问、可达。

//dijkstra 算法

```
for(int i=2;i<=N;i++){//循环 N-1 次
```

```
//寻找集合外最短点
```

```
int index=-1,tmp=INT_MAX;//用来保存最短的点
```

```
for(int j=1;j<=N;j++){
```

```

        if(!visit[j]&&dist[j]<tmp){
            tmp=dist[j];
            index=j;//记录最短的点。
        }
    }
    //最短节点置为 true
    if(index!=-1){
        break;
    }
    visit[index]=true;
    //更新 dist
    for(int j=1;j<=N;j++){
        if(!visit[j]&&Graph[index][j]!=INT_MAX){
            if(dist[index]+Graph[index][j]<dist[j]){
                dist[j]=dist[index]+Graph[index][j];
            }
        }
    }
}
}

```

补充：

1、从某个源点到其余各顶点的最短路径

基本思想：按路径长度递增次序产生最短路径

算法：把V分成两组：

(1) U：已求出最短路径的顶点的集合，从v出发到图上其余各顶点vi可能达到的最短路径的初值为：

$dist[i]=Matrix[v][vi]$

(2) 选择vj，使得 $dist[j]=\min\{dist[i] \mid vi \in V-U\}$
vj就是求得的一条从v出发的最短路径的终点。令

$U=U \cup \{j\}$

(3) 修改从V出发到集合V-U上任一顶点Vk可达的最短路径。

(4) 重复 (2) (3) 共n-1次。

Prim

第一步：

大循环n次 { 1 初始化dist[]

2 初始化path[]

3 初始化tag为unvisited }

初始化起点tag为visited

	0-A	1-B	2-C	3-D	4-E
dist	∞	10	∞	30	100
path	-1	0	-1	0	0
Tag	V	UNV	UNV	UNV	UNV

3.7 其他

描述：这里放不是那么常用的数据结构

3.7.1 并查集

关键：并查集基本概念。

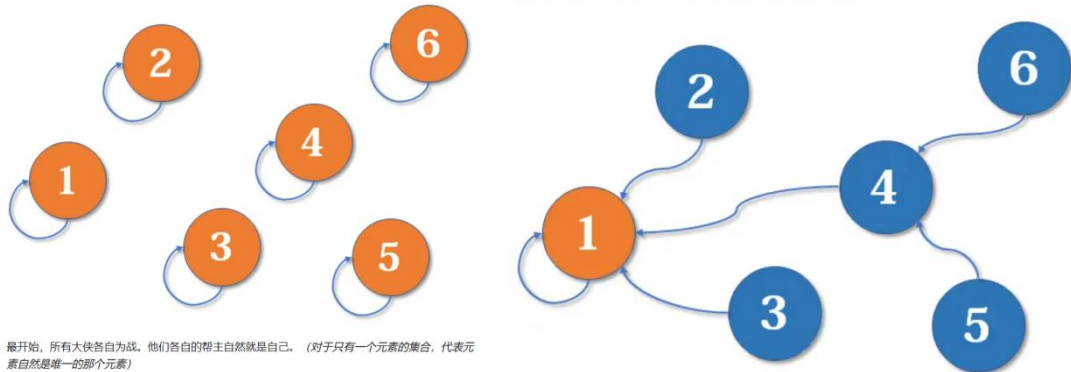
并查集查找父节点。

并查集合并。

补充：[算法学习笔记\(1\)：并查集 - 知乎 \(zhihu.com\)](#)

并查集的重要思想在于，用集合中的一个元素代表集合。我曾看过一个有趣的比喻，把集合比喻成帮派，而代表元素则是帮主。接下来我们利用这个比喻，看看并查集是如何运作的。

现在假设2号想与6号比，跟刚刚说的一样，喊帮主1号和4号出来打一架（帮主真辛苦啊）。1号胜利后，4号认1号为帮主，当然他的手下也都是跟着投降了。



3.7.2 堆

理解：一种特殊的二叉树，用数组来表示二叉树。

基础概念：

若结点数组下标为 i ，则：

父结点数组下标： $(i - 1) / 2$ ；

左孩子数组下标： $2 * i + 1$ ；

右孩子数组下标： $2 * i + 2$ ；

❖ 案例：【模板】堆 [牛客题霸_牛客网 \(nowcoder.com\)](https://www.nowcoder.com)

描述：进行堆的添加、删除。

思路描述：计算脚标并进行比较来进行判断。

基础层：

交换函数

向上调整函数 `up()`：while 循环向上调整。

向下调整的函数 `down()`：递归操作，找到局部最大的做调整，向下调整。

应用层：

往堆添加元素：往数组末尾添加一个元素，再向上调整。

删除堆中元素：把堆末尾元素放到堆顶，再向下调整。

通过数组创建堆：从下往上对每个元素都向下调整。

关键代码：

//下降调整，采用递归的方式实现。当删除一个元素之后，需要将尾元素放到堆顶进行向下调整

```
void down(int root){
    //先计算左右节点
    int left=root*2+1;
    int right=root*2+2;
    int largest=root;
    //在左右节点中选择最大的节点
    if(left<heap.size()&&heap[largest]<heap[left]){
```

```

        largest=left;
    }
    if(right<heap.size()&&heap[largest]<heap[right]){
        largest=right;
    }//如果自己的节点不是最大的，那么调换，并且递归继续判断
    if(largest!=root){
        swap(largest,root);
        down(largest);
    }
}

```

//上升调整，当添加一个元素之后，元素被放在末尾，进行升来调整堆

```

void up(int last) {
    while (last > 0 && heap[last] > heap[(last - 1) / 2]) {
        swap(last, (last - 1) / 2);
        last = (last - 1) / 2;
    }
}

```

知识架构：

底层：

最后一个元素向上调整。

堆顶元素向下调整。

表层：

堆添加。

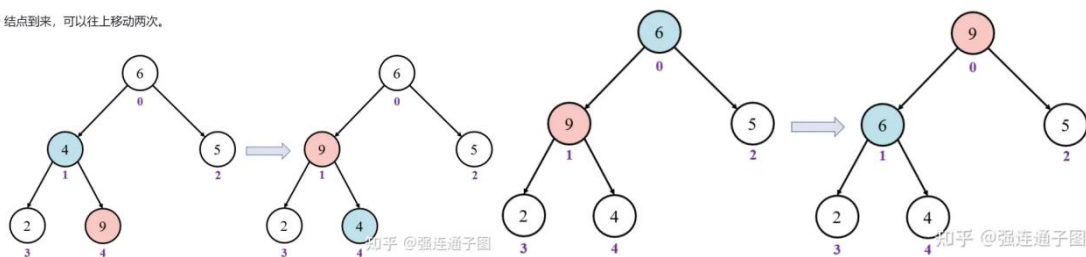
堆删除。

通过数组来构建堆。

补充：

知乎：[【堆 - 专题】堆排序，大根堆，小根堆 - 知乎 \(zhihu.com\)](https://www.zhihu.com/question/20461146)

9 结点到来，可以往上移动两次。



应用案例

● 案例：滑动窗口

时间：2025.2.1， 2h

链接：[239. 滑动窗口最大值 - 力扣 \(LeetCode\)](https://leetcode.cn/problems/sliding-window-maximum/)

关键知识点：优先队列（priority_queue，采用大根堆进行维护的）。

思路：构建大根堆，将 index 绑定输入，如果堆顶元素不在窗口内，则出堆。

- 案例：前 k 个高频元素。

链接：[347. 前 K 个高频元素 - 力扣 \(LeetCode\)](#)

思路：用 hash 记录词频。用排序算法排序。

知识点：采用大小根堆可以排序（堆排序）。

第 4 章 算法

4.1 快速幂、乘

一、快速幂

定义：快速计算幂。

计算： m^n

核心思想： m^n 可以拆分为 $m^{n/2} \times m^{n/2}$ ；任何整数都能转换为 2 进制。

思想拆解：

如果 n 为偶数，则 $m^n = m^{n/2} \times m^{n/2}$

如果 n 为奇数，则 $m^n = m^{n/2} \times m^{n/2} \times m$

说明：n/2 默认为整除；n%2 默认为求余除。

案例：[..\经典算法测试\快速幂\fastPow.cpp](#)

基本思路：如果是偶数，则不断平方，如果遇到是奇数，就乘一个自己。

关键代码：

//快速幂，快速计算 m^n

```
long long res=1;
while(n){
    if(n&1){//如果为奇数
        res=res*m;
    }
    m=m*m;//m*m 相当于幂向左移动了一位
    n>>=1;
}
```

细节：如果数据特别大的时候，会出现数据溢出的情况，需要将所有数据类型都设置为 long long int 类型（即 m、n、mod、res）。

二、快速乘

定义：快速计算 $a*b$

核心思想：任何整数都能被转换为二进制。

案例：[快速乘_牛客题霸_牛客网 \(nowcoder.com\)](https://www.nowcoder.com)

关键代码：

```
ll res=0;
while(b){
    if(b&1){
        res=(res+a)%p;
    }
    a=(a+a)%p;
    b=b/2;
}
```

4.2 搜索算法

一、广度优先搜索 BFS

理解：该算法核心通过队列来实现。

案例：[走迷宫_牛客题霸_牛客网 \(nowcoder.com\)](https://www.nowcoder.com)

说明：路径规划问题。

关键思路：

初始化：

建图；建 DP。

BFS：广度有限搜索

以一个点为基础往四周出发，如果满足约束则入队列，并将该路径堵住（避免重复访问）。

输出：DP[target]

关键代码：

```
for(int i=0;i<4;i++){
    //判断是否能够通过，如果能够通过的话，就通过，将其放入队列中，等待下一次
    的广搜
    int x=pos.first+direction[i].first;
    int y=pos.second+direction[i].second;
    if(1<=x&&x<=n&&1<=y&&y<=m&&Graph[x][y]!='.'){//在区域内，并且无障碍
        position next(x,y);
        que.push(next);
        fit[x][y]=fit[pos.first][pos.second]+1;
        Graph[x][y]='*';
    }
}
```

```

    }
    //通过之后需要判断是不是到达目的地了
}

```

二、深度优先搜索 DFS

理解：通过栈来实现，一般写成递归会方便一些。理解 2024.8.28：递归的本质就是栈。递归常见的剪枝手段就是记忆化。

案例：[kotori 和素因子_牛客题霸_牛客网 \(nowcoder.com\)](https://www.nowcoder.com)

核心思想：

针对每个数字都会有不同的素因子，针对数字的不同的素因子进行递归判断，寻找满足条件的组合方式。

技巧：采用集合来作为“素因子不重复”这个条件的限定。

❖ 案例：[矩阵最长递增路径](#)。栈解法、递归解法。

时间：2024.8.28

链接：[矩阵最长递增路径_牛客题霸_牛客网 \(nowcoder.com\)](https://www.nowcoder.com)

方法一、[栈-无优化](#)。

思路：把每个层面的所有信息（位置，前一个元素，当前路径）保存到栈里面。

定义好栈的每个层次的变量。

基本栈操作。

关键代码：关键在于每层要包含所有递归变量。

变量定义：

`typedef pair<int,int> position;`//位置变量

`typedef pair<int,int> refs;`//first 设定为 prev，second 设定为 curPath

`typedef pair<position,refs> dfs;`//深搜每层的元素

知识点：数组初始化。{ {} }

说明：如下代码段外层，还有两层循环用来循环矩阵上每一个点。

优化方向：使用一个 isvisited 来记录中间过程，避免重复计算。

`pos.push({{x,y},{matrix[x][y],1}});`//知识点：数组初始化

`while(!pos.empty())` { //如果不是空

 //获取当前元素

`dfs now=pos.top();`

`position nowPos=now.first;`

`refs nowRef=now.second;`

`pos.pop();`

`curMax=max(curMax,nowRef.second);`

 //探测四个方向

`for(int i=0;i<=3;i++){`

`position next(nowPos.first+ps[i].first,nowPos.second+ps[i].second);`

 //如果满足如下在矩阵中的条件

`if(0<=next.first&&next.first<=n-1&&0<=next.second&&next.second<=m-1){`

`if(nowRef.first<matrix[next.first][next.second])` { //接下来是方向选择条件

```

        pos.push({next, {matrix[next.first][next.second], nowRef.second + 1}});
    }
}
}
}

```

解法二、**递归。添加剪枝优化（记忆化）。**

思路：递归四个方向，从四个方向里面选择一个最大的方向。

关键代码：

说明：窗口函数两层循环。

```

int dfs(position pos,vector<vector<int>> &memo,vector<vector<int> >& matrix){
    if(memo[pos.first][pos.second]!=-1) return memo[pos.first][pos.second];
    static const vector<position> directions = {{0,1},{1,0},{0,-1},{-1,0}};
    int n=matrix.size();
    int m=matrix[0].size();
    int maxPath=1;
    for(position dir:directions){
        position next({pos.first+dir.first,pos.second+dir.second});//创建并初始化
        if(0<=next.first&&next.first<n&&0<=next.second&&next.second<m){
            if(matrix[pos.first][pos.second]<matrix[next.first][next.second]){//满足递增的
                //从四个方向里面挑选一个最大的方向
                maxPath=max(maxPath,1+dfs(next,memo,matrix));
                //原式： maxPath=1+dfs(next,memo,matrix)
            }
        }
    }
    return memo[pos.first][pos.second]=maxPath;
}

```

小结：递归的本质是栈。记忆化用于剪枝。

4.3 递归、回溯

定义：递归是指遍历所有方案，选出满足条件的方案。

4.3.1 理论基础-递归

递归三要素：

1. 确定递归函数的参数和返回值。确定哪些参数需要处理。
2. 确定终止条件。操作系统使用栈来保存每层递归信息，如果不写终止，则会报错栈溢出。
3. 确定单层递归逻辑。重复调用自己。

递归是方便了程序员，难为了机器（各种保存参数，各种进栈出栈）。在实际项目开发的过程中我们是要尽量避免递归！因为项目代码参数、调用关系都比较复杂，不容易控制递归深度，甚至会**栈溢出**。

❖ 小结 2024.8.26:

回溯: 发现当前路径不可行时回退到上一步，以寻找其他可能的路径。回溯常用于解决组合、排列和划分等问题。

应用场景：如数独、八皇后问题、**子集划分**等问题，回溯法是自然的选择。

步骤:

尝试将某个元素放入一个位置。

递归处理下一个元素。

如果发现某条路径不通（不可行），则回退并尝试其他路径。

剪枝优化（Pruning）：搜索过程中提前判断某些路径不可能得到最优解。

应用场景：例如在背包问题中，如果当前放入一个物品已经超过了背包容量，就可以立即停止对该路径的探索。

❖ 案例：三角形最小路径和。

链接：[三角形最小路径和_牛客题霸_牛客网 \(nowcoder.com\)](https://www.nowcoder.com/problems/11464)

方法一：递归

关键思路:

递归窗口：达到最后一层，返回 `triangle[i][j]`

细节：如果 `memo[i][j]` 被计算过则直接返回 `memo[i][j]`

递归体:

`left=dfs(triangle,i+1,j)`

`right=dfs(triangle,i+1,j+1)`

`memo[i][j]=triangle[i][j]+min(left,right)`

方法二:

关键思路：动态规划

一层一层网上加

❖ 案例：**没有重复项数字的全排列**。

时间：2024.7.21 5+30+5=40min +20min（衍生）=60min

链接：[没有重复项数字的全排列_牛客题霸_牛客网 \(nowcoder.com\)](https://www.nowcoder.com/problems/11464)

描述：随意排列数组，有多少种不同排列方式。

思路：递归。

递归窗口：如果排列序列长度满足，则放入答案。

递归体：遍历数组。用过的放入，用完弹出。

关键代码:

//递归状态转移

```
for(int i=0;i<nums.size();i++){
```

```
    if(used[i]) continue;
```

```
    used[i]=true;//如果没用过，那么使用
```

```

        tmp.push_back(nums[i]);
        traceback(nums,tmp,used,result);
        used[i]=false;//使用完了释放内存
        tmp.pop_back();
    }

```

衍生：有重复项数字的全排列。

链接：[有重复项数字的全排列 牛客题霸 牛客网 \(nowcoder.com\)](https://www.nowcoder.com)

思路：主要思路同上，采用递归。

关键：对于重复项需要跳过。

关键代码：

```

if(nums[i]==nums[i-1]&&!used[i-1]&&i>0) continue;

```

说明：当前项如果与前一项相同，说明递归到该项的时候，前一项已经经过回溯了，这个时候前一项的 `used` 应该是 `false`，但是当前项和前一项明明相同，所以需要跳过，于是设计了 `!used[i-1]`，补充 `i>0` 是为了满足角标条件。

思考 2024.8.26：递归前执行动作，递归，递归后释放动作。

❖ 案例：[划分等和序列](#)。

时间：2024.8.26 60+50+10=120min

链接：[划分等和序列 牛客题霸 牛客网 \(nowcoder.com\)](https://www.nowcoder.com)

思路：

窗口函数：剪枝，判断如果不能被整除 `k`，返回 `false`。升序处理。

递归函数：

递归参数：`index`，`nums` 数组的脚标。

递归窗口：如果 `index` 指向 `nums` 最后一个元素，那么判断是否满足等和划分。

递归体：

递归前：把 `nums[index]` 加入到 `buckets`。

递归执行：`index+1`

递归后：回溯，把 `nums[index]` 从 `buckets` 取出

关键代码：

递归

```

bool traceback(int index,vector<int> &buckets,vector<int> &nums,int target){
    //递归窗口：已经遍历到底了
    if(index==nums.size()){
        for(int bucket : buckets){
            if(bucket!=target) return false;
        }
        return true;
    }
    //递归体
    for(int i=0;i<buckets.size();i++){
        //递归前
        if(buckets[i]+nums[index]>target) continue;
        buckets[i]+=nums[index];

```

```

    if(traceback(index+1,buckets,nums,target)) return true;
    //递归后
    buckets[i]-=nums[index];
    //剪枝
    if(buckets[i]==0) return false;
}
return false;
}

```

4.3.2 基础理论-回溯

时间：第一次时间未知；2026.9.16 第二次学习；

链接：[带你学透回溯算法（理论篇） | 回溯法精讲！哔哩哔哩 bilibili](#)

定义：回溯法也可以叫做回溯搜索法，是一种搜索的方式。回溯是递归的副产品，只要有递归就会有回溯。回溯函数也就是递归函数，指的都是一个函数。

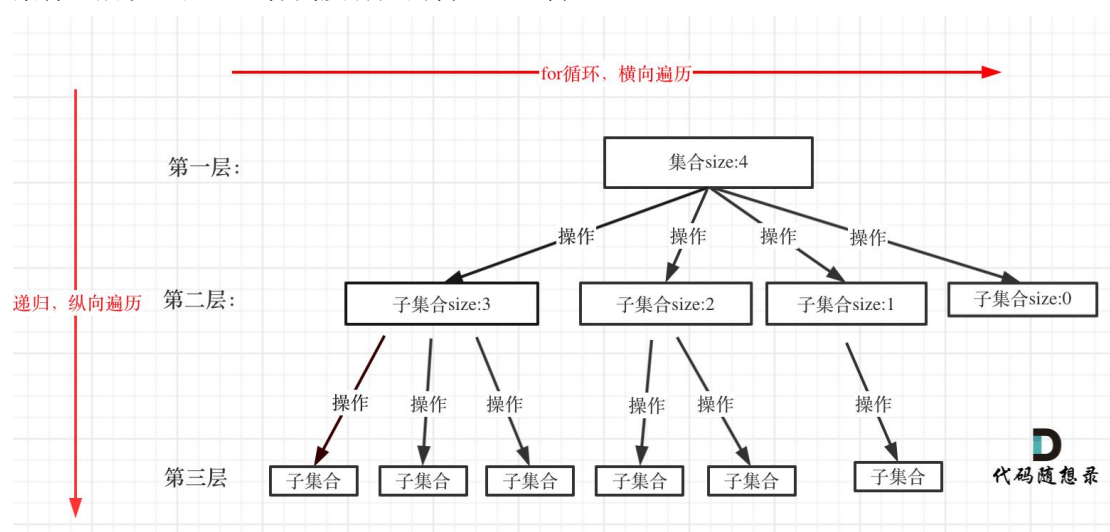
回溯法的效率：回溯的本质是穷举，穷举所有可能，然后选出我们想要的答案，所以效率很低。

误区：不要试着大脑模拟递归回溯流程，太难了，使用树形结构来模拟一下。

回溯法解决的问题：

- (1) 组合问题：N 个数里面按一定规则找出 k 个数的集合
- (2) 切割问题：一个字符串按一定规则有几种切割方式
- (3) 子集问题：一个 N 个数的集合里有多少符合条件的子集
- (4) 排列问题：N 个数按一定规则全排列，有几种排列方式
- (5) 棋盘问题：N 皇后，解数独等等

如何理解回溯法：所有回溯法的问题都可以抽象为树形结构！因为回溯法都是在集合中递归查找子集，集合的大小就构成了树的宽度，递归的深度就构成了树的深度。递归就要有终止条件，所以必然是一棵高度有限的树（N 叉树）。



图：图解回溯法是树结构

回溯三部曲：

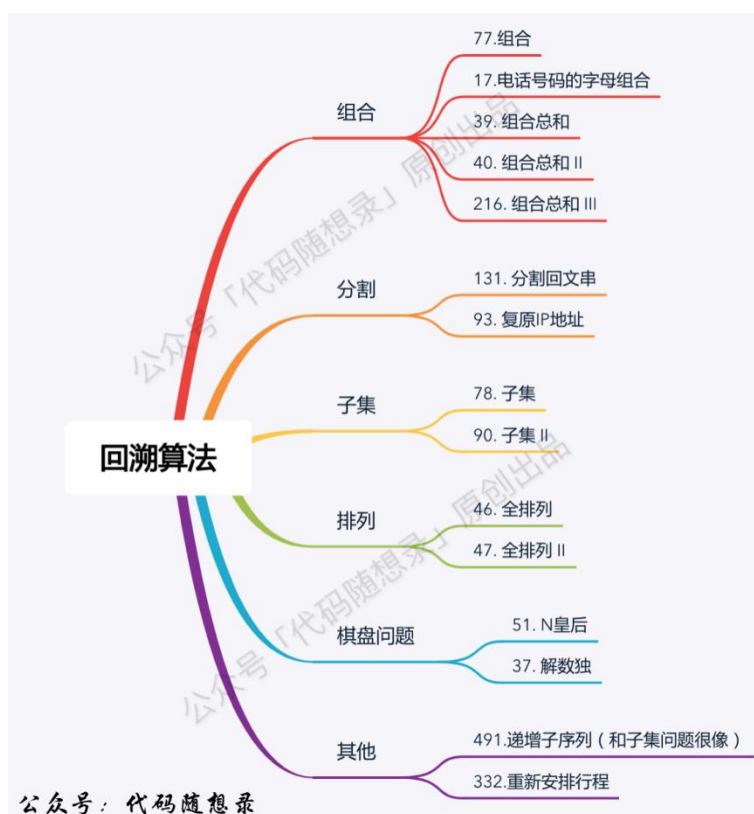
输入参数。

终止条件。
单层递归的动作。

回溯法的模版：

说明：在回溯算法中，业界给函数起名字为 **backtracking**，回溯算法中函数返回值一般为 **void**。

```
void backtracking(参数) {  
    if (终止条件) {  
        存放结果;  
        return;  
    }  
    for (选择：本层集合中元素（树中节点孩子的数量就是集合的大小）) {  
        处理节点;  
        backtracking(路径，选择列表); // 递归  
        回溯，撤销处理结果  
    }  
}
```



图：回溯法可解决问题

一、应用案例

说明：这里是各类案例。

❖ 案例：组合数。

时间：2025.3.3 10min；2026.9.19 10min 做不出来了，应该是能做出来的，出现了一点小问题，记住，每一步要有理有据。

链接：[77. 组合 - 力扣 \(LeetCode\)](#)

卡尔教学：[带你学透回溯算法-组合问题（对应力扣题目：77.组合）| 回溯法精讲！_哔哩哔哩_bilibili](#)

基础思路：回溯算法模版。遍历[1,n]，每次分支都从第 i+1 个位置往后遍历进行分支。

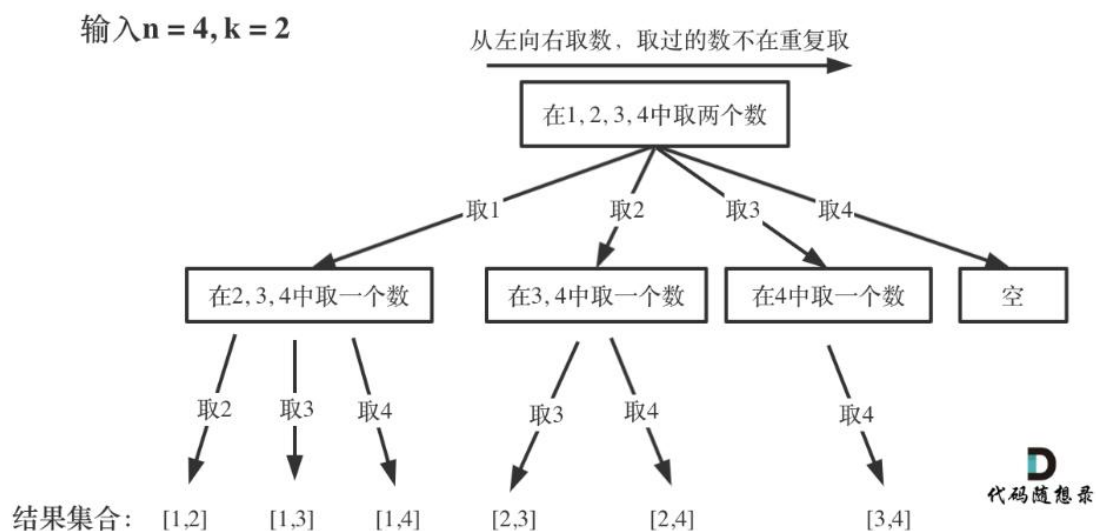
减枝思路：如果当前轮次数组[1,n]的子集的里面剩余的元素已经不足以组成期望的组合的元素数量，可以减掉这部分回溯。

一般是针对划分子集的方式进行限定来达到剪枝的作用。这个题目的剪枝思路是，如果子集剩下的元素不足以使得路径满足 `path.size()==k`，那么可以提前终止后续的分支，以此来提高效率。

算法伪代码：

按照回溯树结构过程，下一次分支是基于原有集合的子集进行的，这个题目需要一个 `startIndex` 来界定下一次分支的子集。

```
void backtracking(path, startIndex, results){ // 回溯参数
    // 回溯终止条件，当 path 达到 k 要求
    if(path.size()==k){
        results.push_back(path);
        return; // 终止后面的递归和回溯动作。不然会陷入死循环。
    }
    // 单次递归的动作，划分子集 进行分支，这里使用 startIndex 进行分支
    start_min = n-(k-path.size()+1); // 这里 "+1" 是因为脚标和数量的关系，比如
    // n=4,k=3,path.size()=0 的情况下，至少从第二个元素开始才能有希望使得 path.size()==k，
    // 不然其他的都可以不用看。
    for(int i=startIndex;i<=start_min;i++){
        path.push_back(i);
        backtracking(path, i+1, results);
        path.pop_back(); // 撤回递归的操作，这里具体的动作是弹出 path 末尾的元素。
    }
}
```



图：递归和回溯过程示意图

❖ 案例：组合总数

时间：2025.3.4, 15min

链接：[216. 组合总和 III - 力扣 \(LeetCode\)](#)

思路：回溯。剪枝：（1）如果数组剩下元素数量不足构成 k 个结果，则提前退出。（2）定义一个变量 sum 来记录 $path$ 路径的和，如果 sum 大于目标值 n 则跳过该循环；同时， sum 也能减少求和计算数量。

❖ 案例：电话号码字母组合

时间：2025.3.4, 25min

链接：[17. 电话号码的字母组合 - 力扣 \(LeetCode\)](#)

和如上 77 和 216 区别：如上两题，是在一个集合里面求组合。该题是在不同的集合里面求组合。

思路：获取不同的集合，遍历的时候实现遍历不同的集合的效果。

❖ 案例：组合总和

时间：2025.4.6, 30min

链接：[39. 组合总和 - 力扣 \(LeetCode\)](#)

思路：回溯算法基本模板。

❖ 案例：全排列

时间：2026.9.16 20min 一次性过；

链接：[46. 全排列 - 力扣 \(LeetCode\)](#)

描述：返回数组元素所有排列方式。

思路：递归加回溯，额外加一个数据结构用来去重，比如用数组。

❖ 案例：子集

时间：2026.9.17 30min 卡住了，因为递归参数写错了。

链接：[78. 子集 - 力扣 \(LeetCode\)](#)

思考启发：代码递归的模版套一下就行。可以带入一个样例，通过树形结构来可视化递归和回溯过程，以此来发现问题。

❖ 案例：括号生成

时间：2026.9.17 50min 卡死了。2026.9.18 20min，看了解析才做出来，没有彻底理解。

链接：[22. 括号生成 - 力扣 \(LeetCode\)](#)

原来的思考：将括号先排成一个字符串，然后进行递归回溯，时间复杂度会提高，而且还会产生重复解，不好去重。

❖ 单词搜索

时间：2026.9.18 25min 刷不出来，不像动态规划那样好刷。

链接：[79. 单词搜索 - 力扣 \(LeetCode\)](#)

原来的思考：深度优先搜索，但是技艺生疏了，写不出来了，避免浪费太多时间，看了解析。接下来需要把卡尔的代码随想录再刷几题，感觉回溯这个章节掌握度太低了。

4.4 贪心算法

时间：2024.10.17

链接：[贪心算法理论基础！哔哩哔哩 bilibili](#)

定义：通过局部最优推导出全局最优。一般没有理论套路，全靠自己归纳总结。

理解 2026.9.9：去思考什么事局部最优，局部最优有没有什么办法推出全局最优，大致找一个递推式子，然后试试。

● 案例：加油站。

时间：2024.10.29, 60min

链接：[134. 加油站 - 力扣 \(LeetCode\)](#)

思路：单次遍历，如果油箱为负数说明从该节点出发不行，切换出发节点。当判断当前节点不能作为出发点时，当前节点以前的节点必然也不能作为出发点。

关键代码：

```
int canCompleteCircuit(vector<int>& gas, vector<int>& cost) {
    int totalGas = 0, totalCost = 0, start = 0, tank = 0;
    int n = gas.size();
    for (int i = 0; i < n; i++) {
        totalGas += gas[i];
        totalCost += cost[i];
        tank += gas[i] - cost[i];
        // 如果无法从当前起点继续，则重新选择下一个加油站作为起点
        if (tank < 0) {
            start = i + 1; // 从下一个加油站重新开始
        }
    }
    return totalGas >= totalCost ? start : -1;
}
```

```

        tank = 0;        // 重置当前油箱为 0
    }
}
// 检查总油量是否能覆盖总消耗量
return (totalGas >= totalCost) ? start : -1;
}

```

● 案例：分发糖果

时间：2024.10.29，50min+10min=60min

链接：[135. 分发糖果 - 力扣 \(LeetCode\)](#)

思路 1：初始化 `candy(n,1)`，从前往后遍历（自己想的，有些超时）

如果当前评分 `ratings[i]` 小于下一个评分 `ratings[i+1]`，那么 `ratings[i+1]` 基于 `ratings[i]` 增加 1。

如果当前评分 `ratings[i]` 大于下一个评分 `ratings[i+1]`，那么 `ratings[i]` 自增 1。然后循环从 `i` 开始往后遍历更新 `candy`。令 `j=i`；

如果 `ratings[j]` 小于 `ratings[j-1]`，因为 `ratings[i]` 自增 1，那么有可能打破条件，所以需要让 `candy[j-1]` 也自增 1。如果 `ratings[j]` 大于 `ratings[j-1]` 那么 `ratings[i-1]` 自增不会打破平衡。

关键代码：

```

for(int i=0;i<n;i++){
    //往前看
    if(i+1<n&&ratings[i]<ratings[i+1]&&candy[i]>=candy[i+1]){
        candy[i+1]=candy[i]+1;
    }else if(i+1<n&&ratings[i]>ratings[i+1]&&candy[i]<=candy[i+1]){
        candy[i]++;
        int j=i;
        while(j-1>=0&&ratings[j]<ratings[j-1]&&candy[j]>=candy[j-1]){
            candy[j-1]++;
            j--;
        }
    }
}
}

```

思路 2：先从前往后遍历，再从后往前遍历。

从前往后遍历，满足 `ratings[i]>ratings[i-1]` 时，`candy[i]>candy[i-1]`。

从后往前遍历，满足 `ratings[i]>ratings[i+1]` 时，`candy[i]<candy[i+1]`。

关键代码：

```

// 从前向后
for (int i = 1; i < ratings.size(); i++) {
    if (ratings[i] > ratings[i - 1]) candyVec[i] = candyVec[i - 1] + 1;
}
// 从后向前

```

```
for (int i = ratings.size() - 2; i >= 0; i--) {  
    if (ratings[i] > ratings[i + 1]) {  
        candyVec[i] = max(candyVec[i], candyVec[i + 1] + 1);  
    }  
}  
}
```

● 案例：根据身高重建队列。

时间：2024.10.30，50min

链接：[406. 根据身高重建队列 - 力扣 \(LeetCode\)](#)

题目描述：有一群人原本有顺序，有身高属性和前面多少人属性。现在打乱顺序，要求还原。

关键思路：两个维度的变量都在变化，不好同时抓，那么先抓一个。

先按照身高从大到小排序，如果身高相同，那么前序人数少的在前面。

最后遍历容器，按照前序人数将元素往前对应位置插入。

代码：

描述：cmp 根据身高从大到小排序，如果身高相等，那么前面人少的排前面。

细节：要新建一个容器进行插入，不能在原容器上插入。因为原容器插入会导致容器容量改变，因为没有删除。

```
sort(people.begin(), people.end(), cmp);  
vector<vector<int>> > queue;  
for(int i=0; i<people.size(); i++){  
    int pos=people[i][1];  
    queue.insert(queue.begin()+pos, people[i]);  
}
```

● 案例：最少箭引爆气球。

时间：2024.10.31，50min

链接：[452. 用最少数量的箭引爆气球 - 力扣 \(LeetCode\)](#)

思路：先按照左边界从小到大排序。遍历。

如果 i 的 left 小于等于 i-1 的 right，说明重叠。如果重叠则说明可以一箭引爆。将 i 的右边界更新一下，更新为 i 和 i-1 的 left 的最小值。下一次遍历的时候，如果满足重叠条件，则这个时候就是多重叠。如果不满足重叠条件，那么就意味着，不管上一个区间如何，都需要额外的箭才行。

关键代码：

提前根据左边界排序的。

```
int cnt=1;  
for(int i=1; i<points.size(); i++){  
    if(points[i][0]<=points[i-1][1]){  
        points[i][1]=min(points[i][1], points[i-1][1]);  
    }else{  
        cnt++;  
    }  
}
```

变式：重叠区间。

时间：2024.11.1, 100min

链接：[435. 无重叠区间 - 力扣 \(LeetCode\)](#)

关键思路：跟上问类似。需要提前按照左边界排序。

```
int cnt=0;
for(int i=1;i<intervals.size();i++){
    if(intervals[i][0]<intervals[i-1][1]){
        cnt++;
        intervals[i][1]=min(intervals[i][1],intervals[i-1][1]);
    }
}
```

小结：对于区间类型的题目，通常都需要按照区间左边界进行排序。

案例：单调递增数字。

时间：2024.11.2, 40min

链接：[738. 单调递增的数字 - 力扣 \(LeetCode\)](#)

关键思路：从后往前遍历。定义一个 flag 用来记录。

如果不满足后者大于等于前者，那么前者减一。

库函数：to_string()将 int 转换为 string,stoi()将 string 转换为 int

```
int monotoneIncreasingDigits(int n) {
    string s=to_string(n);
    int flag=s.size();//定义一个 flag 用来记录从什么时候开始全部变为 9
    for(int i=s.size()-1;i>0;i--){
        if(s[i]<s[i-1]){
            s[i-1]--;//ascii 码的计算
            flag=i;
        }
    }
    for(int i=flag;i<s.size();i++){//将 flag 初始化为 s.size(),如果原数自己满足条件，则不需要调整。
        s[i]='9';
    }
    return stoi(s);
}
```

小结：遍历顺序也是一个结题入手的维度。这里从后往前遍历。

案例：监控二叉树。

时间：2024.11.3, 60min

链接：[968. 监控二叉树 - 力扣 \(LeetCode\)](#)

关键思路：为每个节点设定状态，从叶子节点开始，后续遍历。判断三种情况，对于根节点的单独情况在窗口函数处处理。

关键代码：

//对于每个节点设定三种状态：0 无覆盖，1 有摄像头，2 有覆盖

//有四种情况需要考虑：1.当左右节点有覆盖时，该节点不需要安装摄像头，其父节点安装摄像头。

//如果 2.当左右节点任意一个节点为 0 时，该节点必须安装摄像头。

//如果 3.当左右节点任意一个节点为 1 时，该节点状态应该为有覆盖。

//如果 4.当根节点没有被覆盖时，需要单独覆盖。

//对于叶子节点，需要将空节点状态设定为有覆盖，这样满足设计理念。

```
int result=0;
class Solution {
    static int postOrder(TreeNode *root){
        if(root==nullptr) return 2;
        int left=postOrder(root->left);
        int right=postOrder(root->right);
        if(left==2&&right==2) return 0;
        if(left==0||right==0){//任有一个是 1，优先级高于任有一个是零。
            result++;
            return 1;
        }
        if(left==1||right==1){
            return 2;
        }
        return -1;//如上三种情况包含了所有逻辑，所以这部分不会走到，只是为了保证逻辑完整。
    }
public:
    int minCameraCover(TreeNode* root) {
        result=0;
        if(postOrder(root)==0){
            result++;
        }
        return result;
    }
};
```

❖ 案例：装满石头的背包的最大数量

时间：2026.9.6 10min

链接：[2279. 装满石头的背包的最大数量 - 力扣（LeetCode）](#)

思路启发：要尽可能多的填满背包，石头最好放到具备哪些特征的背包里面。

答：优先放到最快放满的背包里面。

❖ 案例：买卖股票的最佳时机

时间：2026.9.9 20min 一次过。

链接：动态规划章节有一样的。[121. 买卖股票的最佳时机 - 力扣（LeetCode）](#)

思路启发：把价值数组写出来，手动画一遍遍历过程，就能归纳出规律。

本次思路：记录一个最小值，这样遍历的时候，就能计算出每次卖出可能的最大值，在这些局部最大值中就可能产出全局最大值。

❖ 划分字母区间

时间：2026.9.10 50min 差一点点做出来，因为数据结构理解错误导致了一些 bug；

链接：[763. 划分字母区间 - 力扣 \(LeetCode\)](#)

思路 1（自己最开始想的）：按照合并重复区间的思想，先遍历字符串 s 计算出每个单词的区间，该区间默认已经按照左边界排序了，然后再遍历区间进行重叠区间合并即可。

卡点：unordered_map() 是指的没有顺序，不是指的按照插入顺序，所以需要单独加一个变量用来存储字符出现的顺序。map 数据结构会按照 key 值进行升序排序。

思路 2（豆包解析）：对于一个区间字符串，字符串中每一个字符的右边界一定是小于等于区间字符串的右边界的。所以，先遍历字符串，记录一遍所有字符串的右边界。然后再遍历一次，依托左边界来计算区间长度，区间右边界基于遍历字符串的右边界实时更新。

思路 2 代码：

```
vector<int> partitionLabels(string s) {
    vector<int> last(26);
    // 记录每个字母最后出现位置
    for(int i = 0; i < s.size(); i++){
        last[s[i]-'a'] = i;
    }
    vector<int> ans;
    int left = 0;
    int right = 0;
    for(int i = 0; i < s.size(); i++){
        right = max(right, last[s[i]-'a']);
        if(i == right){
            ans.push_back(right - left + 1);
            left = i + 1;
        }
    }
    return ans;
}
```

4.5 动态规划

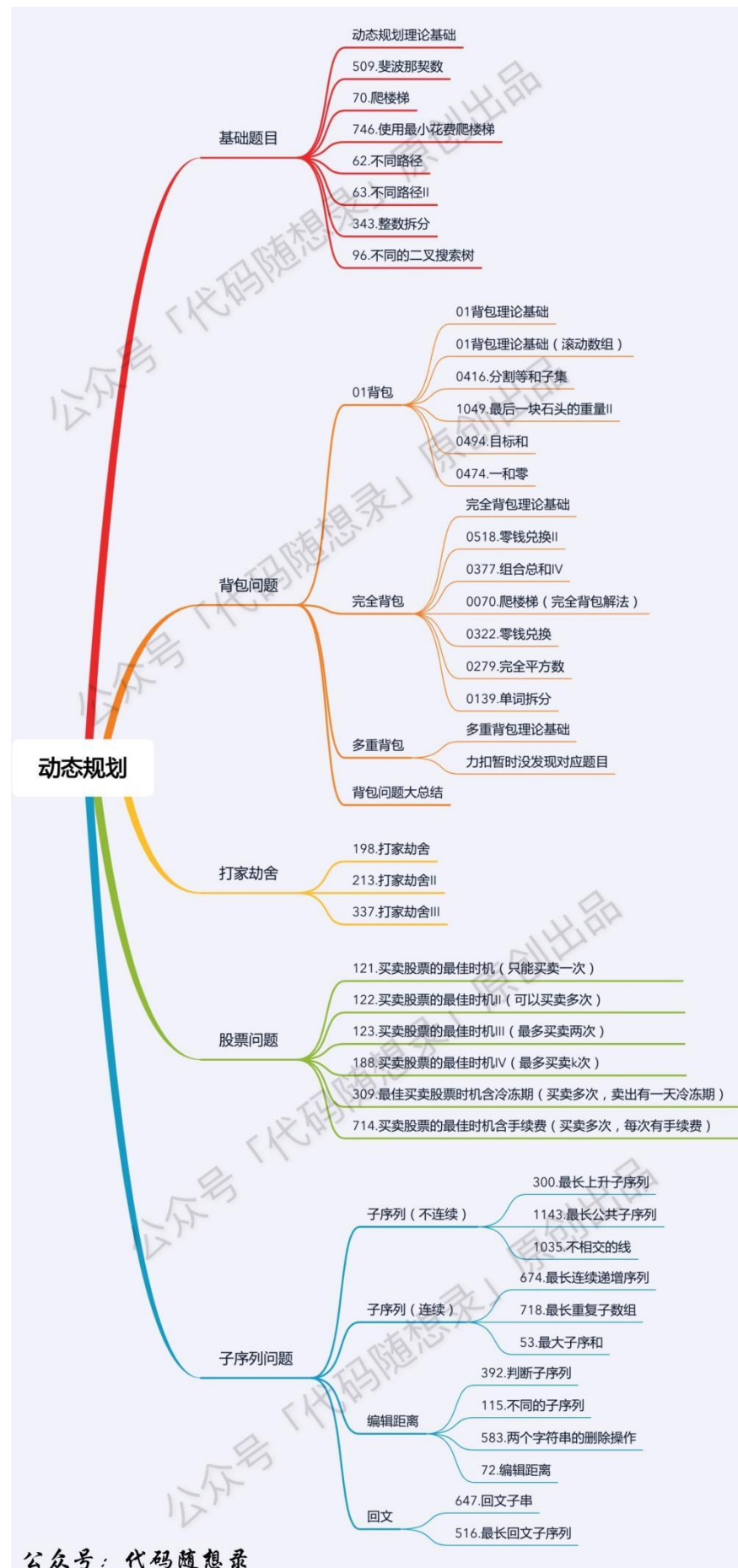
基础理论：动态规划五部曲。

时间：2024.9.4；2026.9.2 二刷；

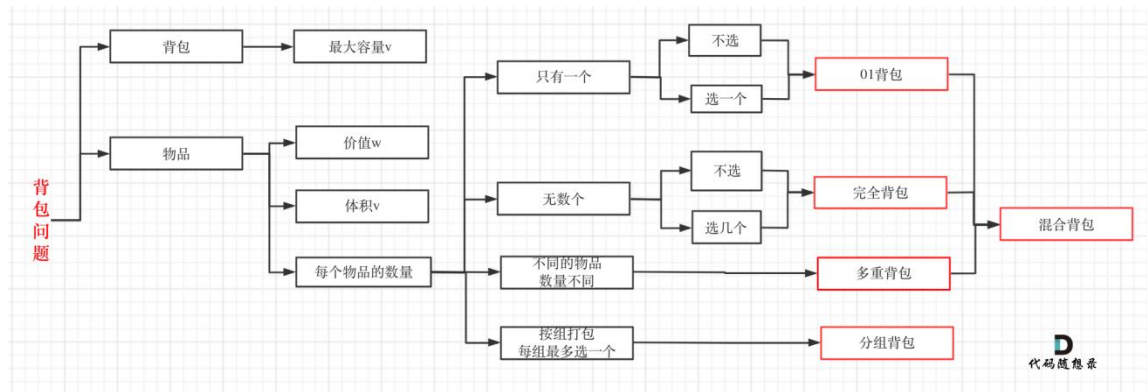
链接：[从此再也不怕动态规划了，动态规划解题方法论大曝光 ！| 理论基础 | 力扣刷题总结 | 动态规划入门_哔哩哔哩_bilibili](#)

动态规划五部曲：不仅仅是递推公式。

1. 动态规划数组以及其下标的含义。
2. 递推公式。
3. DP 数组初始化。零下标，非零下标。
4. 理解 DP 数组遍历顺序。
5. 打印动态数组。



4.5.1 背包问题



明确几个常见面试题目：

1. 二维 dp 数组是否可以颠倒顺序。可以，左上角矩阵推出。
2. 一维实现是否可颠倒顺序。不可，颠倒之后每次循环物品不变。
 - (1) 从后往前遍历。沿用上一次循环内容，01 背包问题。
 - (2) 从前往后遍历。完全背包问题，沿用上一次修改内容。

一、01 背包问题

01 背包问题经典五类题目：

1. 基础 01 背包，给定容量，最多能装多少价值。
2. 给定容量和价值，能不能装满这么多价值。
3. 给定容量，最多能装多少。（和 1 是一样的）
4. 给定容量和价值，最多能有多少种装的方法。
5. 给定容量和价值，装满情况下能放最多多少物品。

2026.9.5 心得：针对背包问题，每个步骤一定要有理有据。特别是针对初始化，遍历顺序两个关键点容易搞得很模糊。

【01 背包问题基础】

时间：2026.9.2

链接：[带你学透 0-1 背包问题！ | 关于背包问题，你不清楚的地方，这里都讲了！ | 动态规划经典问题 | 数据结构与算法_哔哩哔哩_bilibili](#)

基于动态规划五部曲理解：

1. 动态规划数组以及其下标的含义： $dp[i][j]$ 表示编号为 $0 \sim i$ 的物品任意取存，装满容量 j ，最大价值是多少。
2. 递推公式：第 i 个物品取与不取所得价值的最大值。 $dp[i][j] = \max(dp[i-1][j], dp[i-1][j - \text{weight}[i]] + \text{value}[i])$ ，其中 $dp[i-1][j]$ 表示不取物品 i 的最大价值， $dp[i-1][j - \text{weight}[i]] + \text{value}[i]$ 表示取物品 i 的最大价值。
3. DP 数组初始化：画一个二维矩阵，有理有据判定怎么初始化。
4. 理解 DP 数组遍历顺序：结合上面的二维矩阵，判定是否能够满足递推公式。

TODO 这里画一个示意图，结合如下这张图。

5. 打印动态数组。

已知	重量weight	价值value
物品0	1	15
物品1	3	20
物品2	4	30

dp[i][j] 表示第0~i个物品任意存取装满容量为j的背包所能获得的最大价值
vector<vector<int>> dp(nums.size()+1,vector<int>(target+1,0));

方案1：先遍历物品，再遍历容量，容量正序遍历

```
for(int i=1;i<=nums.size();i++){
    for(int j=0;j<=target;j++){
        if(j<weight[i-1]){
            dp[i][j]=dp[i-1][j];
        }else{
            dp[i][j]=max(dp[i-1][j],dp[i-1][j-weight[i-1]]+value[i-1]);
        }
    }
}
```



容量j	j=0	j=1	j=2	j=3	j=4
初始化 dp[0][j]	0	0	0	0	0
第一轮 dp[1][j]	0	15	15	15	15
第二轮 dp[2][j]	0	15	15	20	35
第三轮 dp[3][j]	0	15	15	20	35

方案2：先遍历容量，再遍历物品，容量正序遍历

```
for(int j=0;j<=target;j++){
    for(int i=1;i<=nums.size();i++){
        if(j<weight[i-1]){
            dp[i][j]=dp[i-1][j];
        }else{
            dp[i][j]=max(dp[i-1][j],dp[i-1][j-weight[i-1]]+value[i-1]);
        }
    }
}
```



容量j	j=0	j=1	j=2	j=3	j=4
初始化 dp[0][j]	0	0	0	0	0
第一轮 dp[1][j]	0	15	15	15	15
第二轮 dp[2][j]	0	15	15	20	35
第三轮 dp[3][j]	0	15	15	20	35

方案3：先遍历物品，再遍历容量，容量逆序遍历

```
for(int i=1;i<=nums.size();i++){
    for(int j=target;j>=0;j--){
        if(j<weight[i-1]){
            dp[i][j]=dp[i-1][j];
        }else{
            dp[i][j]=max(dp[i-1][j],dp[i-1][j-weight[i-1]]+value[i-1]);
        }
    }
}
```



容量j	j=0	j=1	j=2	j=3	j=4
初始化 dp[0][j]	0	0	0	0	0
第一轮 dp[1][j]	0	15	15	15	15
第二轮 dp[2][j]	0	15	15	20	35
第三轮 dp[3][j]	0	15	15	20	35

方案4：先遍历容量，再遍历物品，容量逆序遍历

```
for(int j=target;j>=0;j--){
    for(int i=1;i<=nums.size();i++){
        if(j<weight[i-1]){
            dp[i][j]=dp[i-1][j];
        }else{
            dp[i][j]=max(dp[i-1][j],dp[i-1][j-weight[i-1]]+value[i-1]);
        }
    }
}
```



容量j	j=0	j=1	j=2	j=3	j=4
初始化 dp[0][j]	0	0	0	0	0
第一轮 dp[1][j]	0	15	15	15	15
第二轮 dp[2][j]	0	15	15	20	20
第三轮 dp[3][j]	0	15	15	20	30

图 1 背包问题二维动态数组遍历顺序讨论

如图 1 所示，在背包问题二维动态数组下，这里讨论四个方案，针对物品 Vs 容量遍历顺利，容量顺序，一共 4 组方案。可以看到，在容量正序遍历的情况下，先遍历物品还是先遍历容量结果是一样的，因为更新二维动态矩阵的方向都是自左上往右下遍历的，先自左到右，还是先自上而下，不会影响最终结果。

如果先遍历容量再遍历物品，如果这个时候，容量是正序遍历，也正确，因为当遍历到物品 i 的轮次时，第 i-1 物品的轮次已经计算完成，依据递推公式，可以正常递推。但是如果这个时候容量逆序遍历，会错误，因为遍历到 j 时，需要依据 j 前面的容量数据递推，但是 j 前面的容量数据还没计算好，故而会错误。

这里是二维动态数组，遍历物品 i 轮次的时候，i-1 轮次的数据计算好了，依据递推公式，不会存在一个物品会在同一个物品轮次内重复计算的情况，所以一个物品只能选一次。

【01 背包问题之滚动数组】

时间：2026.9.3

链接：[带你学透 01 背包问题（滚动数组篇） | 从此对背包问题不再迷茫！_哔哩哔哩_bilibili](#)

基于动态规划五部曲理解：

1. 动态规划数组以及其下标的含义： $dp[j]$ 表示，装满容量为 j 的背包所能获得的最大价值。

2. 递推公式： $dp[j] = \max(dp[j], dp[j - \text{weight}[i]] + \text{value}[i])$ ，其中 $dp[j]$ 表示上一个轮次循环的最大价值。

3. DP 数组初始化。零下标，非零下标。针对零下标， $dp[0]$ 表示装满容量为 0 的背包所得价值，既然容量为 0，则取值为 0 即可。针对非零下标， $dp[j]$ 依据递推公式，由于递推公式中取 $\max$ ，所以 $dp[j]$ 初始化为一个非负的最小值，及初始化为 0。

4. 理解 DP 数组遍历顺序：第一个要点是，先遍历物品，再遍历背包。第二个要点是，后序遍历。

(1) 先遍历物品，再遍历容量。

(2) 后序遍历要点：如果正序遍历，依据递推公式，同一个物品会被重复使用；如果后序遍历，递归数组从后往前更新，当前轮次更新所得最大值一定是大于等于上一个轮次更新所得最大值的，依据递推公式，同一个物品不会多次参与更新，只会被使用一次。

5. 打印动态数组。

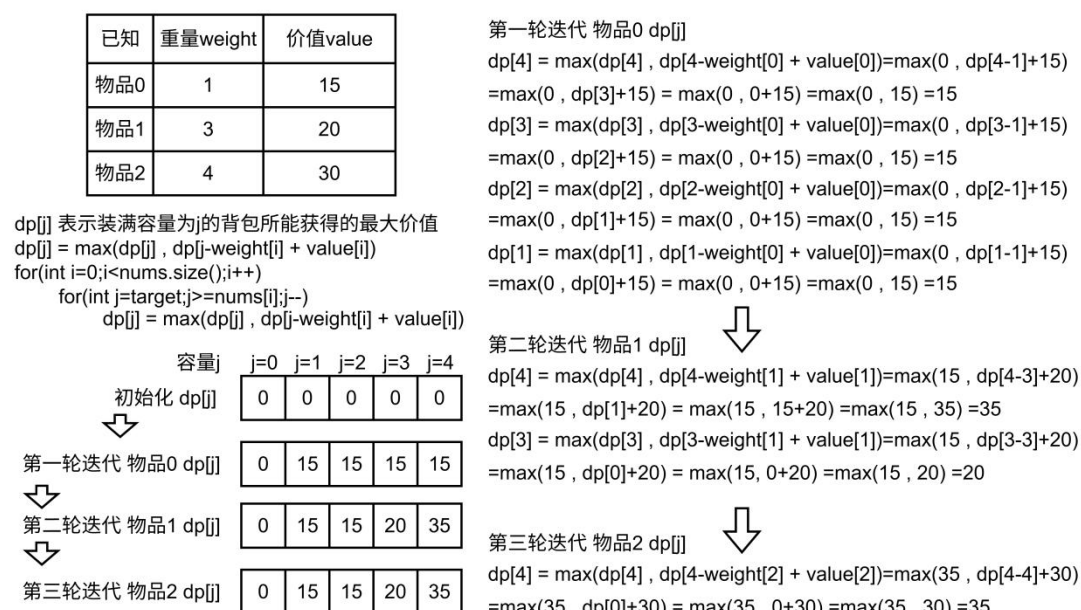
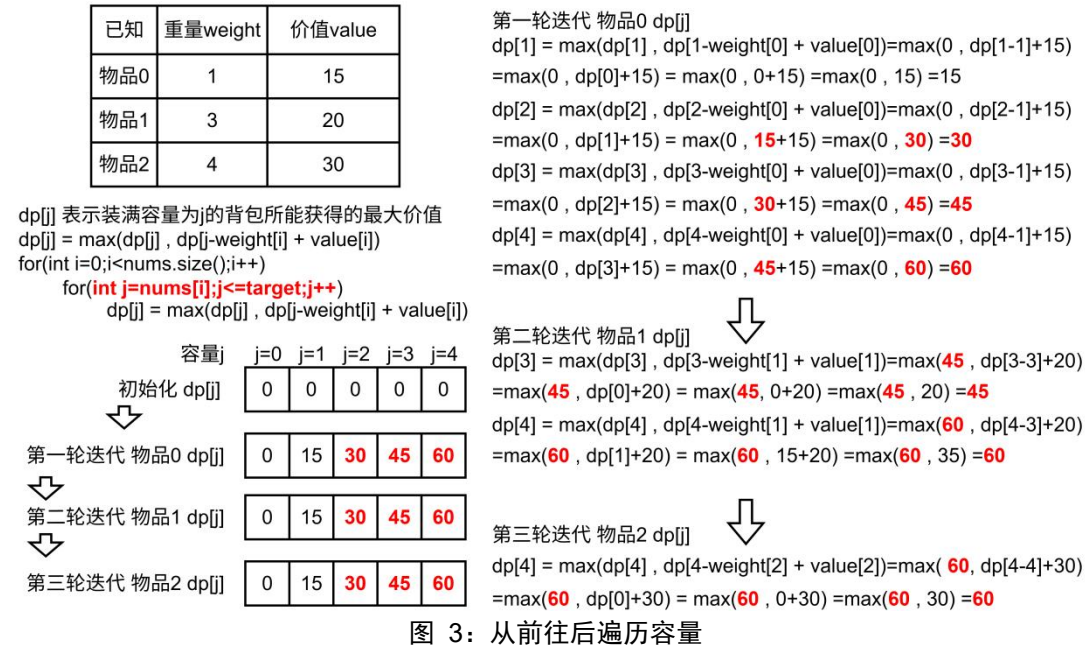


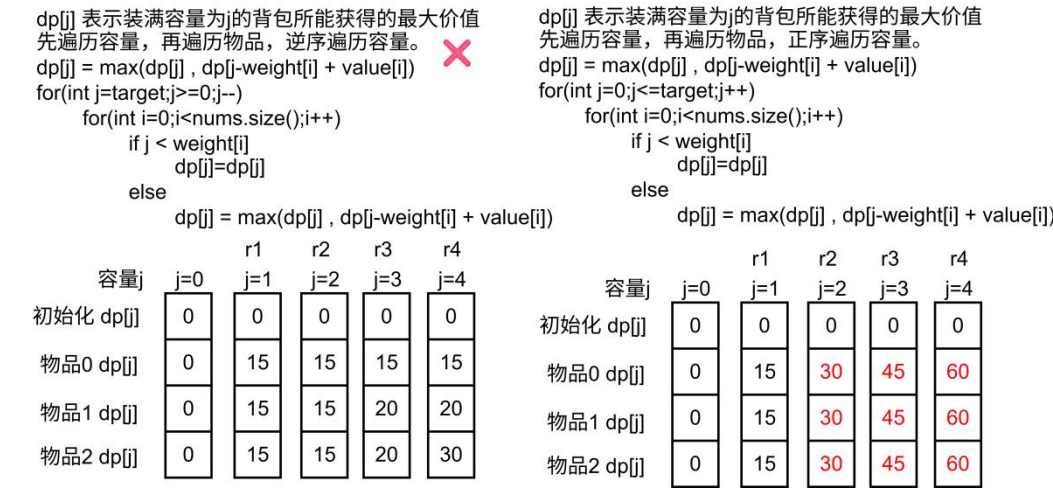
图 2：从后往前遍历容量

如图 2 所示，对于一维滚动数组，对于容量的遍历顺序是从后往前遍历，这样能够保证每个物品只使用一次。由递推公式可知，固定一个物品 i 的循环轮次之后，第 j 个容量的价值由 $j - \text{weight}[i]$ 推导得出，如果靠前的数值后更新，那么在更新靠后数值的时候，依托的是上一个物品 $i-1$ 轮次的价值，而 $i-1$ 轮次的最大价值一定是小于等于 i 轮次的最大价值的，所以对于物品 i ，在容量够的情况下，要么是使得价值增大的时候取，要么是不使得价值增大不取，不会被重复使用。但是，如果是从前往后遍历，由于在叠加更新，如果容量够，那

么该物品可能多次使得价值增大，可能存在重复取的可能。



如图 3 所示，对于容量的遍历改为从前往后的时候，可以保证对于同一个物品重复获取。由递推公式可知，固定一个物品 i 的循环轮次之后，第 j 个容量的价值由 j-weight[i] 推导得出，如果先更新了靠前的数值，那么后续更新会依托于前面的更新。假设有一个物品 A 小质量小价值，一个物品 B 大质量大价值，就有可能出现，多次取 A 获得的价值高于单次取 B 获得的价值，即如图所示，这个时候衍生为完全背包问题。



如图 4 所示，在滚动数组的背景下，先遍历容量再遍历物品，如果容量正序遍历，可能是完全背包问题（TODO 尚待验证），如果容量逆序遍历，则仍然会存在二维动态数组同样条件下的问题，容量 j 的数据需要依托 j 之前的数据，但是 j 之前的数据是没有的，所以错误。

❖ 案例：0-1 背包基础问题。给定容量，最多能装多少价值。

时间：2024.9.6，30min

链接：[46. 携带研究材料（第六期模拟笔试）\(kamacoder.com\)](#)

关键代码：

dp[i]表示容量为 i 时能容纳的价值。

```
vector<int> dp(N+1,0);//dp 数值表示价值，i 表示容量。顺便初始化
for(int i=1;i<=M;i++){
    for(int j=N;j>=space[i];j--){
        dp[j]=max(dp[j],dp[j-space[i]]+value[i]);
    }
}
cout<<dp[N]<<endl;
```

❖ 案例：划分等和子序列。给定容量和价值，能不能装满这么多价值。

时间：2024.9.7，30min；2026.8.26 跟健赫沟通，没做出来；2026.9.5 从最基础的背包问题入手，充分理解了基础，10min 做出来了。

链接：[416. 分割等和子集 - 力扣 \(LeetCode\)](#)

定位：01 背包问题最核心、最本质的代码。

关键代码：

思路：nums[i]既是容量，又是价值。

```
bool canPartition(vector<int>& nums) {
    int sum = accumulate(nums.begin(), nums.end(), 0);//使用库函数求和
    if(sum%2==1) return false;
    int target=sum/2;
    vector<int> dp(target+1,0);//初始化 0，dp 表示价值，i 表示容量
    for(int i=0;i<nums.size();i++){
        for(int j=target;j>=nums[i];j--){
            dp[j]=max(dp[j],dp[j-nums[i]]+nums[i]);
        }
    }
    return dp[target]==target?true:false;
}
```

❖ 案例：最后一块石头的重量。给定容量，最多能装多少。

时间：2024.9.7，30min；2026.9.6 10min，一次过，因为提前把背包问题基础梳理清楚了。

链接：[1049. 最后一块石头的重量 II - 力扣 \(LeetCode\)](#)

类比：分等和子序列。共同点，都是提前分好容量，尽可能填该容量。数组重量和数组价值是一样的。

核心思想：分成两堆重量差不多的石头。

```
int lastStoneWeightII(vector<int>& stones) {
```

```

int sum=accumulate(stones.begin(),stones.end(),0);
int target=sum/2;
vector<int> dp(target+1,0);
for(int i=0;i<stones.size();i++){
    for(int j=target;j>=stones[i];j--){
        dp[j]=max(dp[j],dp[j-stones[i]]+stones[i]);
    }
}
return sum-2*dp[target];
}

```

补充知识点: accumulate 函数用法, accumulate(stones.begin(),stones.end(),0);

❖ 案例: **目标和**。给定容量和价值, 最多能有多少种装的方法。

时间: 2024.9.8, 60min

链接: [494. 目标和 - 力扣 \(LeetCode\)](#)

思路启发: “+” 号和 “-” 号分为两堆, 运算使其目标为 target。由于其求和为 sum, 可以计算出需要+号的容量和需要减号的容量, 分别是:

```

left+right=sum;
left-right=target;

```

left 为加号, right 为减号。所以将其转换为 01 背包问题, 寻找价值使其能加起来达到 right(或者 left)。

关键代码:

递归函数: 容量为 j 时, dp[j]种数为所有除去 nums[i]后 dp[j-nums[i]]的和。

```

int findTargetSumWays(vector<int>& nums, int target) {
    int sum=accumulate(nums.begin(),nums.end(),0);
    if((target+sum)%2==1) return 0;
    if(target>sum||target<=-sum) return 0;
    int left=(target+sum)/2;
    int right=(sum-target)/2;
    vector<int> dp(right+1,0);//dp[i]表示不同组合方法, i 表示容量
    dp[0]=1;//初始化 dp[0]=1
    for(int i=0;i<nums.size();i++){//循环物品
        for(int j=right;j>=nums[i];j--){//循环容量
            dp[j]+=dp[j-nums[i]];//不同种数等于前面所有种数加起来
        }
    }
    return dp[right];
}

```

应用衍射: 通常计算种数的时候, 都是前面的加起来的。

❖ 案例: **一和零**。给定容量和价值, 装满情况下能放最多多少物品。

时间: 2024.9.8, 50min

链接: [474. 一和零 - 力扣 \(LeetCode\)](#)

关键代码：

说明：多维背包，重量有两个。

知识点：count（）函数

```
int findMaxForm(vector<string>& strs, int m, int n) {
    if(strs.empty()) return 0;
    vector<vector<int>> > dp(n+1,vector<int>(m+1,0)); //dp[j][k]表示最大值，j 表示 1 容量，k
    表示 0 容量
    //初始化 dp[0][0]=0;已初始化。

    for(int i=0;i<strs.size();i++){
        int x=count(strs[i].begin(),strs[i].end(),'1');
        int y=count(strs[i].begin(),strs[i].end(),'0');
        for(int j=n;j>=x;j--){
            for(int k=m;k>=y;k--){
                dp[j][k]=max(dp[j][k],dp[j-x][k-y]+1);
            }
        }
    }
    return dp[n][m];
}
```

二、完全背包问题

基础部分：完全背包中，物品可以使用无限次。代码实现上与 01 背包不同的是，完全背包从前往后遍历容量。

面试问题：遍历物品和遍历容量能否切换顺序。可以，每次迭代都是用递归数组前面的数据更新后面的，顺序不影响。

先遍历容量再遍历物品 Vs 先遍历物品再遍历容量：前者返回组合数，后者返回排列数。

时间：2024.9.8，30min

链接：[52. 携带研究材料（第七期模拟笔试）](#) (kamacoder.com)

关键代码 1：先容量再物品。

说明：[得到组合数](#)。

```
for(int j=0;j<=v;j++){ //遍历容量
    for(int i=0;i<=n;i++){ //遍历物品
        if(j<=w[i]) dp[j]=max(dp[j],dp[j-w[i]]+value[i]);
    }
}
```

使用案例：零钱兑换 2。

链接：[518. 零钱兑换 II - 力扣](#) ([LeetCode](https://leetcode.com))

递推公式： $dp[j] += dp[j - coins[i]]$;

关键代码 2：先物品再容量。

说明：[得到排列数](#)。

```
for(int i=1;i<=n;i++){ //循环物品
```

```

    for(int j=w[i];j<=v;j++){//循环重量
        dp[j]=max(dp[j],dp[j-w[i]]+value[i]);
    }
}

```

使用案例：组合总合 4

时间：2026.9.6 没做出来，没有想到要先容量再物品。2026.9.7 10min 昨天看了一遍，所以今天过得很快。

链接：[377. 组合总和 IV - 力扣 \(LeetCode\)](#)

递推公式：if(j>=nums[i]) dp[j]+=dp[j-nums[i]];

衍生使用案例：爬楼梯进阶版，每次至多可以爬 m 个台阶。

链接：[57. 爬楼梯（第八期模拟笔试） \(kamacoder.com\)](#)

✧ 衍生使用案例：零钱兑换 1，最少需要多少枚硬币达到要求。

时间：2026.8.4 二刷 15min。

链接：[322. 零钱兑换 - 力扣 \(LeetCode\)](#)

思路启发：动规五步曲。

关键代码：vector<int> dp(amount+1, amount+1); // 非零下标初始化为一个不可能的大数

❖ 案例：单词拆分

时间：2024.9.10, 60min; 2026.8.4 30min（有瑕疵）

链接：[139. 单词拆分 - 力扣 \(LeetCode\)](#)

思路启发：动规五步曲。字符串是背包，单词是物品。将单词放到背包里面，如果放入前是可以的，并且放入后也是可以的，则字符串到当前位置都是可以的。

关键代码：

知识点：子串 substr(pos,len)

```

bool wordBreak(string s, vector<string>& wordDict) {
    int n=s.size();
    vector<bool> dp(n+1,false);//dp[i]前 i 个字符串是否能由 wordDict 构成
    dp[0]=true;//初始化，第一个一定能
    for(int j=1;j<=n;j++){//求排列数，先遍历背包再遍历物品
        for(int i=0;i<wordDict.size();i++){
            //递推公式
            int wordsize=wordDict[i].size();
            if(j>=wordsize){
                string word=s.substr(j-wordsize,wordsize);
                if(word==wordDict[i]&&dp[j-wordsize]){//刨除这个单词后，如果之前序列
可行，则可行
                    dp[j]=true;
                    break;
                }
            }
        }
    }
    return dp[n];
}

```

}

❖ 案例：完全平方数

时间：2026.8.3 30min（看了解析）；

链接：[279. 完全平方数 - 力扣（LeetCode）](#)

思路启发：动规五部曲。

思路： $dp[i]$ 表示最少需要多少个数能够以平方和的形式构成数值 i 。

卡住原因：递推关系部分没有梳理清楚。

记录：2026.8.4 将其归类为完全背包问题。本质可以理解为，平方数是物品，被构造数是背包。

4.5.2 子序列问题

理解（精辟）：对子序列进行比较， $s[i-1]$ 与 $t[j-1]$ ，如果相等要进行什么操作，如果不相等又将采取什么操作。2024.10.15

一、递增子序列

描述：递增子序列，连续和不连续。

❖ 案例：最长递增子序列

链接：[300. 最长递增子序列 - 力扣（LeetCode）](#)

时间：2024.9.16, 30min；2026.3.31 20min（错在对于递推数组的理解），2026.8.10 40min（看了答案）。2026.9.6 10min。

说明：求最长上升子序列。序列可以不连续，顺序必须一致。

关键代码：

$dp[i]$ 数组含义： $dp[i]$ 表示包含第 i 个元素的最大递增子序列。

初始化： $dp[i]=1$ ，只有一个字符时，最大递增子序列都是自己，即 1。

```
int lengthOfLIS(vector<int>& nums) {  
    vector<int> dp(nums.size(),1); //  $dp[i]$ 表示包含第  $i$  个元素的最长公共子序列  
    // 初始化都为 1  
    int curMax=1;  
    for(int i=1;i<nums.size();i++){  
        for(int j=0;j<i;j++){  
            if(nums[j]<nums[i])  $dp[i]=\max(dp[j]+1,dp[i]);$   
        }  
        curMax=max(curMax,dp[i]);  
    }  
    return curMax;  
}
```

三刷：没有理解好初始化的步骤。

【衍生】最长递增子序列（序列要求连续）

时间：2024.9.16 10min；2026.4.1 10min；2026.9.6 10min；

链接：[674. 最长连续递增序列 - 力扣 \(LeetCode\)](#)

说明：最长递增子序列要求连续。

关键代码：

描述：与上问不一样的地方在于， $dp[i]$ 由 $dp[i-1]$ 推出。上问中 $dp[i]$ 由 $dp[0\sim i-1]$ 推出。

```
for(int i=1;i<nums.size();i++){
    if(nums[i-1]<nums[i]) dp[i]=dp[i-1]+1;
    curMax=max(curMax,dp[i]);
}
```

二、最长公共子序列

描述：最长公共子序列，连续和不连续。

❖ 案例：[最长公共子序列（不连续的）](#)。

描述：两个字符串，求最长公共子序列。

时间：2024.9.16, 30min；2026.4.1 40min（卡主）；2026.9.8 20min 一次性过。

链接：[最长公共子序列\(二\)_牛客题霸_牛客网 \(nowcoder.com\)](#)

思路启发 2026.9.8：涉及到字符串、数组，肯定要和便利挂钩，这里有两个字符串，大概率是二维动态数组；想到这一步了，只需要再想想，状态 (i,j) 是由什么状态迁移过来的，想明白了，就知道初始化要怎么初始化、脚标要怎么设计了。

算法思想：分别遍历两个数组 i 、 j ，如果元素相等，那么最长公共子序列加 1，如果元素不相等，则继承 $i-1$ 和 $j-1$ 当中较大者的 dp 值。回溯的时候，如果两个元素相等则输出，如果不相等则往 $i-1$ 和 $j-1$ 对应 dp 值较小的方向回溯。

思路：动态规划；回溯。

动态规划：

两字符串 i 和 j 如果相等，那么动态规划数组 $dp[i][j]$ 加 1

如果不相等，那么 $dp[i][j]$ 在 $dp[i-1][j]$ 和 $dp[i][j-1]$ 里面选一个大的继承

回溯：

两字符串 i 和 j 如果相等，输出

如果不相等，那么往 dp 大的方向回溯

细节：数组索引从 0 开始，所以两字符串 i 和 j 表示为 $s1[i-1]$ 和 $s2[j-1]$

关键代码：

动态规划部分： $dp[i][j]$ 表示前 $(i-1,j-1)$ 中最长公共子序列。

`vector<vector<int>> dp(n+1,vector<int>(m+1,0));`

//开始动态规划

```
for(int i=1;i<=n;i++){
```

```
    for(int j=1;j<=m;j++){
```

```
        if(s1[i-1]==s2[j-1]){//如果 s1 第 i 元素和 s2 第 j 个元素比较，相等，那么 dp 数组加一
```

```

        dp[i][j]=dp[i-1][j-1]+1;
    }else{//如果不相等，那么选择大的一个进行继承
        dp[i][j]=max(dp[i-1][j],dp[i][j-1]);
    }
}
}
}

```

回溯部分：如果不相等，则往大的方向回溯。

```

int i=n,j=m;
while(i>0&&j>0){
if(s1[i-1]==s2[j-1]){//如果字符相等，那么都后退
    lcs+=s1[i-1];
    i--;j--;
}else{
if(dp[i-1][j]>dp[i][j-1]){
    i--;
}else{
    j--;
}
}
}
}

```

【衍生】**最长公共子序列（连续的）**。

时间：2024.9.16，20min；2026.4.1 20min

描述：两个字符串，求最长公共子序列。

链接：[最长公共子串_牛客题霸_牛客网 \(nowcoder.com\)](https://www.nowcoder.com)

思路：动态规划

两字符串 i 和 j 如果相等，那么动态规划数组 $dp[i][j]$ 加 1

如果不相等则清零

分析和前面一个题目的区别：该题目子串要求是连续的，所以不连续了就要清零。

细节：设定了一个 `maxLength` 和 `endIndex` 用来记录最长子串的位置和长度。

心得 2026.4.1：可以用一个用例在草稿上运行一遍，很快就能归纳出规律。

【变式】不相交的线

时间：2026.4.1 5min

链接：[1035. 不相交的线 - 力扣 \(LeetCode\)](https://leetcode.cn/problems/1035.不相交的线)

描述：相等则连线，连线不相交，最多交多少。

思路：序列相对位置不变，有多长的最长公共子序列，即不连续的最长公共子序列。

心得（2026.4.1）：如果已知是最长公共子序列的变式，这道题目可以秒。

三、操作子序列

❖ 案例：子序列的个数。

时间：2024.10.14，40min

链接: [115. 不同的子序列 - 力扣 \(LeetCode\)](#)

思路: $dp[i][j]$ 表示 $s[0\sim i-1]$, 有多少种删除元素的方法, 使得删除后, 其与 $t[0\sim j-1]$ 相等。

$s[i-1]$ 与 $t[j-1]$ 进行比较, 有两种情况, 相等与不相等。(s 和 t 脚标从 0 开始)

若相等, 那么 $dp[i][j]$ 继承 $dp[i-1][j-1]$ 的方法数。考虑到 $s[i-2][t-1]$ 可能是相等的, $s[i-k][t-1]$ 也可能是相等的, 所以 $dp[i][j]$ 还需要继承 $dp[i-1][j]$ 的数量, 在继承 $dp[i-1][j]$ 的时候, 由递推公式可知, 默认已经继承了 $s[i-k][t-1]$ 的所有情景。即代码:

$dp[i][j]=dp[i-1][j-1]+dp[i-1][j]$

若不相等, 那么 $dp[i][j]$ 直接继承上一个 $s[i-2]$ 状态, 即 $dp[i-1][j]$ 状态。

$dp[i][j]=dp[i-1][j]$

❖ 案例: 删除子序列数量

时间: 2024.10.15, 10+10=20min

链接: [583. 两个字符串的删除操作 - 力扣 \(LeetCode\)](#)

描述: 只有删除操作。

思路: $dp[i][j]$ 表示对于以 $s[i-1], t[j-1]$ 结尾的字符串需要删除多少步数。 $s[i-1]$ 与 $t[j-1]$ 进行比较。初始化 $dp[0][j]=j; dp[i][0]=i$;

如果 $s[i-1]$ 与 $t[j-1]$ 相等, 则 $dp[i][j]$ 继承 $dp[i-1][j-1]$, 因为不需要删除。 $dp[i][j]=dp[i-1][j-1]$

如果 $s[i-1]$ 与 $t[j-1]$ 不相等, 则 $dp[i][j]$ 从所有可达的方法中选取最小的方法。可以通过删除 s 达到, 也可以通过删除 t 达到, 还可以通过删除 s 和 t 达到。即 $dp[i][j]=\min(dp[i-1][j]+1, dp[i][j-1]+1, dp[i-1][j-1]+2)$ 。

❖ 案例: 增删替子序列数量。

时间: 2024.10.15, 30+10=40min; 2026.9.8 35min 一次性解决, 在“替换”这个操作这里卡了一会儿。

链接: [72. 编辑距离 - 力扣 \(LeetCode\)](#)

描述: 两个序列, 可以增加、可以删除、也可以替换, 求最少多少步可使其相等。

思路启发: 和“最小公共子串”的启发很像, 需要去思考 (i, j) 的状态可以由哪些状态依据什么样的条件迁移过来, 然后自然而然就能想到, 比如 $(i-1, j)$ 和 $(i, j-1)$ 两个状态可以删除或者增加一个字符就能迁移到 (i, j) , $(i-1, j-1)$ 状态可以通过替换的方式迁移到 (i, j) 。

思路: 和上题“删除子序列数量”类似。初始化一样。

如果 $s[i-1]$ 与 $t[j-1]$ 相等, 继承 $dp[i-1][j-1]$ 的结果。 $dp[i][j]=dp[i-1][j-1]$

如果 $s[i-1]$ 与 $t[j-1]$ 不相等, 可以进行增删替。如果是删除, 那么情景和上文一样; 如果是增加, 那么其效果跟删除是一样的(采用归纳法可证明); 如果是替换, 那么只需进行一步操作即可使得 $s[i-1]$ 与 $t[j-1]$ 相等。所以 $dp[i][j]=\min(dp[i-1][j]+1, dp[i][j-1]+1, dp[i-1][j-1]+1)$ 。

四、回文子序列

说明: 区间 dp。 $dp[i][j]$ 表示区间 $s[i\sim j]$ 范围内最长子序列长度。子序列和子串的区别, 前者不要求连续。

时间: 2024.10.16, 50min

链接: [516. 最长回文子序列 - 力扣 \(LeetCode\)](#)

思路: 初始化 $d[i][i]=1$; 判断 $s[i]$ 与 $s[j]$ 是否相等。

如果 $s[i]=s[j]$, 那么 $dp[i][j]$ 从 $s[i+1\sim j-1]$ 中继承。

如果 $s[i]\neq s[j]$, 那么 $dp[i][j]$ 从 $s[i\sim j-1]$ 何 $s[i+1\sim j]$ 当中选取较大的继承。

关键代码:

遍历顺序: 从下到上, j 是区间右边, 默认大于区间左边的 i 。

```
for(int i=n-1;i>=0;i--){
    for(int j=i+1;j<n;j++){
        if(s[i]==s[j]){
            dp[i][j]=dp[i+1][j-1]+2;
        }else{//s[i]!=s[j]
            dp[i][j]=max(dp[i+1][j],dp[i][j-1]); //要么保留左边, 要么保留右边
        }
    }
}
```

五、其他

❖ 案例: 最大连续子数组和。

时间: 2024.10.13, 30min; 2026.4.15 20min (没有一次性做出来)

链接: [53. 最大子数组和 - 力扣 \(LeetCode\)](#)

思路: $dp[i]$ 表示前 i 个序列的最大连续子数组和。如果前 i 个小于 0, 说明是副作用, 舍弃掉。

❖ 案例: 乘积最大子序列

时间: 2026.8.10 40 min+看解析;

链接: [152. 乘积最大子数组 - 力扣 \(LeetCode\)](#)

思路: 关键卡点在于, 数组有正负的区分, 所以需要维护两个动态规划数组。如果 $nums[i]$ 是正数, 那么希望前 $i-1$ 个数构成的乘积尽可能大, 这样再乘上 $nums[i]$ 就会更大; 如果 $nums[i]$ 是负数, 那么希望前 $i-1$ 个数构成的乘积尽可能小, 这样乘上 $nums[i]$ 才会更大。

关键代码:

关键: 维护两个动态规划数组。一个维护前 $i-1$ 个数构成的最大乘积, 一个维护前 $i-1$ 个数构成的最小乘积。递推关系里面需要和三个数值进行比较更新, 分别是 $dp[0][i-1]*nums[i]$, $dp[1][i-1]*nums[i]$, $nums[i]$ 。

```
int maxProduct(vector<int>& nums) {
    int n=nums.size();
    if(n==1) return nums[0];
    vector<vector<int>> dp(2,vector<int>(n,0)); // dp[0][i] 表示包含第 i 个元素的非空连续子
    数组,正数, 乘积最大; dp[1][i]表示负数, 乘积最小。
    // 初始化极小值;第一个元素初始化为 nums[0], 对于 size==1 的数组特判。
    dp[0][0]=nums[0];
    dp[1][0]=nums[0];
    int max_pro=nums[0];
    for(int i=1;i<n;i++){
        dp[0][i]=max(max(dp[0][i-1]*nums[i],dp[1][i-1]*nums[i]),nums[i]);
        dp[1][i]=min(min(dp[0][i-1]*nums[i],dp[1][i-1]*nums[i]),nums[i]);

        max_pro = max(max_pro,dp[0][i]);
    }
```

```
}  
return max_pro;  
}
```

● 案例：可被 3 整除的最大和

时间：2026.3.27, 90min, 腾讯风控算法面试题目；

链接：[1262. 可被三整除的最大和 - 力扣 \(LeetCode\)](#)

思路：大致思路是针对余数是 0,1,2 分别维护一个动态数组

$dp[i][j]$ 表示余数是 i ，遍历到 $j-1$ 个数时的最大和，数组。

遍历数组更新 $dp[i][j]$ 。当遍历到第 j 个数时，需要遍历余数 0,1,2，基于第 j 个数更新 dp 数组， $dp[next_mode][j+1] = \max(dp[next_mode][j], dp[i][j] + nums[j])$ 。

关键代码：递推公式思路，遍历到该数的时候，判断加上该数和不加该数的区别。

```
int maxSumDivThree(vector<int>& nums) {  
    // 余数为 0,1,2 的数组，动态规划  
    int n=nums.size();  
    vector<vector<int>> dp(3, vector<int>(n+1, -inf));  
    dp[0][0]=0;  
    for(int i=0;i<n;i++){  
        for(int j=0;j<3;j++){  
            int next_mode=(nums[i]+j)%3;  
            dp[next_mode][i+1]=max(dp[next_mode][i], dp[j][i]+nums[i]); //更新  
        }  
    }  
    return dp[0][n];  
}
```

❖ 案例：最长有效括号。

时间：2026.9.6 没做出来，递推公式太难想。

链接：[32. 最长有效括号 - 力扣 \(LeetCode\)](#)

4.5.3 打家劫舍和股票问题

一、打家劫舍问题

描述：有一个序列表示每家金币，小偷去偷。满足条件，相邻屋子不能偷。

❖ 案例：打家劫舍 1

时间：2024.9.10, 30min; 2026.8.3 15min (once)

链接：[198. 打家劫舍 - 力扣 \(LeetCode\)](#)

思路启发：动规五步曲。

关键代码：

注意：dp[i]表示考虑第 i 家。包括第 i 家偷与不偷两种情况。

```
int rob(vector<int>& nums) {
    if(nums.empty()) return 0;//如果出现数据溢出，考虑特判 size==1、2
    if(nums.size()==1) return nums[0];
    if(nums.size()==2) return max(nums[0],nums[1]);
    vector<int> dp(nums.size()+2,0);//dp[i]表示考虑第 i 个房间的状态最大受益值
    dp[0]=nums[0];
    dp[1]=max(nums[0],nums[1]);
    for(int i=2;i<nums.size();i++){
        dp[i]=max(dp[i-1],dp[i-2]+nums[i]);//对比第 i 家偷与不偷
    }
    return dp[nums.size()-1];
}
```

衍生：打家劫舍 2

时间：2024.9.10, 30min

链接：[213. 打家劫舍 II - 力扣 \(LeetCode\)](#)

说明：第一户人和最后一户人相邻，构成一个环。

关键思路：假设去掉第一家，假设去掉最后家，假设同时去掉第一家和最后一家，一共三种情况分开讨论。对于去掉最后一家情况包含了同时去掉第一家和最后一家情况。通过去掉首尾的形式，将其转换为线性问题，再取最大值。

知识点：vector 可以用两个花括号临时补充。

```
if(root==nullptr) return vector<int>{0,0};
```

衍生：打家劫舍 3

时间：2024.9.10, 30min

链接：[337. 打家劫舍 III - 力扣 \(LeetCode\)](#)

说明：树状 dp 类型题目。

思路：后续遍历，对于偷不偷分两种情况判断。

如果当前节点偷，那么子节点都不能偷

```
root->val+dpLeft[0]+dpRight[0]
```

如果当前节点不偷，那么子节点可偷可不偷，选取偷与不偷最大值

```
max(dpLeft[0],dpLeft[1])+max(dpRight[0],dpRight[1])
```

关键代码：

知识点：定义可调用对象 function。

语法：`function<<返回参数类型>(输入参数类列表> 函数名字=[&](参数列表){函数体}`

```
int rob(TreeNode* root) {
    function<vector<int>(TreeNode*)> searchRob=[&](TreeNode *root){//后续遍历
        if(root==nullptr) return vector<int>{0,0};
        vector<int> dpLeft=searchRob(root->left);
        vector<int> dpRight=searchRob(root->right);
        int rob=root->val+dpLeft[0]+dpRight[0];//偷当前节点
    }
}
```

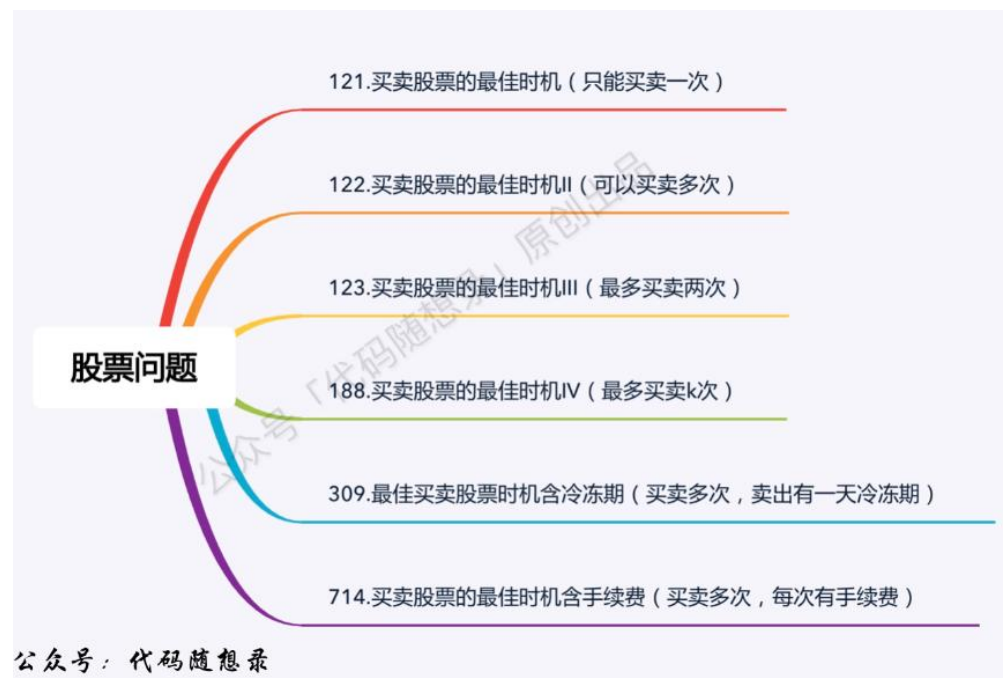
```

        int norob=max(dpLeft[0],dpLeft[1])+max(dpRight[0],dpRight[1]);//不偷当前节点
        return vector<int> {norob,rob};
    };
    //递归数组，dp[i],dp[0]不偷情况下最值，dp[1]偷情况下最值
    vector<int> result=searchRob(root);
    return max(result[0],result[1]);
}

```

二、股票问题

描述：一支股票，买入卖出，求得最高利润。



❖ 案例：股票问题 1

时间：2024.9.14 20min

链接：[121. 买卖股票的最佳时机 - 力扣 \(LeetCode\)](https://leetcode.cn/problems/best-time-to-buy-and-sell-stock/)

说明：这支股票只能买卖一次。

思路：将持有股票和不持有股票作为两个不同的状态分别进行动态规划。

如果第 i 天持有股票，那么第 $i-1$ 天可能持有股票，也可能不持有。不持有股票时通过买进股票使其成为持有状态，由于利润基于第 0 天的利润，即 $-prices[i]$ 。

如果第 i 天不持有股票，那么第 $i-1$ 天可能不持有股票，也可能持有股票。如果持有股票通过卖出股票达到不持有状态，即 $dp[i-1][0]+prices[i]$ 。

关键代码：

说明： $dp[i][0]$ 表示第 i 天持有股票的最大收益。 $dp[i][1]$ 表示第 i 天不持有股票的最大收益。

```

int maxProfit(vector<int>& prices) {
    vector<vector<int>> > dp(prices.size(),vector<int>(2,0));
    dp[0][0]=-prices[0];
    dp[0][1]=0;
}

```

```
for(int i=1;i<prices.size();i++){
    dp[i][0]=max(dp[i-1][0],-prices[i]); //第 i-1 天的股票
    dp[i][1]=max(dp[i-1][1],dp[i-1][0]+prices[i]);
}return dp[prices.size()-1][1];
}
```

✧ 衍生：股票问题 2

时间：2024.9.14 20min；2024.10.19，40min 二刷。

链接：[122. 买卖股票的最佳时机 II - 力扣 \(LeetCode\)](#)

说明：可以多次买卖股票。

关键代码：

思路：利润基于上一天不持有股票展开。

```
for(int i=1;i<prices.size();i++){
    dp[i][0]=max(dp[i-1][0],dp[i-1][1]-prices[i]); //第 i-1 天的股票
    dp[i][1]=max(dp[i-1][1],dp[i-1][0]+prices[i]);
}
```

✧ 衍生：股票问题 3

时间：2024.9.15 20min

链接：[123. 买卖股票的最佳时机 III - 力扣 \(LeetCode\)](#)

描述：至多可以买卖 2 次股票。

关键代码：

dp[i][0]表示不操作。dp[i][1]表示第 i 天第一次持有股票的最大收益。dp[i][2]表示第 i 天第一次不持有股票的最大收益。dp[i][3]表示第 i 天第二次持有股票的最大收益。dp[i][4]表示第 i 天第二次不持有股票的最大收益。

```
for(int i=1;i<prices.size();i++){
    dp[i][0]=dp[i-1][0];
    dp[i][1]=max(dp[i-1][1],dp[i-1][0]-prices[i]); //第 i-1 天的股票
    dp[i][2]=max(dp[i-1][2],dp[i-1][1]+prices[i]);
    dp[i][3]=max(dp[i-1][3],dp[i-1][2]-prices[i]); //第 i-1 天的股票
    dp[i][4]=max(dp[i-1][4],dp[i-1][3]+prices[i]);
}
```

✧ 衍生：股票问题 4

时间：2024.9.15 20min

链接：[188. 买卖股票的最佳时机 IV - 力扣 \(LeetCode\)](#)

描述：至多可以买卖 k 次。

关键代码：

从问题 3 衍生过来，将次数写成一次循环。

```
for(int j=0;j<2*k;j+=2){
    for(int i=1;i<prices.size();i++){
        dp[i][j+1]=max(dp[i-1][j+1],dp[i-1][j]-prices[i]);
        dp[i][j+2]=max(dp[i-1][j+2],dp[i-1][j+1]+prices[i]);
    }
}
```

```
}  
}
```

✧ 衍生：股票问题 5

时间：2024.9.15 20min

链接：[309. 买卖股票的最佳时机含冷冻期 - 力扣（LeetCode）](#)

关键思路：将冷冻期单独罗列出个状态。

关键代码：

动态数组：dp 表示当前利润，dp[i][0]持有股票，dp[i][1~3]不持有股票，dp[i][1]保持卖出状态，dp[i][2]立马卖出状态，dp[i][3]冷冻期。

```
dp[0][0]=-prices[0];  
dp[0][1]=0;dp[0][2]=0;dp[0][3]=0;//不持有股票的状态都是 0  
for(int i=1;i<prices.size();i++){  
    //从 dp[i-1][0],dp[i-1][1],dp[i-1][2]三个状态转移过来  
    dp[i][0]=max(dp[i-1][0],dp[i-1][1]-prices[i]);  
    dp[i][0]=max(dp[i][0],dp[i-1][3]-prices[i]);  
    dp[i][1]=max(dp[i-1][1],dp[i-1][3]);  
    dp[i][2]=dp[i-1][0]+prices[i];  
    dp[i][3]=dp[i-1][2];  
}  
int max_=max(dp[prices.size()-1][1],dp[prices.size()-1][2]);  
return max(max_,dp[prices.size()-1][3]);
```

4.5.4 其他案例

说明：尚未分类整理的。

❖ 案例：[整数拆分](#)。

时间：2024.9.4, 60min

链接：[343. 整数拆分 - 力扣（LeetCode）](#)

思路：动态规划。

dp[i]表示乘积，i 表示要拆的数。

初始化 dp[2]=1

递归函数 $dp[i]=\max(j*(i-j),j*dp[i-j],dp[i])$

循环有两层，外层循环 i，需要从 3 开始；内层循环拆分。

❖ 案例：[不同二叉搜索树](#)

时间：2024.9.5, 30min

链接：[96. 不同的二叉搜索树 - 力扣（LeetCode）](#)

关键代码：

思路启发：归纳法，按照已有条件找规律，推导出递推公式。

```
int numTrees(int n) {
```

```
vector<int> dp(n+1,0);//dp 数值表示种数，i 表示节点个数
dp[0]=1;
for(int i=1;i<=n;i++){
    for(int j=1;j<=i;j++){
        dp[i]+=dp[j-1]*dp[i-j];
    }
}
return dp[n];
}
```

❖ 案例：爬楼梯

时间：2026.8.3 5min（once）。

链接：[70. 爬楼梯 - 力扣（LeetCode）](#)

思路启发：动规五步曲。

❖ 案例：杨辉三角

时间：2026.8.3 20min（once）；

链接：[118. 杨辉三角 - 力扣（LeetCode）](#)

思路启发：动规五步曲。

❖ 不同路径

时间：2026.9.7 15min 因为已经刷了几天动态规划，之前又有点基础，所以这次过得很快；

链接：[62. 不同路径 - 力扣（LeetCode）](#)

提示：二维动态数组。

思路启发：去思考，在位置(i,j)的时候，是由哪些位置迁移过来的，然后就自然而然的想出解法了。

❖ 最小路径和

时间：2026.9.7 15min 刷了几天，有感觉了。

链接：[64. 最小路径和 - 力扣（LeetCode）](#)

思路启发：按照正常的思考链路，去想(i,j)位置是由哪几个位置转移过来的，然后自然就会了。

❖ 跳跃游戏

时间：2026.9.9 10min 一次过；

链接：[55. 跳跃游戏 - 力扣（LeetCode）](#)

标签：动态规划、贪心。

思路启发：想一下当前这一步能够到达哪些下一步，按照这个思路想一下，递推公式一下子就出来了。

✧ 衍生：跳跃游戏 II

时间：2026.9.9 10min 一次过。

链接：[45. 跳跃游戏 II - 力扣 \(LeetCode\)](#)

思路启发：和“跳跃游戏 1”是一样的，想一下当前位置能到哪儿，很自然的就能想到递推公式。

4.6 组合优化问题

组合优化问题，它涉及排列组合和子集选择：

4.7 排序算法

时间：2026.9.15

说明：这里主要放置排序算法。

排序算法	平均时间复杂度	最好情况	最坏情况	空间复杂度	排序方式	稳定性
冒泡排序	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	In-place	稳定
选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	In-place	不稳定
插入排序	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	In-place	稳定
希尔排序	$O(n \log n)$	$O(n \log^2 n)$	$O(n \log^2 n)$	$O(1)$	In-place	不稳定
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Out-place	稳定
快速排序	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	In-place	不稳定
堆排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	In-place	不稳定
计数排序	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(k)$	Out-place	稳定
桶排序	$O(n + k)$	$O(n + k)$	$O(n^2)$	$O(n + k)$	Out-place	稳定
基数排序	$O(n \times k)$	$O(n \times k)$	$O(n \times k)$	$O(n + k)$	Out-place	稳定

4.7.1 归并排序

大致思路：自顶向下排序，自顶向下拆分，拆到最小元素之后，再对最小元素进行逐层合并。特别适合链表的排序。

❖ 排序链表

时间：2026.9.14 40min 能做出来，但是运行超时了。2026.9.15 20min，昨天看了解析，今

天做的比较快。

链接: [148. 排序链表 - 力扣 \(LeetCode\)](#)

AI 题解: [总结链表排序思路](#)

核心知识点: 归并排序。

思路 1 (超时): 遍历 n 次链表, 每次挑选一个最大值, 将该最大值从链表里面剥离, 然后再使用头插法放到新的链表开头。

思路 2: 使用归并排序-自顶向上。

大致思路: (1) 划分链表。(2) 划分到足够小的时候排序。(3) 合并排序好的子链表。

关键词: 快慢指针找中点 + 归并排序的分治思想 + 合并两个有序链表 + dummy node

时间: $O(n \log n)$

空间: $O(\log n)$ 。虽然没有手动创建新的空间, 但是这里使用了递归, 递归会使用递归调用栈, 所以会额外造成空间消耗。

因为边界条件被卡了一下 2026.9.15: “while(slow->next&&fast&&fast->next)” 判定条件里面需要额外加一个 fast, 解决思路是, 带入了一个样例在稿纸上推理了一遍运行过程, 发现如果 fast 已经为空了, 那么 fast->next 就会报指针错误引用的错。

❖ 合并 K 个升序链表

时间: 2026.9.16 20min 一次过;

链接: [23. 合并 \$K\$ 个升序链表 - 力扣 \(LeetCode\)](#)

描述: 有 k 个升序链表, 要合并为 1 个。

思路: 自底向上的归并排序。两两合并, 最终合并为 1 个链表。

4.8 其他算法

1. 扫描线性法

案例: 主持人调度二

时间: 2024.7.15 30+40+50=120min

描述: 有很多活动线段, 重合的活动线段由不同主持人主持, 需要最多多少主持人。

链接: [主持人调度 \(二\) - 牛客题霸 - 牛客网 \(nowcoder.com\)](#)

方法一 (自己思考的): 类比了剪映思想, 把不同主持人分层不同的图层。

s1. 对活动按照左边界排序。

s2. 外循环遍历活动, 内循环遍历图层添加活动, 要是图层不够就加图层。

问题: 时间复杂度 $O(n^2)$, 只能过一部分, 运行超时。

方法二: 扫描线性法。

s1. 将活动左右边界拆成两类事件, 开始事件和结束事件。

s2. 将所有事件排序, 如果时间相同, 则结束事件排在开始事件前面 (不影响结束)。

s3. 遍历事件, 设立 currentHosts 和 maxHosts。

关键代码:

// 定义数据结构

```
struct event{
```

```
    int val;//事件数值
```

```

int type;//事件类型，-1 为结束事件，1 为开始事件
event(int val_,int type_):val(val_),type(type_){};
};

```

```

//查找最大值
int curHosts=0;
int mostHosts=0;
for(int i=0;i<events.size();i++){
    curHosts+=events[i].type;
    if(curHosts>mostHosts) mostHosts=curHosts;
}

```

线性扫描法通用性应用：常用于处理一维或二维区间问题的通用算法，它可以高效地解决一类涉及 **区间覆盖、重叠问题**，如会议室安排、天空线问题、最大重叠区间等。

案例：字段合并

描述：给定 n 个活动，每个活动安排的时间为 $[a_i, b_i)$ 。求最多可以选择多少个活动，满足选择的活动时间两两之间没有重合。

案例：[活动安排_牛客题霸_牛客网 \(nowcoder.com\)](http://www.nowcoder.com)

关键思路：

按照右界限排序，然后挨个放入时间线。

如果上一个区间左边界允许的话，就放入。

关键代码：

```

int curent=activities[0].second;
int cnt=1;
for(int i=0;i<activities.size()-1;i++){
    if(curent<=activities[i+1].first){
        curent=activities[i+1].second;
        cnt++;
    }
}

```

第 5 章 心得

作用：不太好归类到的部分

5.1 报错

❖ 头文件循环引用

```
#include"Card00.h"
```

```
#include"text.cpp"
```

理解：在成员函数实现部分包含主函数头文件就会出现 bug。

❖ 常识：闰年判断。

时间：2024.9.21，10min

链接：[打印日期_牛客题霸_牛客网 \(nowcoder.com\)](https://www.nowcoder.com)

能被 4 整除且能被 100 整除；或，能被 400 整除。